Checkpoint 2 - Redes Neurais Recorrentes

Prof. Orlando Junior

Nota Final: 10,0/10,0

Estudante 1:

- **Matrícula:** `564212`
- **Nome:** `João Henrique Arantes Vieira`

Estudante 2:

- **Matrícula:** `563388`
- **Nome:** `Anna Luiza Torra de Oliveira`

Estudante 3:

- **Matrícula:** `566381`
- **Nome:** `Thalita Soares Barbosa`

Instruções

1. Ao participar desta atividade, você concorda com o método de avaliação.
2. O estudante poderá solicitar a revisão da correção e da nota desta atividade apenas durante o dia agendado para vista da atividade e de maneira presencial, mediante justificativa fundamentada por questão.
3. A mera solicitação de revisão da correção sem justificativa será automaticamente indeferida. A "insistência exagerada" ou a solicitação de "pontos a mais" implicará na perda de 50% da nota total da atividade.
4. Esta avaliação pode ser realizada em grupo de até 3 pessoas, no máximo.
5. É vedada a comunicação entre os grupos, estando todos os estudantes sujeitos à anulação da atividade e à nota zero.
6. A interpretação dos enunciados faz parte da aferição de conhecimentos; não cabem, portanto, esclarecimentos.
7. Esta avaliação permite o uso de consulta a qualquer material, físico ou digital, inclusive IAs, desde que os prompts estejam no documento de entrega.
8. Todas as respostas deverão estar no campo apropriado.

9. Não é permitido sair de sala nos primeiros 30 minutos do horário da avaliação. Em caso de necessidade, a avaliação terá a nota zerada.
10. Após os 30 primeiros minutos do início da avaliação, o estudante que sair de sala estará necessariamente concluindo a sua avaliação.
11. Nenhum estudante poderá ingressar após os 30 primeiros minutos da avaliação.
12. A lista de frequência deve ser assinada pelo estudante antes de sair da sala.
13. Ao final da avaliação, renomeie este arquivo com o seguinte formato **Check2_Matrícula_Nome_Matrícula_Nome.ipynb** e submeta-o em formato PDF. Por exemplo: "Check2_2025012635_João_2025012637_Maria.pdf".

Informações sobre Pontuação e Correção

Critério	Descrição	Pontuação Máxima
Preparação de Dados	Limpeza de texto e tratamento de embeddings. Estratégia clara, criativa e funcional para a criação ou injeção de anomalias (spam/fake reviews) na base do IMDB, resolvendo a ausência de rótulos originais.	**3,0 pontos**
Arquitetura LSTM (Modelagem)	Implementação correta das camadas na rede. Definição adequada da função de perda e otimizador, com treinamento funcional e controle básico de *overfitting*.	**2,0 pontos**
Explicabilidade Visual (XAI)	Implementação técnica do extrator de importância de palavras. A visualização gerada (heatmap/cores) destaca com precisão os termos que mais influenciaram a predição de anomalia.	**3,0 pontos**
Validação e Análise de Erros	Avaliação das métricas do modelo e análise crítica cruzada: as palavras destacadas pelo XAI realmente fazem sentido lógico para um humano? Demonstração de casos de sucesso e de falha.	**2,0 pontos**
TOTAL		**10,0 pontos**

🎬 Detetives de Dados: Identificando e Explicando Anomalias em Avaliações de Filmes

Plataformas de avaliação como o IMDB sofrem constantemente com *fake reviews* (avaliações falsas), spam estruturado ou críticas geradas por bots. Identificar essas anomalias é importante para manter a credibilidade do sistema. No entanto, em sistemas de Inteligência Artificial modernos, não basta apenas dizer "Isto é Spam" — é preciso explicar o porquê.

O objetivo do seu grupo é construir um modelo baseado em Redes Neurais Recorrentes (LSTM) capaz de detectar anomalias, spam ou avaliações falsas na base de dados de reviews do IMDB. Além de classificar, o sistema deverá possuir uma camada de IA Explicável (XAI - Explainable AI), gerando uma visualização (em cores) que destaque quais palavras ou trechos do texto levaram o modelo a considerar aquela avaliação suspeita.

O Desafio: a base de dados tradicional do IMDB não possui rótulos de "Spam/Não Spam" (ela é focada em sentimento). Seu grupo terá total liberdade para definir o que é uma "anomalia" e como treinar o modelo para detectá-la.

> **O projeto não deve ser apenas um amontoado de código. Vocês deverão documentar e executar o projeto seguindo as seis fases do CRISP-DM:**

1. Entendimento do Negócio

- O que o seu grupo define como spam ou anomalia neste contexto?
- Qual o impacto de resolver este problema para uma plataforma de filmes?
- Definam claramente o problema exato que o modelo de vocês vai resolver.

2. Entendimento dos Dados

- Exploração da base do IMDB (distribuição de tamanho dos textos, vocabulário, etc.).
- Como vocês vão lidar com a falta de rótulos de spam?

3. Preparação dos Dados

- Limpeza de texto, tokenização, remoção de stopwords, padding.
- Transformação das palavras em números.
- Geração do dataset final que alimentará a rede.

4. Modelagem

- Construção da arquitetura LSTM.
- Dica: Vocês podem usar um Autoencoder LSTM (onde o erro de reconstrução alto indica spam), podem usar uma LSTM com Mecanismo de Atenção (Attention) ou um classificador tradicional acoplado a bibliotecas de explicabilidade como LIME ou SHAP.

5. Avaliação

- Como saber se o modelo é bom?
- Quais métricas fazem sentido (Precision, Recall, F1-Score, Curva ROC)?
- A explicação visual faz sentido para um humano?

6. Implantação

- Crie uma interface simples onde o usuário cola um texto, clica em "Analisar", recebe o veredito e vê o texto com as palavras destacadas em cores.

Fase 1 — Entendimento do Negócio (CRISP-DM)

Esta seção responde às três perguntas propostas pelo enunciado para a primeira fase do CRISP-DM: o que o grupo define como anomalia, qual o impacto de resolvê-la e qual é o problema exato que o modelo resolverá. Nenhuma implementação é realizada nesta etapa; trata-se exclusivamente da formalização do problema.

1.1 Contexto e problema de negócio

Plataformas de avaliação de filmes, como o IMDB, dependem de textos escritos por usuários para orientar a escolha de outros espectadores. Esse mecanismo é vulnerável a conteúdos que não refletem uma opinião genuína sobre a obra: propaganda disfarçada de resenha, texto repetitivo gerado em massa e avaliações escritas com o propósito explícito de inflar ou deprimir a reputação de um título.

O problema de negócio é **identificar, entre as avaliações textuais de uma plataforma de filmes, aquelas que apresentam padrões linguísticos suspeitos, e explicar ao analista quais palavras ou trechos motivaram a suspeita**. O sistema não pretende substituir a moderação humana, mas priorizar casos para revisão e tornar a decisão do modelo auditável.

1.2 Importância para uma plataforma de filmes

Detectar avaliações suspeitas é relevante pelos seguintes motivos:

- **Credibilidade da plataforma:** avaliações pouco confiáveis reduzem a confiança do público no sistema como um todo.
- **Qualidade da informação:** notas e resenhas influenciam decisões de consumo e a percepção pública das obras; conteúdo manipulado distorce essa referência.
- **Integridade de recomendações e rankings:** sistemas que usam avaliações como entrada herdam o viés introduzido por conteúdo artificial.
- **Eficiência da moderação:** a revisão manual de grandes volumes de texto é cara; um classificador pode ordenar os casos por suspeita e reduzir o esforço humano.
- **Transparência:** a explicação por palavra permite que o moderador valide ou conteste a decisão, o que reduz o risco de penalizar usuários legítimos por falsos positivos.

1.3 Objetivos

Objetivo geral. Desenvolver um classificador baseado em Rede Neural Recorrente (LSTM) capaz de identificar avaliações de filmes com padrões textuais anômalos e de explicar visualmente, em nível de palavra, os termos que mais contribuíram para essa classificação.

Objetivos específicos.

1. Definir de forma operacional o que o grupo considera uma anomalia textual neste contexto.
2. Construir, a partir da base do IMDB, um conjunto de dados rotulado em que parte das avaliações recebe anomalias sintéticas segundo padrões previamente definidos.
3. Treinar um classificador LSTM binário e controlar o *overfitting*.
4. Avaliar o modelo com métricas adequadas a classes possivelmente desbalanceadas (*precision*, *recall*, *F1-score* e curva ROC).
5. Gerar explicações por palavra com LIME e apresentá-las em uma visualização colorida.
6. Verificar criticamente se os termos destacados fazem sentido para um leitor humano, documentando casos de acerto e de falha.
7. Disponibilizar uma interface simples em Gradio para análise de textos inseridos pelo usuário.

1.4 Definição do problema de classificação

O problema é formulado como uma **classificação binária supervisionada de textos**:

Classe	Significado	Descrição
0	Avaliação presumidamente normal	Avaliação original do IMDB, sem alteração. É considerada normal por presunção: a base não oferece garantia de que seja livre de spam real.
1	Avaliação com anomalia sintética	Avaliação original modificada pelo grupo para conter um dos padrões anômalos definidos na seção 1.5.

O alvo do modelo é a **presença de um padrão anômalo**, e não a opinião do autor sobre o filme.

1.5 Padrões de anomalia sintética

Inicialmente, o grupo adota três padrões. Eles foram escolhidos por serem reconhecíveis em plataformas reais de avaliação e por poderem ser descritos de forma objetiva. A sua forma exata de geração será definida nas fases seguintes.

Padrão	Descrição	Exemplo ilustrativo
Spam promocional	Inserção de conteúdo publicitário ou de chamada à ação sem relação com o filme, como convites para acessar sites, ofertas ou canais de download.	"*... great movie, visit www.free-movies-now.example and get 50% off today!*"

Padrão	Descrição	Exemplo ilustrativo
Repetição excessiva	Reiteração anormal de palavras, expressões ou frases, característica de texto gerado em massa ou de preenchimento artificial.	*"best best best movie movie movie ever ever ever ..."*
Linguagem de manipulação de avaliações	Expressões que indicam a intenção de influenciar a nota ou a reputação do título, em vez de descrever uma experiência de assistir ao filme.	*"Give this film 10 stars, everyone must vote 10 to push its rating up."*

Os exemplos acima são apenas ilustrativos e não representam avaliações reais da base.

1.6 Sentimento do IMDB não é spam

A base do IMDB é classicamente rotulada por **sentimento** (positivo ou negativo). Esse rótulo mede a opinião do autor sobre o filme e **não** informa se a avaliação é spam, repetitiva ou manipulativa. Em particular:

- uma crítica fortemente negativa ou fortemente positiva pode ser perfeitamente legítima;
- uma avaliação anômala pode ser positiva ou negativa;
- portanto, os rótulos originais de sentimento **não serão usados como rótulos de anomalia**, e o modelo não deve ser interpretado como um analisador de sentimento.

Na construção do conjunto de dados, o grupo deverá garantir que a presença de anomalia não fique correlacionada com o sentimento original, para que o classificador não aprenda esse atalho.

1.7 Limitações da abordagem com anomalias sintéticas

- **Ausência de verdade de campo:** os rótulos da classe 1 são produzidos pelo próprio grupo. O desempenho medido indica a capacidade de reconhecer os padrões injetados, e não de detectar fraude real.
- **Rótulos da classe 0 são presumidos:** a base original pode conter spam real que será tratado como normal, o que introduz ruído nos rótulos.
- **Risco de aprendizado de artefatos:** se os padrões forem muito regulares, o modelo pode memorizar modelos de texto (*templates*) em vez de generalizar.
- **Cobertura restrita:** apenas três padrões são considerados. Spam real é mais variado e adaptativo, e o modelo pode falhar diante de estratégias não previstas.
- **Distribuição artificial das classes:** a proporção entre as classes no conjunto de treino é uma decisão do grupo e provavelmente difere da frequência de anomalias em produção.
- **Explicações aproximadas:** o LIME gera explicações locais por aproximação e pode apresentar instabilidade entre execuções.
- **Alcance das conclusões:** os resultados **não comprovam** a existência ou a detecção de fraude real; servem como prova de conceito de um pipeline de classificação e explicação.

1.8 Solução planejada

- **Classificação:** uma rede LSTM supervisionada, treinada para distinguir avaliações presumidamente normais (classe 0) de avaliações com anomalia sintética (classe 1).
- **Explicabilidade:** o LIME será aplicado às predições do modelo para atribuir importância a cada palavra, permitindo uma visualização com destaque em cores.
- **Demonstração:** uma interface em Gradio, em que o usuário insere um texto, solicita a análise e recebe o veredito junto ao texto com as palavras destacadas.

O desenvolvimento seguirá as seis fases do CRISP-DM, de forma incremental, nas seções subsequentes deste notebook.

1.9 Resumo das respostas da Fase 1

Pergunta do enunciado	Resposta do grupo
O que o grupo define como spam ou anomalia?	Avaliação que apresenta spam promocional, repetição excessiva ou linguagem de manipulação de avaliações. Neste projeto, a anomalia é sintética e definida operacionalmente pelo grupo (seções 1.4 e 1.5).
Qual o impacto de resolver o problema?	Maior credibilidade, melhor qualidade das avaliações, apoio eficiente à moderação e decisões auditáveis (seção 1.2).
Qual o problema exato que o modelo resolverá?	Classificação binária de textos: avaliação presumidamente normal (0) versus avaliação com anomalia sintética (1), com explicação por palavra (seções 1.3 e 1.4).

Fase 2 — Entendimento dos Dados (CRISP-DM)

Esta fase explora a base do IMDB para responder: qual é a estrutura dos dados, qual a qualidade dos textos, como se distribuem comprimento e vocabulário e o que isso implica para a LSTM e para a futura geração de anomalias sintéticas.

Escopo: análise exploratória apenas. Os textos originais **não são modificados**, não há geração de anomalias, tokenização definitiva, modelo, LIME ou interface. A tokenização usada abaixo serve somente para medir comprimento e vocabulário. Os rótulos de sentimento são tratados como um atributo da base e **não** como indicador de anomalia.

2.1 Fonte e carregamento

O enunciado se refere à "base de dados de reviews do IMDB", sem indicar uma fonte ou biblioteca específica. Foi adotado o **Large Movie Review Dataset v1.0** (Maas et al., 2011), distribuído por Stanford em `aclImdb_v1.tar.gz`, que é a origem original do corpus. Alternativas consideradas:

Alternativa	Motivo da não adoção
`keras.datasets.imdb`	Entrega sequências de inteiros, já em minúsculas e sem pontuação. O texto original se perde, o que inviabiliza a exibição de palavras destacadas e a preservação dos textos.
`datasets` (Hugging Face)	Exigiria instalar uma biblioteca adicional neste ambiente.

O arquivo é baixado com `keras.utils.get_file`, que guarda uma cópia em cache e evita novo download, e lido diretamente do `.tar.gz` com `tarfile`, sem extrair os 100 mil arquivos. São usadas apenas as partições rotuladas (`train` e `test`); a pasta `unsup` (avaliações sem rótulo) não é utilizada. O nome do arquivo de cada avaliação (`id_nota.txt`) fornece a nota original, de 1 a 10.

```
import hashlib
import os
import re
import tarfile
from collections import Counter

import keras
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS

plt.rcParams.update({"figure.figsize": (7, 4), "axes.grid": True, "grid.alpha": 0.3,
                     "axes.spines.top": False, "axes.spines.right": False})
pd.options.display.float_format = "{:,.2f}".format
```

```
URL = "https://ai.stanford.edu/~amaas/data/sentiment/aclImdb_v1.tar.gz"
ARQUIVO = keras.utils.get_file("aclImdb_v1.tar.gz", URL)

with open(ARQUIVO, "rb") as f:
    sha256 = hashlib.sha256(f.read()).hexdigest()
print(f"Fonte:   {URL}")
print(f"Arquivo: {ARQUIVO} ({os.path.getsize(ARQUIVO) / 1e6:.1f} MB)")
print(f"SHA-256: {sha256}")

registros = []
```

```
with tarfile.open(ARQUIVO, "r:gz") as tar:
    for membro in tar:
        partes = membro.name.split("/")
        if membro.isfile() and len(partes) == 4 and partes[1] in ("train", "test") and partes[2] in ("pos", "neg"):
            id_review, nota = partes[3][:-4].split("_")
            texto = tar.extractfile(membro).read().decode("utf-8")
            registros.append((partes[1], "positivo" if partes[2] == "pos" else "negativo", int(id_review), int(nota), texto))

df = pd.DataFrame(registros, columns=["particao", "sentimento", "id_review", "nota", "texto"])
HASH_TEXTOS = pd.util.hash_pandas_object(df["texto"], index=False).sum()  # para conferir, ao final, que os textos não mudaram
df.head(3)
```

```
Fonte:   https://ai.stanford.edu/~amaas/data/sentiment/aclImdb_v1.tar.gz
Arquivo: C:\Users\joaoh\.keras\datasets\aclImdb_v1.tar.gz (84.1 MB)
SHA-256: c40f74a18d3b61f90feba1e17730e0d38e8b97c05fde7008942e91923d1658fe
```

	particao	sentimento	id_review	nota	texto
0	test	negativo	127	3	I love sci-fi and am willing to put up with a ...
1	test	negativo	126	4	Worth the entertainment value of a rental, esp...
2	test	negativo	125	3	its a totally average film with a few semi-alr...

2.2 Análise estrutural

A seguir são examinados o tamanho da base, as colunas, os tipos, a distribuição dos sentimentos e a qualidade básica (ausentes, vazios e duplicados).

```
print(f"Registros: {len(df):,} | Colunas: {list(df.columns)}\n")
df.info()

print("\nReviews por partição e sentimento:")
display(df.groupby(["particao", "sentimento"]).size().unstack())
print("Reviews por nota original (escala de 1 a 10):")
display(df["nota"].value_counts().sort_index().to_frame("reviews").T)
```

```
Registros: 50,000 | Colunas: ['particao', 'sentimento', 'id_review', 'nota', 'texto']

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 5 columns):
 #   Column      Non-Null Count  Dtype 
---  ------      --------------  ----- 
 0   particao    50000 non-null  object
 1   sentimento  50000 non-null  object
 2   id_review   50000 non-null  int64 
 3   nota        50000 non-null  int64 
 4   texto       50000 non-null  object
dtypes: int64(2), object(3)
memory usage: 1.9+ MB

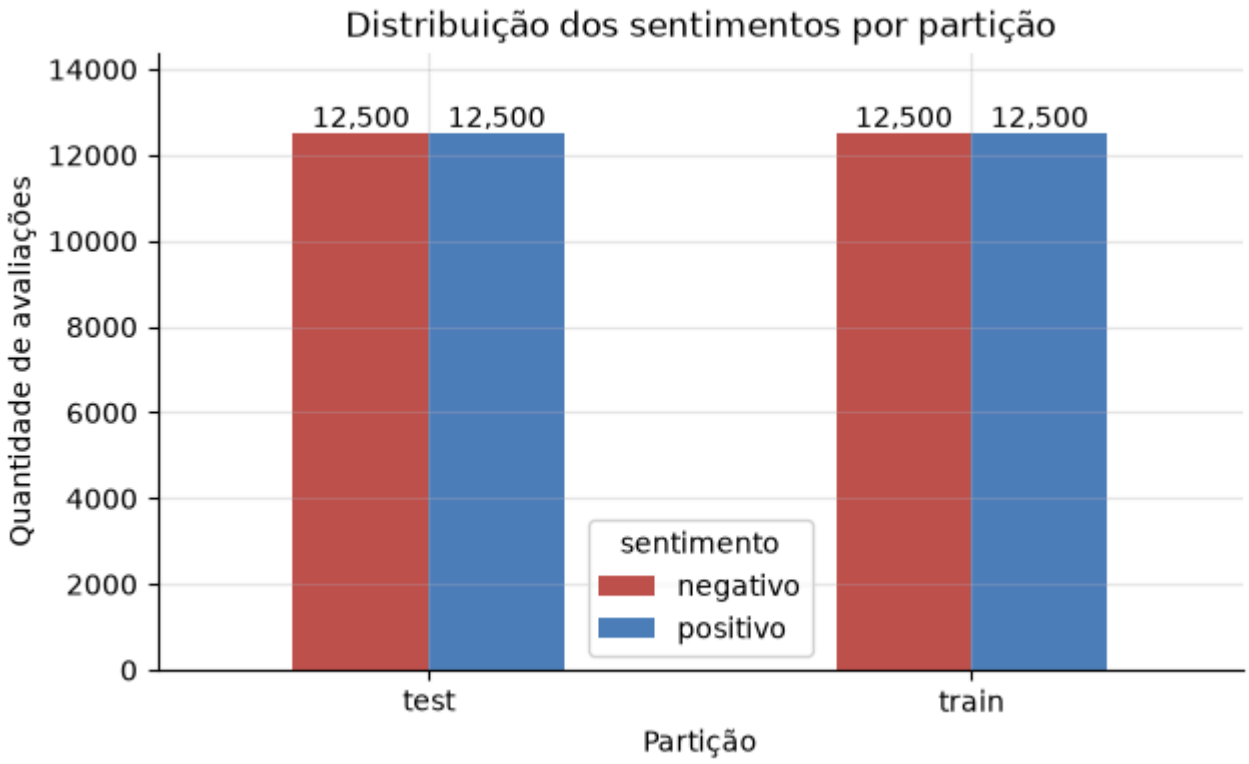
Reviews por partição e sentimento:
```

sentimento	negativo	positivo
particao		
test	12500	12500
train	12500	12500

Reviews por nota original (escala de 1 a 10):

nota	1	2	3	4	7	8	9	10
reviews	10122	4586	4961	5331	4803	5859	4607	9731

```
contagem = df.groupby(["particao", "sentimento"]).size().unstack()
ax = contagem.plot.bar(rot=0, color=["#c0504d", "#4f81bd"])
for barras in ax.containers:
    ax.bar_label(barras, fmt="{:,.0f}")
ax.set_title("Distribuição dos sentimentos por partição")
ax.set_xlabel("Partição")
ax.set_ylabel("Quantidade de avaliações")
ax.set_ylim(0, contagem.values.max() * 1.15)
plt.show()
```




```
duplicados = df[df.duplicated("texto", keep=False)].groupby("texto")
qualidade = pd.Series({
    "valores ausentes (todas as colunas)": df.isna().sum().sum(),
    "textos vazios ou só com espaços": (df["texto"].str.strip() == "").sum(),
    "textos duplicados (cópias excedentes)": df.duplicated("texto").sum(),
    "grupos de textos duplicados": duplicados.ngroups,
    "grupos duplicados entre treino e teste": (duplicados["particao"].nunique() > 1).sum(),
    "grupos duplicados com sentimentos distintos": (duplicados["sentimento"].nunique() > 1).sum(),
}, name="quantidade").astype(int)
qualidade.to_frame()
```

	quantidade
valores ausentes (todas as colunas)	0
textos vazios ou só com espaços	0
textos duplicados (cópias excedentes)	418
grupos de textos duplicados	406
grupos duplicados entre treino e teste	123
grupos duplicados com sentimentos distintos	0

Interpretação.

- **Fonte e tamanho:** 50.000 avaliações obtidas do arquivo `aclImdb_v1.tar.gz` (84,1 MB, SHA-256 registrado na saída da célula de carga), em cinco colunas: partição, sentimento, `id_review`, `nota` e `texto`.
- **Balanceamento:** 25.000 avaliações em cada partição e exatamente 12.500 por sentimento em cada uma. O desbalanceamento de classes da futura tarefa será, portanto, uma decisão de construção do dataset sintético, e não herdada da base.
- **Nota e sentimento:** o sentimento deriva da nota (1 a 4 negativo; 7 a 10 positivo; não existem notas 5 e 6). As notas extremas, 1 e 10, concentram cerca de 40% da base. Ambos são rótulos de **opinião**, não de spam.
- **Qualidade básica:** nenhum valor ausente e nenhum texto vazio. Há **418 cópias excedentes** de textos, em 406 grupos de duplicatas; em **123 grupos** a mesma avaliação aparece em treino e em teste, o que seria vazamento de dados se não tratado. Nenhum grupo duplicado tem sentimentos conflitantes.

2.3 Análise dos textos

O comprimento é medido em caracteres (sobre o texto original) e em palavras. Para contar palavras e montar o vocabulário é usada uma tokenização exploratória simples: minúsculas, remoção da etiqueta HTML `<br />` e

extração de sequências alfanuméricas, mantendo contrações como *don't*. Ela não substitui a tokenização definitiva da Fase 3.

```
TOKEN = re.compile(r"[a-z0-9]+(?:'[a-z]+)?")
BR = re.compile(r"<br\s*/?>")


def tokenizar(texto):
    """Tokenização apenas exploratória; a definitiva será feita na Fase 3."""
    return TOKEN.findall(BR.sub(" ", texto.lower()))


contagens = {chave: Counter() for chave in df.groupby(["particao", "sentimento"]).groups}
n_palavras, rep_consec = [], []
for particao, sentimento, texto in zip(df["particao"], df["sentimento"], df["texto"]):
    tokens = tokenizar(texto)
    n_palavras.append(len(tokens))
    rep_consec.append(sum(a == b for a, b in zip(tokens, tokens[1:])))
    contagens[(particao, sentimento)].update(tokens)

df["n_palavras"] = n_palavras
df["n_caracteres"] = df["texto"].str.len()
df["rep_consec"] = rep_consec  # pares de palavras idênticas consecutivas
```

```
percentis = [0.05, 0.25, 0.5, 0.75, 0.9, 0.95, 0.99]
print("Comprimento das avaliações (palavras e caracteres):")
display(df[["n_palavras", "n_caracteres"]].describe(percentiles=percentis).T)

print("Palavras por sentimento:")
display(df.groupby("sentimento")["n_palavras"].describe(percentiles=percentis))

print(f"Correlação palavras x caracteres: {df['n_palavras'].corr(df['n_caracteres']):.3f}\n")
cortes = [100, 128, 200, 256, 300, 400, 500, 600]
print("Fração das avaliações que cabem inteiras em L palavras:")
pd.Series({L: (df["n_palavras"] <= L).mean() for L in cortes}, name="fração").to_frame().T
```

```
Comprimento das avaliações (palavras e caracteres):
```

	count	mean	std	min	5%	25%	50%	75%	90%	95%	99%	max
n_palavras	50,000.00	231.00	170.90	6.00	63.00	127.00	173.00	280.00	451.00	588.05	902.01	2,473.00
n_caracteres	50,000.00	1,309.43	989.73	32.00	349.00	699.00	970.00	1,590.25	2,581.00	3,391.00	5,179.01	13,704.00

```
Palavras por sentimento:
```

	count	mean	std	min	5%	25%	50%	75%	90%	95%	99%	max
sentimento												
negativo	25,000.00	229.25	164.42	6.00	67.00	128.00	174.00	278.00	437.00	573.00	889.00	1,522.00
positivo	25,000.00	232.75	177.13	10.00	61.00	125.00	172.00	283.00	464.00	607.00	918.00	2,473.00

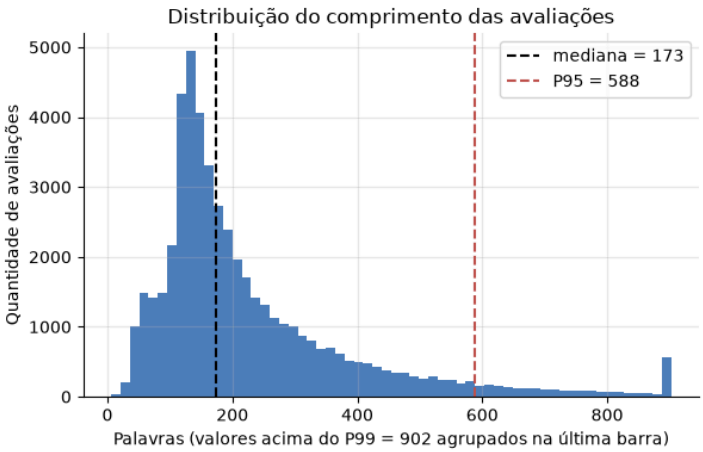
```
Correlação palavras x caracteres: 0.997

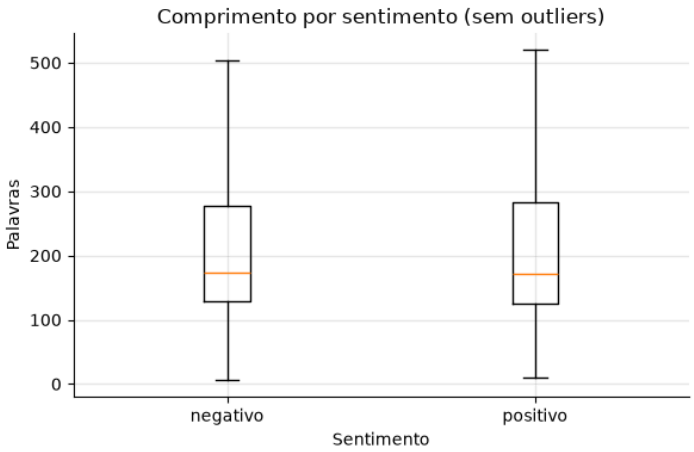
Fração das avaliações que cabem inteiras em L palavras:
```

	100	128	200	256	300	400	500	600
fração	0.12	0.26	0.59	0.71	0.78	0.87	0.92	0.95

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
p99 = df["n_palavras"].quantile(0.99)
ax1.hist(df["n_palavras"].clip(upper=p99), bins=60, color="#4f81bd")
ax1.axvline(df["n_palavras"].median(), color="k", linestyle="--", label=f"mediana = {df['n_palavras'].median():.0f}")
ax1.axvline(df["n_palavras"].quantile(0.95), color="#c0504d", linestyle="--", label=f"P95 = {df['n_palavras'].quantile(0.95):.0f}")
ax1.set_title("Distribuição do comprimento das avaliações")
ax1.set_xlabel(f"Palavras (valores acima do P99 = {p99:.0f} agrupados na última barra)")
ax1.set_ylabel("Quantidade de avaliações")
ax1.legend()

grupos = [df.loc[df["sentimento"] == s, "n_palavras"] for s in ("negativo", "positivo")]
ax2.boxplot(grupos, showfliers=False)
ax2.set_xticks([1, 2])
ax2.set_xticklabels(["negativo", "positivo"])
ax2.set_title("Comprimento por sentimento (sem outliers)")
ax2.set_xlabel("Sentimento")
ax2.set_ylabel("Palavras")
plt.tight_layout()
plt.show()
```





Interpretação do comprimento.

- A distribuição é **assimétrica à direita**: mediana de 173 palavras, média de 231, P95 de 588 e P99 de 902. Há avaliações de 6 palavras e uma de 2.473 palavras (uma descrição de lutas de wrestling).
- Em caracteres, a mediana é 970 e a média 1.309; a correlação com a contagem de palavras é 0,997, então as duas medidas são redundantes para os fins deste projeto.
- O comprimento é **praticamente igual entre sentimentos** (mediana 174 palavras nas negativas e 172 nas positivas), portanto o tamanho do texto não separa as classes de sentimento.
- Um comprimento máximo de 256 palavras manteria 71% das avaliações inteiras; 400 manteria 87% e 500, 92%.

```
freq_treino = sum((c for (p, _), c in contagens.items() if p == "train"), Counter())
freq_teste = sum((c for (p, _), c in contagens.items() if p == "test"), Counter())
freq_total = freq_treino + freq_teste
total_treino, total_teste = sum(freq_treino.values()), sum(freq_teste.values())

resumo_vocab = pd.Series({
    "tokens totais": sum(freq_total.values()),
    "vocabulário único (total)": len(freq_total),
    "vocabulário único (treino)": len(freq_treino),
    "palavras que aparecem 1 vez (total)": sum(1 for c in freq_total.values() if c == 1),
    "palavras com ao menos 5 ocorrências (treino)": sum(1 for c in freq_treino.values() if c >= 5),
    "palavras do teste ausentes no treino": len(set(freq_teste) - set(freq_treino)),
}, name="valor").astype(int)
display(resumo_vocab.to_frame())

linhas = {}
for n in (5_000, 10_000, 20_000, 30_000):
    top = {w for w, _ in freq_treino.most_common(n)}
    linhas[n] = {"cobertura dos tokens do treino": sum(freq_treino[w] for w in top) / total_treino,
                 "cobertura dos tokens do teste": sum(freq_teste[w] for w in top) / total_teste}
print("Cobertura de tokens pelas N palavras mais frequentes do treino:")
pd.DataFrame(linhas).T.rename_axis("N palavras")
```

	valor
tokens totais	11550138
vocabulário único (total)	111531
vocabulário único (treino)	81055
palavras que aparecem 1 vez (total)	43614
palavras com ao menos 5 ocorrências (treino)	30151
palavras do teste ausentes no treino	30476

Cobertura de tokens pelas N palavras mais frequentes do treino:

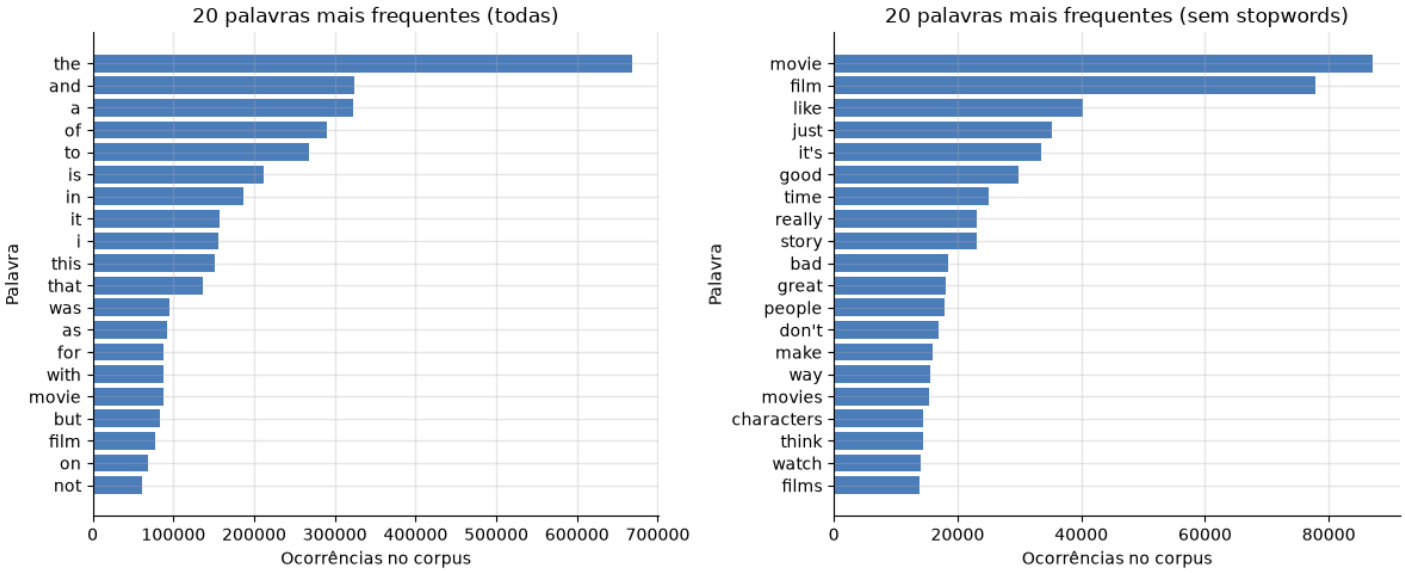
N palavras	cobertura dos tokens do treino	cobertura dos tokens do teste
5000	0.90	0.90
10000	0.94	0.94
20000	0.97	0.97
30000	0.99	0.98

```
sem_stop = [(w, c) for w, c in freq_total.most_common() if w not in ENGLISH_STOP_WORDS][:20]
todas = freq_total.most_common(20)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
for ax, dados, titulo in ((ax1, todas, "20 palavras mais frequentes (todas)"),
                          (ax2, sem_stop, "20 palavras mais frequentes (sem stopwords)")):
```

```
    palavras, valores = zip(*dados)
    ax.barh(palavras, valores, color="#4f81bd")
    ax.invert_yaxis()
    ax.set_title(titulo)
    ax.set_xlabel("Ocorrências no corpus")
    ax.set_ylabel("Palavra")
plt.tight_layout()
plt.show()

negacoes = [w for w in ("no", "not", "nor", "never", "cannot") if w in ENGLISH_STOP_WORDS]
print("Negações presentes na lista de stopwords do scikit-learn:", negacoes)
```




```
Negações presentes na lista de stopwords do scikit-learn: ['no', 'not', 'nor', 'never', 'cannot']
```

Interpretação do vocabulário.

- O corpus tem 11,55 milhões de tokens e **111.531 palavras distintas** (81.055 apenas no treino). Cerca de 39% do vocabulário (43.614 palavras) aparece uma única vez, o que indica uma cauda longa de grafias raras, nomes próprios e erros de digitação.
- A distribuição é muito concentrada: as 5.000 palavras mais frequentes do treino cobrem 90% dos tokens, 10.000 cobrem 94% e 20.000 cobrem 97%. A cobertura sobre o teste é praticamente a mesma, o que sugere que um vocabulário limitado generaliza bem entre as partições.
- Sem filtro, as palavras mais frequentes são palavras funcionais (*the*, *and*, *a*, *of*). Sem stopwords, dominam termos do domínio (*movie*, *film*, *story*). Observou-se que a lista de stopwords do scikit-learn **inclui negações** (*no*, *not*, *nor*, *never*, *cannot*) e **não inclui contrações** como *it's* e *don't*, que aparecem entre as mais frequentes.

2.4 Inspeção qualitativa

Comparação de estrutura e vocabulário entre avaliações positivas e negativas, com exemplos de comprimento típico e extremos. Esta comparação descreve a **opinião** dos autores; ela não indica que avaliações positivas ou negativas sejam mais ou menos suspeitas.

```
sent = df["sentimento"]
comparacao = pd.DataFrame({
    "mediana de palavras": df.groupby("sentimento")["n_palavras"].median(),
    "média de palavras": df.groupby("sentimento")["n_palavras"].mean(),
    "% com <br />": df["texto"].str.contains("<br />", regex=False).groupby(sent).mean() * 100,
    "'!' por avaliação": df["texto"].str.count("!").groupby(sent).mean(),
    "% com palavra em MAIÚSCULAS": df["texto"].str.contains(r"\b[A-Z]{4,}\b").groupby(sent).mean() * 100,
}).T
display(comparacao)

fp = sum((c for (_, s), c in contagens.items() if s == "positivo"), Counter())
fn = sum((c for (_, s), c in contagens.items() if s == "negativo"), Counter())
tp, tn = sum(fp.values()), sum(fn.values())
candidatas = [w for w in freq_total if freq_total[w] >= 500 and w not in ENGLISH_STOP_WORDS]
```

```
razao = {w: (fp[w] / tp) / (fn[w] / tn) for w in candidatas}
ordenadas = sorted(razao, key=razao.get)
print("Palavras (>= 500 ocorrências) mais associadas a cada sentimento:")
pd.DataFrame({"mais negativas": ordenadas[:10], "mais positivas": ordenadas[::-1][:10]})
```

sentimento	negativo	positivo
mediana de palavras	174.00	172.00
média de palavras	229.25	232.75
**% com **	59.72	57.08
'!' por avaliação	1.03	0.94
% com palavra em MAIÚSCULAS	26.01	19.35

```
Palavras (>= 500 ocorrências) mais associadas a cada sentimento:
```

	mais negativas	mais positivas
0	waste	wonderfully
1	unfunny	beautifully
2	worst	superb
3	awful	outstanding
4	redeeming	touching
5	laughable	friendship
6	poorly	wonderful
7	pointless	finest
8	sucks	gem
9	lame	magnificent

```
def mostrar(titulo, linha, limite=350):
    texto = linha["texto"]
    corte = texto[:limite] + ("..." if len(texto) > limite else "")
    print(f"[{titulo}] nota {linha['nota']}, {linha['n_palavras']} palavras\n{corte}\n")


mediana = df["n_palavras"].median()
for s in ("negativo", "positivo"):
    tipicas = df[(df["sentimento"] == s) & df["n_palavras"].between(0.9 * mediana, 1.1 * mediana)]
    mostrar(f"{s}, comprimento típico", tipicas.sample(1, random_state=42).iloc[0])

mostrar("menor avaliação da base", df.loc[df["n_palavras"].idxmin()])
mostrar("maior avaliação da base", df.loc[df["n_palavras"].idxmax()], limite=250)
```

```
[negativo, comprimento típico] nota 4, 157 palavras
A fine line up of actors and a seemingly nice plot -- though not original -- promised me a nice evening in front of the TV. I was
disappointed. The actors delivered up to standard (Juliette Lewis cuddly as ever; William Hurt solid but in the background; Shelley Duvall
convincing as ever) but the story was too thin to keep me engaged to the story an...

[positivo, comprimento típico] nota 10, 160 palavras
I was lucky enough to have seen this on a whim during a film festival and was smacked so hard with what I saw I returned the next night for
its second of three screenings. A funny, savage and sharp-toothed attack on every aspect of mainstream entertainment passively swallowed
without tasting by the lowest-common-denominator target audience waged by...

[menor avaliação da base] nota 1, 6 palavras
Primary plot!Primary direction!Poor interpretation.

[maior avaliação da base] nota 9, 2473 palavras
Match 1: Tag Team Table Match Bubba Ray and Spike Dudley vs Eddie Guerrero and Chris Benoit Bubba Ray and Spike Dudley started things off with
a Tag Team Table Match against Eddie Guerrero and Chris Benoit. According to the rules of the match, both o...
```

Interpretação.

- **Vocabulário:** é claramente distinto por sentimento: *waste*, *worst*, *awful* e *lame* do lado negativo; *superb*, *wonderful*, *beautifully* e *gem* do lado positivo. Esse é o sinal que um classificador de sentimento exploraria.
- **Estrutura:** as diferenças são pequenas. As avaliações negativas têm um pouco mais de palavras em MAIÚSCULAS (26,0% contra 19,4%) e de pontos de exclamação (1,03 contra 0,94 por avaliação); a presença de `<br />` é semelhante (59,7% e 57,1%).
- **Exemplos:** os textos típicos (cerca de 160 palavras) são parágrafos coerentes e opinativos. Os extremos mostram variabilidade legítima: a menor avaliação é uma frase telegráfica de 6 palavras, e a maior é um relato quase tabular de uma luta, sem opinião explícita.
- **Consequência para o projeto:** como o sentimento está associado a vocabulário e a traços de estilo, qualquer anomalia injetada precisará ser distribuída de forma equivalente entre avaliações positivas e negativas, para que o modelo não confunda sentimento com anomalia (seção 2.7).

2.5 Ocorrências naturais dos padrões e problemas de qualidade

Antes de definir como injetar os três padrões da Fase 1, verifica-se o quanto de linguagem semelhante já existe nas avaliações originais. Os números abaixo são apenas **ocorrências textuais** e não representam rótulos de spam nem de fraude.

```
minusculo = df["texto"].str.lower()
padroes = {
    "URL ou site (http, www, .com)": r"http|www\.|\.com\b",
    "nota no formato 'x/10'": r"\b\d{1,2}\s*/\s*10\b",
    "voto ('vote', 'voted', 'votes')": r"\bvot(?:e|ed|es|ing)\b",
    "'rating' ou 'rate'": r"\brat(?:e|ed|ing|ings)\b",
    "termos comerciais (buy, free, discount, click, subscribe, visit)": r"\b(?:buy|free|discount|click|subscribe|visit)\b",
    "caracteres não ASCII": r"[^\x00-\x7f]",
    "etiqueta HTML <br />": r"<br\s*/?>",
}
ocorrencias = pd.DataFrame({nome: (minusculo.str.contains(rx).sum()) for nome, rx in padroes.items()}, index=["avaliações"]).T
ocorrencias["% da base"] = ocorrencias["avaliações"] / len(df) * 100
display(ocorrencias)

print("Repetição consecutiva natural (pares de palavras idênticas seguidas, ex.: 'very very'):")
rep = df["rep_consec"]
pd.Series({"% das avaliações com ao menos 1 par": (rep >= 1).mean() * 100,
           "mediana": rep.median(), "P99": rep.quantile(0.99), "máximo": rep.max()}, name="valor").to_frame()
```

	avaliações	% da base
URL ou site (http, www, .com)	384	0.77
nota no formato 'x/10'	2980	5.96
voto ('vote', 'voted', 'votes')	725	1.45
'rating' ou 'rate'	3592	7.18
termos comerciais (buy, free, discount, click, subscribe, visit)	3210	6.42
caracteres não ASCII	4660	9.32
**etiqueta HTML **	29200	58.40

```
Repetição consecutiva natural (pares de palavras idênticas seguidas, ex.: 'very very'):
```

	valor
% das avaliações com ao menos 1 par	16.16
mediana	0.00
P99	3.00
máximo	18.00

Interpretação.

- **Etiquetas HTML:** 58,4% das avaliações contêm `<br />`. É ruído de formatação, a ser removido na limpeza da Fase 3.
- **Linguagem semelhante aos padrões já existe nos dados originais:** 0,8% das avaliações citam URL ou site; 6,0% trazem uma nota no formato "x/10"; 1,5% mencionam "vote"; 7,2% mencionam "rating" ou "rate"; 6,4% usam termos como *free*, *buy* ou *visit* em contexto normal. Logo, nenhuma palavra isolada identifica uma anomalia, e a classe 0 pode conter esses elementos legitimamente.
- **Repetição natural:** 16,2% das avaliações têm ao menos um par de palavras idênticas consecutivas (como *very very*), mas a mediana é zero, o P99 é 3 e o máximo é 18. Repetições sintéticas muito acima desses valores seriam facilmente distinguíveis das naturais.
- **Caracteres não ASCII** aparecem em 9,3% das avaliações (acentos, aspas tipográficas e símbolos).

2.6 Validações

A célula a seguir confere automaticamente a estrutura esperada (50.000 avaliações, 12.500 por partição e sentimento), a coerência entre nota e sentimento, a integridade dos textos originais por *hash*, a consistência entre a soma das contagens de palavras e as frequências do vocabulário, a correlação com a contagem por espaços e o comportamento do tokenizador em um exemplo conhecido. Qualquer divergência interrompe a execução com `AssertionError`.

```
assert len(df) == 50_000 and (df.groupby(["particao", "sentimento"]).size() == 12_500).all(), "estrutura inesperada"
assert (df["nota"].le(4) == df["sentimento"].eq("negativo")).all(), "nota incompatível com o sentimento"
assert pd.util.hash_pandas_object(df["texto"], index=False).sum() == HASH_TEXTOS, "textos originais foram alterados"
assert df["n_palavras"].sum() == sum(freq_total.values()), "contagem de palavras inconsistente com as frequências"
```

```
por_espacos = df["texto"].str.split().str.len()
assert df["n_palavras"].corr(por_espacos) > 0.99, "contagem por tokens diverge da contagem por espaços"
exemplo = "Great movie!<br /><br />Don't miss it, it's 10/10."
assert tokenizar(exemplo) == ["great", "movie", "don't", "miss", "it", "it's", "10", "10"]
print(f"Validações concluídas. Correlação entre contagem por tokens e por espaços: {df['n_palavras'].corr(por_espacos):.4f}")
```

```
Validações concluídas. Correlação entre contagem por tokens e por espaços: 0.9995
```

2.7 Conclusões da EDA

Principais características. Base de 50.000 avaliações, balanceada e rotulada apenas por sentimento (derivado da nota). O comprimento tem cauda longa (mediana de 173 palavras; P95 de 588) e é semelhante entre sentimentos. O vocabulário tem 111.531 palavras, muito concentrado: 20.000 palavras cobrem 97% dos tokens.

Problemas de qualidade.

- 58,4% das avaliações contêm a etiqueta `<br />`.
- 418 textos duplicados, com 123 grupos presentes em treino e teste, o que causa risco de vazamento.
- Cauda longa de palavras raras (39% do vocabulário aparece uma única vez) e contrações não tratadas pelas listas de stopwords usuais.
- Avaliações muito curtas (mínimo de 6 palavras) e muito longas (máximo de 2.473).
- Os únicos rótulos disponíveis medem opinião, não spam, e a classe "normal" pode conter spam real não identificado, conforme já registrado na Fase 1.

Impacto do comprimento na LSTM. Preencher todas as sequências até o maior texto (2.473 palavras) deixaria a maior parte de cada lote como *padding*, com custo de memória e tempo sem ganho. Sequências muito longas também dificultam o aprendizado de dependências distantes. Por outro lado, truncar descarta o final dos textos mais longos, e isso importa para a tarefa: **uma anomalia que cair na parte descartada deixaria de ser observável pelo modelo** e viraria ruído de rótulo.

Hipóteses preliminares de tokenização e *padding* (a serem testadas, não são hiperparâmetros definitivos):

- **Comprimento máximo:** testar valores entre 256 e 400 palavras (71% a 87% das avaliações inteiras), considerando o custo computacional disponível.
- **Vocabulário:** testar entre 10.000 e 20.000 palavras, mais um token para palavras desconhecidas e outro para *padding* (cobertura de 94% a 97%), ou um corte por frequência mínima.
- **Limpeza:** remover `<br />`, padronizar caixa e tratar contrações de forma explícita.
- **Stopwords:** a remoção não é obrigatória. Se aplicada, deve preservar as negações, e a sua influência sobre os padrões de repetição precisa ser verificada.
- **Padding:** usar máscara (*masking*) para que a LSTM ignore posições de preenchimento; a escolha entre preencher e truncar no início ou no fim deve considerar a posição das anomalias.

Cuidados para gerar anomalias sem associá-las ao sentimento (insumos para a Etapa 03):

1. A proporção de anomalias deve ser a **mesma em avaliações positivas e negativas**, e a conferência dessa proporção deve ser registrada.
2. Nem a nota nem o sentimento devem ser usados como entrada ou critério de seleção da anomalia.
3. O estilo dos modelos de anomalia (caixa alta, exclamações, `<br />`) deve ser neutro, pois esses traços já variam levemente com o sentimento.
4. Como URL, "x/10", "vote" e termos comerciais ocorrem naturalmente, os modelos de spam e de manipulação precisam de **variedade lexical** e não devem se reduzir a uma palavra-gatilho.
5. A repetição sintética deve superar com folga a repetição natural (P99 de 3 pares consecutivos).

6. A anomalia deve ser inserida em posições variadas e dentro do trecho que será efetivamente visto pelo modelo.
7. Os duplicados devem ser tratados antes de dividir treino, validação e teste, evitando que a mesma avaliação apareça em partições diferentes.

Como lidar com a falta de rótulos de spam (pergunta da Fase 2 do enunciado): mantém-se a decisão da Fase 1, em que as avaliações originais são presumidamente normais (classe 0) e uma fração delas é alterada com um dos três padrões sintéticos (classe 1), de acordo com os cuidados acima. Os resultados continuarão sendo uma prova de conceito, sem pretensão de comprovar fraude real.

Conclusão da Fase 2. A estrutura, a qualidade, o comprimento e o vocabulário da base foram documentados, e há informação suficiente para planejar a geração das anomalias na Etapa 03. Os textos originais permanecem inalterados.

Fase 3 — Preparação dos Dados (CRISP-DM) — Etapa 03: dataset supervisionado

Esta etapa constrói o dataset rotulado que alimentará a LSTM: classe **0** (avaliação original presumidamente normal) e classe **1** (avaliação modificada com anomalia sintética). A prioridade é a confiabilidade: nenhum texto idêntico ou derivado da mesma origem pode aparecer em partições diferentes, e nenhuma regra é ajustada com dados de teste.

Ficam para a etapa seguinte a tokenização definitiva, o *padding* e a conversão em números. Nenhum modelo é treinado aqui.

Decisão	Escolha
Duplicatas	Cópias equivalentes removidas (1 representante); quase-duplicatas mantidas, mas agrupadas; registros do teste em conflito com o treino são excluídos
Partições	Nenhuma avaliação do teste original vai para treino ou validação; validação tirada do treino original; um subconjunto do teste original reservado para generalização
Proporção das classes	50% / 50% em cada partição e sentimento
Seleção das anomalias	Sorteio com semente fixa (42), estratificado por partição e sentimento; não usa nota, sentimento nem comprimento
Posição da anomalia	Variável, sempre terminando até a 200ª palavra
Texto do classificador	Texto original limpo (` ` removido, espaços normalizados) mais o trecho inserido
Teste de generalização	1.200 avaliações com anomalias de mecanismos inéditos, fora do treinamento

3.1 Tratamento das duplicatas

Política (definida antes de qualquer modelo):

1. **Cópias equivalentes** são textos iguais após uma normalização básica (remoção de `<br />`, minúsculas, remoção de pontuação e espaços repetidos). Mantém-se **um representante** por grupo, dando prioridade ao treino original; as demais cópias são removidas e registradas em uma tabela com o representante correspondente. Se um grupo tivesse sentimentos distintos, seria descartado por inteiro.
2. **Quase-duplicatas** são textos diferentes que compartilham os mesmos 20 primeiros ou os mesmos 20 últimos tokens normalizados (reedições com pequenas correções ou assinaturas do mesmo autor). Elas **não são removidas**, mas recebem um `grupo_id` e **nunca são separadas** entre partições. Se um grupo reúne avaliações do treino original e do teste original, **preserva-se o treino e excluem-se os registros do teste**; eles nunca são transferidos para o treino. Cada exclusão é listada com os registros do treino a que se liga.
3. **Vínculo com a origem:** `id_origem` reproduz o caminho do arquivo original (`partição/sentimento/id_nota`) e é mantido em todo o dataset.

A célula de configuração define a semente e as funções de limpeza e de normalização, e as seguintes investigam as duplicatas.

```
import random

from scipy.stats import chi2_contingency
from sklearn.metrics import roc_auc_score

SEMENTE = 42
FRACAO_VALIDACAO = 0.15   # parcela da partição original de treino reservada à validação
N_GENERALIZACAO = 1_200   # avaliações da partição original de teste reservadas ao teste de generalização
JANELA_SEGURA = 200       # palavras iniciais que o classificador deverá enxergar (ver 3.3)
POS_MAX = 160             # maior posição de inserção, em palavras
SEG_MAX = 40              # maior trecho inserido, em palavras
TIPOS = ["spam_promocional", "repeticao_excessiva", "manipulacao_avaliacoes"]


def limpar(texto):
    """Limpeza mínima: remove <br /> e normaliza espaços; mantém caixa e pontuação."""
    return re.sub(r"\s+", " ", BR.sub(" ", texto)).strip()


def chave_normalizada(texto):
    """Chave de equivalência: limpeza, minúsculas e remoção de pontuação."""
    return re.sub(r"[\W_]+", " ", limpar(texto).lower()).strip()
```

```
base = pd.DataFrame({
    "id_origem": df["particao"] + "/" + df["sentimento"].str[:3] + "/" + df["id_review"].astype(str) + "_" + df["nota"].astype(str),
    "particao_original": df["particao"],
    "sentimento": df["sentimento"],
    "texto_original": df["texto"],
})
base["chave"] = base["texto_original"].map(chave_normalizada)


def resumo_duplicatas(chave):
    dup = base.loc[chave.duplicated(keep=False)].assign(k=chave)
    g = dup.groupby("k")
    return {"cópias excedentes": int(chave.duplicated().sum()), "grupos de textos equivalentes": g.ngroups,
            "grupos entre treino e teste": int((g["particao_original"].nunique() > 1).sum()),
            "grupos com sentimentos distintos": int((g["sentimento"].nunique() > 1).sum())}


niveis = {"1. texto exato": base["texto_original"],
          "2. limpeza + minúsculas": base["texto_original"].map(lambda t: limpar(t).lower()),
          "3. chave normalizada (sem pontuação)": base["chave"]}
pd.DataFrame({nome: resumo_duplicatas(chave) for nome, chave in niveis.items()})
```

	1. texto exato	2. limpeza + minúsculas	3. chave normalizada (sem pontuação)
cópias excedentes	418	422	424
grupos de textos equivalentes	406	410	412
grupos entre treino e teste	123	123	123
grupos com sentimentos distintos	0	0	0

```
tokens = base["chave"].str.split()
base["chave_inicio"] = tokens.map(lambda t: " ".join(t[:20]) if len(t) >= 20 else None)
base["chave_fim"] = tokens.map(lambda t: " ".join(t[-20:]) if len(t) >= 20 else None)

unicos = base.drop_duplicates("chave")  # quase-duplicatas além das equivalentes já tratadas
linhas = {}
for coluna in ("chave_inicio", "chave_fim"):
    d = unicos.dropna(subset=[coluna])
    d = d[d.duplicated(coluna, keep=False)]
    g = d.groupby(coluna)
    linhas[coluna] = {"grupos de quase-duplicatas": g.ngroups, "avaliações envolvidas": len(d),
                      "grupos entre treino e teste": int((g["particao_original"].nunique() > 1).sum())}
display(pd.DataFrame(linhas))

d = unicos.dropna(subset=["chave_inicio"])
exemplo = d[d.duplicated("chave_inicio", keep=False)].groupby("chave_inicio").head(2).head(2)
for _, linha in exemplo.iterrows():
    print(f"{linha['id_origem']}: ...{linha['chave'][-90:]}")
```

	chave_inicio	chave_fim
grupos de quase-duplicatas	34	31
avaliações envolvidas	71	66
grupos entre treino e teste	5	3

```
test/neg/1045_3: ...0 but few as undeserving as this one due to a shallow script and an unappealing resolution
test/neg/2053_4: ...lved and again sara obvious writing talent this film really should have delivered far more
```

Interpretação da investigação.

- A normalização encontra poucas duplicatas além das exatas: 418 cópias excedentes no texto exato, 422 após limpeza e minúsculas e **424** com a chave normalizada (412 grupos). Em todos os níveis, **123 grupos** cruzam treino e teste e **nenhum** tem sentimentos distintos.
- Há ainda 34 grupos de quase-duplicatas pelo início do texto (71 avaliações; 5 cruzam treino e teste) e 31 pelo final (66 avaliações; 3 cruzam). No exemplo exibido, duas avaliações compartilham o início e terminam de forma diferente. Na amostra inspecionada, os grupos por início eram em geral a mesma avaliação com pequenas edições ou continuações, e os grupos por final eram em geral assinaturas ou trechos finais copiados. Por isso o critério de agrupamento é conservador.

```
base["prioridade"] = (base["particao_original"] == "test").astype(int)  # em empate, o treino original é mantido
base = base.sort_values(["prioridade", "id_origem"]).reset_index(drop=True)

por_chave = base.groupby("chave")
conflito = por_chave["sentimento"].transform("nunique") > 1     # política: descartar o grupo inteiro (não ocorre)
eh_copia = base.duplicated("chave")
base["n_copias_removidas"] = por_chave["id_origem"].transform("size") - 1
removidas = base.loc[eh_copia, ["id_origem", "particao_original", "sentimento"]].assign(
    id_representante=por_chave["id_origem"].transform("first")[eh_copia],
    particao_representante=por_chave["particao_original"].transform("first")[eh_copia])
n_conflito = int(conflito.sum())
base = base.loc[~eh_copia & ~conflito].reset_index(drop=True)

# quase-duplicatas (mesmo início ou mesmo fim): permanecem, mas formam um grupo que não pode ser separado
pai = np.arange(len(base))


def raiz(i):
    while pai[i] != i:
        pai[i] = pai[pai[i]]
        i = pai[i]
    return i


for coluna in ("chave_inicio", "chave_fim"):
    for posicoes in base.groupby(coluna).indices.values():
        for j in posicoes[1:]:
            pai[raiz(j)] = raiz(posicoes[0])
base["grupo_id"] = [raiz(i) for i in range(len(base))]

# o teste original é preservado: se um grupo toca o treino original, os registros do teste são excluídos (nunca transferidos)
tem_treino = (base["particao_original"] == "train").groupby(base["grupo_id"]).transform("max")
conflitantes = (base["particao_original"] == "test") & tem_treino
ids_treino = base[base["particao_original"] == "train"].groupby("grupo_id")["id_origem"].agg(", ".join)
excluidas = base.loc[conflitantes, ["id_origem", "grupo_id", "sentimento"]].assign(
    ids_treino_do_grupo=base.loc[conflitantes, "grupo_id"].map(ids_treino),
    motivo="quase-duplicata de avaliação do treino original")
base = base.loc[~conflitantes].reset_index(drop=True)
assert base.groupby("grupo_id")["particao_original"].nunique().max() == 1, "grupo com avaliações de treino e de teste"

print("Vínculo entre cada cópia removida e o representante mantido (amostra):")
display(removidas.head(3))
print("Exclusões do teste original por quase-duplicata de avaliação do treino (todas):")
display(excluidas.reset_index(drop=True))
pd.Series({
    "avaliações originais": len(df),
    "cópias equivalentes removidas": len(removidas),
    "  das quais removidas do teste original (representante no treino)": int(((removidas["particao_original"] == "test") &
(removidas["particao_representante"] == "train")).sum()),
    "grupos descartados por sentimento conflitante": n_conflito,
    "avaliações mantidas": len(base),
    "  originárias do treino original": int((base["particao_original"] == "train").sum()),
    "  originárias do teste original": int((base["particao_original"] == "test").sum()),
    "grupos de quase-duplicatas (2 ou mais avaliações)": int((base.groupby("grupo_id").size() > 1).sum()),
    "avaliações do teste original excluídas por quase-duplicata do treino": len(excluidas),
}, name="quantidade").to_frame()
```

```
Vínculo entre cada cópia removida e o representante mantido (amostra):
```

	id_origem	particao_original	sentimento	id_representante	particao_representante
120	train/neg/10108_1	train	negativo	train/neg/10089_1	train
200	train/neg/10180_1	train	negativo	train/neg/10179_1	train
2016	train/neg/11815_3	train	negativo	train/neg/11209_3	train

```
Exclusões do teste original por quase-duplicata de avaliação do treino (todas):
```

	id_origem	grupo_id	sentimento	ids_treino_do_grupo	motivo
0	test/neg/1045_3	3306	negativo	train/neg/1731_3	quase-duplicata de avaliação do treino original
1	test/neg/11500_4	3307	negativo	train/neg/1732_4, train/pos/9629_8	quase-duplicata de avaliação do treino original
2	test/neg/3050_3	2012	negativo	train/neg/11813_1	quase-duplicata de avaliação do treino original
3	test/pos/10315_10	18208	positivo	train/pos/3957_10, train/pos/5822_10	quase-duplicata de avaliação do treino original
4	test/pos/11566_8	14635	positivo	train/pos/11988_8	quase-duplicata de avaliação do treino original
5	test/pos/11880_10	24709	positivo	train/pos/9828_10	quase-duplicata de avaliação do treino original
6	test/pos/3327_8	24709	positivo	train/pos/9828_10	quase-duplicata de avaliação do treino original
7	test/pos/4011_8	14635	positivo	train/pos/11988_8	quase-duplicata de avaliação do treino original
8	test/pos/5059_10	15994	positivo	train/pos/1963_8	quase-duplicata de avaliação do treino original
9	test/pos/6398_9	14628	positivo	train/pos/11981_9	quase-duplicata de avaliação do treino original
10	test/pos/7314_10	18208	positivo	train/pos/3957_10, train/pos/5822_10	quase-duplicata de avaliação do treino original
11	test/pos/8708_10	24709	positivo	train/pos/9828_10	quase-duplicata de avaliação do treino original

	quantidade
avaliações originais	50000
cópias equivalentes removidas	424
das quais removidas do teste original (representante no treino)	123
grupos descartados por sentimento conflitante	0
avaliações mantidas	49564
originárias do treino original	24901
originárias do teste original	24663
grupos de quase-duplicatas (2 ou mais avaliações)	42
avaliações do teste original excluídas por quase-duplicata do treino	12

Resultado da política.

- **424 cópias removidas:** 99 do treino original e 325 do teste original; em 123 dos casos do teste, o representante estava no treino (a cópia do teste foi descartada, e o teste permanece sem textos vistos no treino).
- **12 avaliações do teste original excluídas** por pertencerem a grupos de quase-duplicatas com avaliações do treino. A tabela de exclusões acima lista cada uma delas com os registros do treino correspondentes; nenhuma foi transferida para o treino.
- **49.564 avaliações mantidas** (24.901 do treino original e 24.663 do teste original). Nenhum grupo foi descartado por sentimento conflitante.
- **42 grupos de quase-duplicatas** com duas ou mais avaliações permanecem no dataset e ficam juntos em uma só partição.

3.2 Divisão dos dados

Decisões e justificativas.

- **Origem preservada:** treino e validação contêm **somente** avaliações do treino original; teste e generalização contêm **somente** avaliações do teste original. Isso é verificado por asserções na própria célula de divisão e nas validações (3.5).
- **Teste:** é a partição de teste original (após a deduplicação e a exclusão de 12 conflitos), exceto o subconjunto de generalização. Assim a separação original é respeitada, e o teste não contém textos do treino.
- **Validação:** 15% dos grupos do treino original, sorteados com semente fixa e estratificados por sentimento, para o controle de *overfitting* na Etapa de modelagem.
- **Generalização:** 1.200 grupos do teste original (600 por sentimento) são reservados para o teste complementar da seção 3.6, sem participar do treinamento nem de ajustes.
- **Proporção:** o teste ficou maior que o treino porque a partição original do IMDB é 50/50. Optou-se por não descartar dados nem reorganizar a separação original além do necessário.
- **Sem ajuste com dados de teste:** as regras de geração e seus parâmetros foram decididos a partir do desenho da solução e de diagnósticos feitos **apenas em treino e validação** (seção 3.5). Um único número

da EDA entra como parâmetro de verificação, a repetição natural P99, e foi recalculado só com o treino original.

```
grupos = base.groupby("grupo_id").agg(origem=("particao_original", "first"), sentimento=("sentimento", "first")).reset_index()
rng_divisao = np.random.default_rng(SEMENTE)
destino = {}
for (origem, sentimento), sub in grupos.groupby(["origem", "sentimento"]):
    ids = rng_divisao.permutation(sub["grupo_id"].to_numpy())
    corte = round(FRACAO_VALIDACAO * len(ids)) if origem == "train" else N_GENERALIZACAO // 2
    rotulos = ("validacao", "treino") if origem == "train" else ("generalizacao", "teste")
    destino.update(dict.fromkeys(ids[:corte], rotulos[0]))
    destino.update(dict.fromkeys(ids[corte:], rotulos[1]))
base["particao"] = base["grupo_id"].map(destino)
assert base.loc[base["particao"].isin(["treino", "validacao"]), "particao_original"].eq("train").all(), "treino/validação com avaliação do teste original"
assert base.loc[base["particao"].isin(["teste", "generalizacao"]), "particao_original"].eq("test").all(), "teste/generalização com avaliação do treino original"

ordem = ["treino", "validacao", "teste", "generalizacao"]
tabela = pd.crosstab(base["particao"], base["sentimento"]).loc[ordem]
tabela["total"] = tabela.sum(axis=1)
tabela["% do total"] = tabela["total"] / tabela["total"].sum() * 100
tabela["% positivo"] = tabela["positivo"] / tabela["total"] * 100
tabela
```

sentimento particao	negativo	positivo	total	% do total	% positivo
treino	10567	10597	21164	42.70	50.07
validacao	1864	1873	3737	7.54	50.12
teste	11661	11802	23463	47.34	50.30
generalizacao	600	600	1200	2.42	50.00

Interpretação e alterações em relação à partição original.

Partição	Avaliações	% do total	Origem
treino	21.164	42,7%	85% dos grupos do treino original (24.901 avaliações mantidas)
validacao	3.737	7,5%	15% dos grupos do treino original
teste	23.463	47,3%	teste original menos a generalização
generalizacao	1.200	2,4%	600 grupos de cada sentimento do teste original

Em relação ao IMDB original (25.000 em cada partição), as alterações foram: remoção de 424 cópias equivalentes; exclusão de 12 avaliações do teste por quase-duplicata de avaliações do treino; criação da validação a partir do treino; e reserva de 1.200 avaliações do teste. **Nenhuma avaliação mudou de partição original.** Os sentimentos permanecem equilibrados em todas as partições (50,0% a 50,3% de positivos), de modo que **não foi necessário reamostrar**.

3.3 Geração das anomalias sintéticas

As três categorias aprovadas na Fase 1 são implementadas como **inserções de um trecho** no texto limpo da avaliação. Os trechos são sintéticos e fictícios (domínios, telefones e marcas inventados) e **não representam fraudes comprovadas**.

Categoria	Mecanismos do treino (padrão)	Mecanismos inéditos (generalização)
Spam promocional	7 modelos de mensagem combinando ganchos, ofertas, chamadas e referências externas (site, @usuário, e-mail, telefone fictício, código promocional); 4 dos 7 modelos incluem um site	relato pessoal que esconde a promoção, mensagens em outros idiomas, link ofuscado, conteúdo patrocinado
Repetição excessiva	palavra (8 a 24 vezes), expressão de 2 a 4 palavras (4 a 10 vezes) ou frase de 4 a 8 palavras (3 a 5 vezes), tiradas do próprio texto da avaliação ou de um repertório neutro	blocos alternados, alongamento de caracteres, sequência sem sentido, enumeração numerada
Manipulação de avaliações	10 modelos de pedidos de nota, voto ou marcação de "útil" e de troca de avaliações; 70% pedem nota alta e 30% nota baixa	pedido indireto por ranking de revisores, troca mútua, revelação de pagamento, campanha coordenada, outros idiomas

Regras aplicadas a todas as anomalias.

- **Seleção independente de nota e sentimento:** em cada combinação de partição e sentimento, metade das avaliações é sorteada com semente fixa; os tipos se alternam entre as sorteadas. Nem a nota, nem o sentimento, nem o comprimento participam da escolha.
- **Variedade:** os trechos são montados a partir de listas de ganchos, ofertas, chamadas e números sorteados, com variação de pontuação e de caixa. Isso evita um único modelo repetido em todas as

amostras.

- **Posição variável:** o ponto de inserção é sorteado entre as fronteiras de palavra até a 160ª palavra; em metade dos casos ele é o fim de uma frase. Os trechos têm no máximo 40 palavras, de modo que a anomalia **termina até a 200ª palavra**. A Fase 2 indicou um comprimento máximo entre 256 e 400 como hipótese; a janela de 200 deixa margem para essa faixa. A tokenização definitiva ainda não existe, então a Etapa 04 deverá conferir essa garantia com as posições gravadas.
- **Manipulação nos dois sentidos:** a Fase 1 define manipulação como tentativa de inflar ou deprimir a reputação do título. Por isso uma parte dos pedidos é de nota baixa, o que impede que um "tom positivo" sirva de atalho. A maioria continua sendo de nota alta, conforme a definição do projeto.
- **Rastreabilidade:** cada anomalia registra tipo, variante, trecho inserido, posições em caracteres e em palavras. O texto original é preservado, e remover o trecho recupera exatamente o texto limpo.
- **Intensidade da repetição** acima do natural: a repetição consecutiva natural tem P99 = 3 pares (treino original), e a repetição de palavra do dataset tem ao menos 7.

```
RAIZ = ["film", "movie", "cine", "flix", "stream", "reel", "screen", "vid", "show", "tube"]
SUFIXO = ["deal", "zone", "hub", "vault", "club", "mania", "world", "base", "now", "max"]
TLD = ["com", "net", "org", "biz", "info", "tv", "co", "xyz"]


def dominio(rng):
    return rng.choice(RAIZ) + rng.choice(SUFIXO) + rng.choice(["", "", "24", "hd", str(rng.randint(2, 99))]) + "." + rng.choice(TLD)


def referencia(rng):
    d = dominio(rng)
    return rng.choice([f"www.{d}", d, f"http://{d}", f"https://www.{d}/{rng.choice(['offer', 'join', 'promo', 'deals'])}"])


SLOTS_SPAM = {
    "gancho": ["Special offer!", "Hot deal today only!", "Tired of paying full price?", "Want more movies for less?",
               "Don't miss out!", "Exclusive for readers:", "Limited time only!", "Breaking news for movie fans:"],
    "cta": ["Visit", "Check out", "Go to", "Click", "Head over to", "Log on to", "Order now at", "Sign up at"],
    "oferta": ["free HD streaming of every new release", "{pct}% off your first month", "cheap tickets and unlimited downloads",
               "a free gift card for new members", "buy one get one free on all DVDs", "exclusive discounts on blu-ray box sets",
               "a chance to win a free vacation", "the lowest prices guaranteed", "premium membership for only $4.99",
               "a {pct}% bonus on your first order"],
    "social": ["Follow @{handle} for {oferta}", "Subscribe to our channel for {oferta}", "Join @{handle} and get {oferta}", "Link in bio:
{oferta}"],
    "contato": ["Call 555-01{d2}", "Email deals@{dom}", "DM @{handle}", "WhatsApp 555-01{d2}", "Text YES to 555-01{d2}"],
    "cod": ["code SAVE{d2}", "promo code {w}{d2}", "coupon {w}{d2}"],
    "w": ["FILM", "MOVIE", "CINE", "FLIX", "DEAL", "VIP"],
    "ref": referencia, "dom": dominio,
    "handle": lambda rng: f"{rng.choice(RAIZ)}_{rng.choice(SUFIXO)}{rng.randint(1, 99)}",
    "pct": lambda rng: str(rng.choice([10, 15, 20, 25, 30, 40, 50, 60, 70])),
    "d2": lambda rng: f"{rng.randint(0, 99):02d}",
}
MODELOS_SPAM = ["{cta} {ref} for {oferta}!", "{gancho} Get {oferta}. {cta} {ref}", "Get {oferta} today, {cod}.",
                "{gancho} {social}", "{ref} - {oferta}", "{contato} to claim {oferta}.", "{gancho} {oferta}! {cta} {ref} and use {cod}."]

SLOTS_MANIP = {
    "nota_alta": ["10 stars", "10/10", "ten out of ten", "five stars", "a 10", "full marks"],
    "nota_baixa": ["1 star", "1/10", "zero stars", "a 1", "the lowest score"],
    "obra": ["movie", "film", "title"],
    "grupo": ["Our fan club is", "Everybody in our group is", "All of us are", "The whole community is", "My friends and I are"],
    "convite": ["join us", "do the same", "please do the same", "you should too", "help us out"],
    "util": ["mark it as helpful", "click the helpful button", "press the thumbs up"],
    "voto": ["vote for this movie", "rate it high", "give it a top score"],
    "imperativo": ["Vote", "Rate it", "Upvote"],
    "motivo": ["It needs to reach the top 250.", "Help it climb the rankings.", "Let's push its average up.", "More votes means more
viewers."],
    "troca": ["rate mine", "vote for my review", "mark my reviews helpful", "follow my profile"],
}
MODELOS_MANIP = {
    "alta": ["Please give this {obra} {nota_alta}!", "Everyone should vote {nota_alta} for this {obra}.",
             "{grupo} rating this {obra} {nota_alta}, {convite}.", "If you liked my review, {util} and {voto}.",
             "{imperativo}! {motivo}", "I will rate your {obra} {nota_alta} too if you {troca}."],
    "baixa": ["Vote {nota_baixa} to sink this {obra}.", "Let's all give this {obra} {nota_baixa} and drop its rating.",
              "{grupo} rating this {obra} {nota_baixa}, {convite}.", "Downvote this {obra} and bring its score down."],
}

POOL_REPETICAO = {
    "palavra": ["movie", "film", "watch", "really", "very", "just", "this", "now", "see", "time", "scene"],
    "expressao": ["this movie", "watch it", "see it", "so much", "every time", "you know", "check this", "right now"],
    "frase": ["this is what i think", "you know what i mean", "let me say it again", "i could not believe it"],
}
```

```python
def valor(opcoes, rng):
    return opcoes(rng) if callable(opcoes) else rng.choice(opcoes)


def preencher(modelo, slots, rng):
    """Substitui {slots} (que podem conter outros slots) por escolhas aleatórias."""
    while "{" in modelo:
        modelo = re.sub(r"\{(\w+)\}", lambda m: valor(slots[m.group(1)], rng), modelo)
    return modelo


def gerar_spam(palavras, rng):
    i = rng.randrange(len(MODELOS_SPAM))
    return f"modelo_{i}", preencher(MODELOS_SPAM[i], SLOTS_SPAM, rng)


def gerar_manipulacao(palavras, rng):
    direcao = "alta" if rng.random() < 0.7 else "baixa"   # a maioria pede nota alta; parte pede nota baixa
    i = rng.randrange(len(MODELOS_MANIP[direcao]))
    return f"{direcao}_{i}", preencher(MODELOS_MANIP[direcao][i], SLOTS_MANIP, rng)


def limpa_palavra(w):
    return re.sub(r"^\W+|\W+$", "", w)


def unidade_repetida(palavras, rng, n, pool):
    """n palavras consecutivas do próprio texto ou, na falta delas, um item de repertório neutro."""
    ok = [i for i in range(len(palavras) - n + 1) if all(re.fullmatch(r"[A-Za-z']{2,}", limpa_palavra(w)) for w in palavras[i:i + n])]
    if ok and rng.random() < 0.5:
        i = rng.choice(ok)
        return [limpa_palavra(w) for w in palavras[i:i + n]]
    return rng.choice(pool).split()


def gerar_repeticao(palavras, rng):
    tipo = rng.choice(["palavra", "expressao", "frase"])
    n = {"palavra": 1, "expressao": rng.randint(2, 4), "frase": rng.randint(4, 8)}[tipo]
    unidade = unidade_repetida(palavras, rng, n, POOL_REPETICAO[tipo])
    reps = {"palavra": rng.randint(8, 24), "expressao": rng.randint(4, 10), "frase": rng.randint(3, 5)}[tipo]
    reps = min(reps, SEG_MAX // len(unidade))
    return tipo, rng.choice([" ", " ", ", ", "! "]).join([" ".join(unidade)] * reps)


GERADORES_PADRAO = {"spam_promocional": gerar_spam, "repeticao_excessiva": gerar_repeticao,
                    "manipulacao_avaliacoes": gerar_manipulacao}
```

```python
MARCAS = ["Zentro", "Lumavue", "Quibo", "Vantiq", "Orbello", "Kyndra", "Blixa", "Narvo"]
SLOTS_INEDITOS = {
    "marca": MARCAS, "tld": TLD,
    "pct": SLOTS_SPAM["pct"], "handle": SLOTS_SPAM["handle"],
    "indicacao": lambda rng: "".join(rng.choices("ABCDEFGHJKLMNPQRSTUVWXYZ23456789", k=6)),
    "lista": ["ranking", "hall of fame", "list of top reviewers", "leaderboard"],
    "alvo": ["the average", "its score", "the rating"],
}
MODELOS_INEDITOS = {
    "spam_promocional": {
        "camuflado": ["Funny story, I only watched this because a subscription service gave me a free month, ask me how to get yours with
referral {indicacao}.",
                      "Thanks to the {marca} giveaway I saw this for free, and their sponsor page has {pct}% off for newcomers.",
                      "Watched it on a freebie from {marca}, the first month costs nothing if you mention my name."],
        "multilingue": ["Ganhe {pct}% de desconto assinando hoje, link na bio!", "Películas gratis en HD, solo hoy, escribe a {handle}.",
                        "Jetzt kostenlos streamen und {pct}% sparen bei {marca}!", "Oferta exclusiva da {marca}: filmes sem pagar nada, chame
no WhatsApp."],
        "link_ofuscado": ["{marca} [dot] com has every new release in HD, no signup needed.", "w w w dot {marca} dot c o m for cheap
tickets.",
                          "h t t p : / / {marca} . {tld} (copy the address into your browser)."],
        "patrocinado": ["Sponsored content: this review is brought to you by {marca}, our official partner.",
                        "Paid partnership with {marca}, details on their official page.", "Ad: {marca} supports this channel, explore their
catalog."],
    },
    "manipulacao_avaliacoes": {
        "ranking_pessoal": ["Marking this helpful keeps me on the reviewer {lista}, thank you kindly.",
                            "Every helpful vote moves my profile up the {lista}, so thanks in advance."],
        "troca_mutua": ["Follow my profile and I will follow back and score every one of your reviews.", "Swap deal: you boost my reviews, I
boost yours."],
        "pago": ["Full disclosure, the studio asked me to push this one to a top rating, so consider doing the same.",
                 "I was told to bring {alvo} up before the weekend, join in."],
        "campanha": ["Coordinated review-bombing starts tonight, everyone drop their scores to the bottom.",
                     "The one-star campaign is on, spread the word and sink {alvo}."],
        "multilingue": ["Por favor, deem nota 10 para este filme!", "Vota una estrella para hundirla, entre todos podemos.", "Bitte gebt zehn
Punkte, das hilft uns sehr."],
    },
}


def gerar_inedito(tipo):
```

```
    def gerar(palavras, rng):
        variante = rng.choice(list(MODELOS_INEDITOS[tipo]))
        return variante, preencher(rng.choice(MODELOS_INEDITOS[tipo][variante]), SLOTS_INEDITOS, rng)
    return gerar


def gerar_repeticao_inedita(palavras, rng):
    variante = rng.choice(["blocos", "alongamento", "sem_sentido", "enumeracao"])
    if variante == "blocos":      # A A A A B B B A A A B B B
        a, b = rng.sample(["yes", "no", "up", "down", "left", "right", "fast", "slow", "on", "off"], 2)
        k = rng.randint(2, 3)
        return variante, " ".join(([a] * k + [b] * k) * rng.randint(2, 3))
    if variante == "alongamento":  # repetição de caracteres dentro das palavras
        alvos = rng.sample(["so", "love", "good", "wow", "please", "yes", "best", "more"], rng.randint(3, 5))
        return variante, " ".join(w[:1] + w[1] * rng.randint(6, 14) + w[2:] for w in alvos)
    if variante == "sem_sentido":  # sequência sem significado repetida
        return variante, " ".join(["".join(rng.choices("abcdefghijklmnopqrstuvwxyz", k=rng.randint(3, 6)))] * rng.randint(6, 15))
    objeto = rng.choice(["film", "movie", "scene", "actor", "part"])
    return variante, " ".join(f"{i} {objeto}" for i in range(1, rng.randint(6, 12) + 1))


GERADORES_INEDITOS = {"spam_promocional": gerar_inedito("spam_promocional"), "repeticao_excessiva": gerar_repeticao_inedita,
                      "manipulacao_avaliacoes": gerar_inedito("manipulacao_avaliacoes")}
```

```
def estilizar(trecho, rng):
    """Varia pontuação (metade dos '!' viram '.') e caixa: MAIÚSCULAS (6%), minúsculas (25%) ou como escrito."""
    if rng.random() < 0.5:
        trecho = trecho.replace("!", ".")
    r = rng.random()
    return trecho.upper() if r < 0.06 else trecho.lower() if r < 0.31 else trecho


def escolher_posicao(palavras, rng):
    """Posição de inserção (em palavras): metade das vezes em fim de frase, o restante em qualquer fronteira."""
    limite = min(len(palavras), POS_MAX)
    if rng.random() < 0.5:
        fins = [i + 1 for i, w in enumerate(palavras[:limite]) if w.endswith((".", "!", "?"))]
        if fins:
            return rng.choice(fins)
    return rng.randint(0, limite)


def injetar(palavras, gerador, rng):
    """Devolve (texto, trecho, início, fim, posição em palavras, variante)."""
    for _ in range(10):
        variante, trecho = gerador(palavras, rng)
        segmento = estilizar(trecho, rng).split()
        if 0 < len(segmento) <= SEG_MAX:
            break
    else:
        raise ValueError("trecho fora do tamanho permitido")
    pos = escolher_posicao(palavras, rng)
    antes = " ".join(palavras[:pos])
    ini = len(antes) + (1 if pos > 0 else 0)
    trecho = " ".join(segmento)
    return " ".join(palavras[:pos] + segmento + palavras[pos:]), trecho, ini, ini + len(trecho), pos, variante


def atribuir_classes(base, semente):
    """Metade de cada (partição, sentimento) recebe anomalia; os tipos se alternam entre as escolhidas."""
    ds = base.sort_values("id_origem").reset_index(drop=True)
    ds["classe"], ds["tipo_anomalia"] = 0, "nenhuma"
    rng = np.random.default_rng([semente, 1])
    indices = ds.groupby(["particao", "sentimento"]).indices
    for chave in sorted(indices):
        escolhidos = rng.permutation(indices[chave])[: len(indices[chave]) // 2]
        ds.loc[escolhidos, "classe"] = 1
        ds.loc[escolhidos, "tipo_anomalia"] = np.resize(TIPOS, len(escolhidos))
    return ds


def gerar_textos(ds, semente):
    saidas = []
    for id_origem, texto, tipo, particao in zip(ds["id_origem"], ds["texto_original"], ds["tipo_anomalia"], ds["particao"]):
        palavras = limpar(texto).split(" ")
        if tipo == "nenhuma":
            saidas.append((" ".join(palavras), "", -1, -1, -1, ""))
        else:
            geradores = GERADORES_INEDITOS if particao == "generalizacao" else GERADORES_PADRAO
            saidas.append(injetar(palavras, geradores[tipo], random.Random(f"{semente}|{id_origem}")))
    return pd.DataFrame(saidas, columns=["texto_modelo", "trecho_inserido", "ini_anomalia", "fim_anomalia", "pos_palavra", "variante"],
index=ds.index)


COLUNAS = ["id_origem", "grupo_id", "n_copias_removidas", "particao_original", "particao", "sentimento", "texto_original",
           "texto_modelo", "classe", "tipo_anomalia", "variante", "sintetico", "trecho_inserido", "ini_anomalia", "fim_anomalia",
"pos_palavra"]
```

```
def construir_dataset(base, semente):
    ds = atribuir_classes(base, semente)
    ds = ds.join(gerar_textos(ds, semente))
    ds["sintetico"] = ds["classe"] == 1
    return ds[COLUNAS]


dataset = construir_dataset(base, SEMENTE)
dataset.shape
```

```
(49564, 16)
```

Amostras reproduzíveis (semente 42) das três categorias, com inserções no início (palavras 0 a 20), no meio (21 a 100) e no final da janela (101 a 160). No texto original, ˆ marca o ponto de inserção; no texto modificado, o trecho inserido aparece entre « ». O original traz `<br />`, que é normalizado no texto do classificador. Em seguida há uma amostra de cada categoria com mecanismos inéditos (generalização) e a proporção de inserções que ocorrem no início do texto ou logo após o fim de uma frase.

```
def janela(linha, margem=110):
    """Contexto antes e depois da inserção; ˆ marca o ponto de inserção no texto original limpo."""
    t, ini, fim = linha["texto_modelo"], linha["ini_anomalia"], linha["fim_anomalia"]
    original = limpar(linha["texto_original"])
    cortes = ("..." if ini > margem else "", "..." if fim + margem < len(t) else "")
    antes = cortes[0] + original[max(0, ini - margem):ini] + "ˆ" + original[ini:ini + margem] + cortes[1]
    depois = cortes[0] + t[max(0, ini - margem):ini] + "«" + t[ini:fim] + "»" + t[fim:fim + margem] + cortes[1]
    return antes, depois


faixas = {"início (palavras 0 a 20)": (0, 20), "meio (21 a 100)": (21, 100), "final da janela (101 a 160)": (101, 160)}
anomalias_treino = dataset[(dataset["classe"] == 1) & (dataset["particao"] == "treino")]
for tipo in TIPOS:
    for nome, (a, b) in faixas.items():
        linha = anomalias_treino[(anomalias_treino["tipo_anomalia"] == tipo) & anomalias_treino["pos_palavra"].between(a, b)].sample(1,
random_state=SEMENTE).iloc[0]
        antes, depois = janela(linha)
        print(f"[{tipo} | {nome} | variante '{linha['variante']}' | {linha['sentimento']} | posição {linha['pos_palavra']}]")
        print("  ANTES :", antes)
        print("  DEPOIS:", depois, "\n")

print("Mecanismos inéditos (generalização), uma amostra por categoria:")
for tipo in TIPOS:
    linha = dataset[(dataset["particao"] == "generalizacao") & (dataset["tipo_anomalia"] == tipo)].sample(1, random_state=SEMENTE).iloc[0]
    print(f"[{tipo} | variante '{linha['variante']}' | {linha['sentimento']} | posição {linha['pos_palavra']}]")
    print("  DEPOIS:", janela(linha)[1], "\n")

apos_frase = anomalias_treino.apply(lambda l: l["ini_anomalia"] == 0 or l["texto_modelo"][:l["ini_anomalia"]].rstrip().endswith((".", "!",
"?")), axis=1)
print("Inserções no início do texto ou logo após o fim de uma frase (%), por tipo:")
(apos_frase.groupby(anomalias_treino["tipo_anomalia"]).mean() * 100).to_frame("%")
```

[spam_promocional | início (palavras 0 a 20) | variante 'modelo_5' | positivo | posição 10]
  ANTES : Although this movie has some weaknesses, it is worth seeing. ˆI chose it because of the cast, and applaud Bonham Carter and Branagh for choosing roles different from those ...
  DEPOIS: Although this movie has some weaknesses, it is worth seeing. «Call 555-0104 to claim free HD streaming of every new release.» I chose it because of the cast, and applaud Bonham Carter and Branagh for choosing roles different from those...

[spam_promocional | meio (21 a 100) | variante 'modelo_3' | negativo | posição 24]
  ANTES : ... be honest I knew what to expect before I watched this film, and I've got to say it has the worst acting I've ˆever seen. It does have its moments, and on a comedy level its very entertaining, but i'm afraid its not scary...
  DEPOIS: ... be honest I knew what to expect before I watched this film, and I've got to say it has the worst acting I've «hot deal today only! subscribe to our channel for buy one get one free on all dvds» ever seen. It does have its moments, and on a comedy level its very entertaining, but i'm afraid its not scar...

[spam_promocional | final da janela (101 a 160) | variante 'modelo_2' | positivo | posição 112]
  ANTES : ... a manga and was soon followed by also popular anime series which included over 100 episodes. It was aired in ˆ1978. A year later came this anime film, which isn't a sequel to the series, but summaries the main points of ...
  DEPOIS: ... a manga and was soon followed by also popular anime series which included over 100 episodes. It was aired in «get premium membership for only $4.99 today, coupon cine71.» 1978. A year later came this anime film, which isn't a sequel to the series, but summaries the main points of...

[repeticao_excessiva | início (palavras 0 a 20) | variante 'frase' | positivo | posição 2]
  ANTES : The film ˆgets my stamp of approval. The scene in the museum demands acting without dialogue. This is one of the most in...
  DEPOIS: The film «i could not believe it i could not believe it i could not believe it» gets my stamp of approval. The scene in the museum demands acting without dialogue. This is one of the most i...

[repeticao_excessiva | meio (21 a 100) | variante 'frase' | negativo | posição 73]
  ANTES : ...aghan fighting against Butch Cassidy and Ironhead because their gang killed his one (he's the only survivor). ˆThen you have Butch Cassidy and Ironhead fighting each other because they quarreled and the gang split. And yo...
  DEPOIS: ...aghan fighting against Butch Cassidy and Ironhead because their gang killed his one (he's the only survivor). «by klaus kinski as reverend cotton is by klaus kinski as reverend cotton is by klaus kinski as reverend cotton is by klaus kinski as reverend cotton is» Then you have Butch Cassidy and Ironhead fighting each other because they quarreled and the gang split. And y...

[repeticao_excessiva | final da janela (101 a 160) | variante 'expressao' | negativo | posição 106]
  ANTES : ...focuses on the same plot. The show revolves around Timmy Turner, who was granted two Fairy Godparents because ˆbasically his life was horrible - his parents were never around, and he had a nasty baby-sitter. The plot of t...
  DEPOIS: ...focuses on the same plot. The show revolves around Timmy Turner, who was granted two Fairy Godparents because «fairy godparents because basically fairy godparents because basically fairy godparents because basically fairy godparents because basically fairy godparents because basically fairy godparents because basically fairy godparents because basically fairy godparents because basically» basically his life was horrible - his parents were never around, and he had a nasty baby-sitter. The plot of ...

[manipulacao_avaliacoes | início (palavras 0 a 20) | variante 'alta_1' | positivo | posição 12]
  ANTES : First, I did like this film. It was well acted by all. ˆHowever, I don't understand comments I hear from people about a surprise ending. I knew nothing about this mov...
  DEPOIS: First, I did like this film. It was well acted by all. «everyone should vote a 10 for this film.» However, I don't understand comments I hear from people about a surprise ending. I knew nothing about this mo...

[manipulacao_avaliacoes | meio (21 a 100) | variante 'alta_0' | positivo | posição 68]
  ANTES : ...ade in 1973 attempts to prophesies a future earth , in 2022, that is so overpopulated that the human race has ˆbeen manipulated by authorities to eat a universally produced food product called "Soylent Green" which is man...
  DEPOIS: ...ade in 1973 attempts to prophesies a future earth , in 2022, that is so overpopulated that the human race has «Please give this movie 10/10!» been manipulated by authorities to eat a universally produced food product called "Soylent Green" which is ma...

[manipulacao_avaliacoes | final da janela (101 a 160) | variante 'alta_3' | positivo | posição 117]
  ANTES : ... was dedicated to making the movie worth watching. Hell, the biggest gun battle takes place off screen -- and ˆthe scene where it is heard is all the better for that background noise, that adds to the suspense -- who's wi...
  DEPOIS: ... was dedicated to making the movie worth watching. Hell, the biggest gun battle takes place off screen -- and «if you liked my review, click the helpful button and rate it high.» the scene where it is heard is all the better for that background noise, that adds to the suspense -- who's w...

Mecanismos inéditos (generalização), uma amostra por categoria:
[spam_promocional | variante 'multilingue' | negativo | posição 34]
  DEPOIS: ... by Rock Hudson's real-life dilemma from the 1950s: what to do with a screen idol who is secretly homosexual? «Ganhe 60% de desconto assinando hoje, link na bio!» Marry him off to an unsuspecting woman in order to quell the gossips (and keep him working). Wispy-thin idea ...

[repeticao_excessiva | variante 'blocos' | negativo | posição 69]
  DEPOIS: ...is film is best watched with other like minded individuals so you have someone to laugh with. You'll laugh as «right right down down right right down down right right down down» Greg leaps and shuffles around the lab, petting his pet rabbit, while his hunchback shifts from right to left...

[manipulacao_avaliacoes | variante 'troca_mutua' | negativo | posição 129]
  DEPOIS: ...a crew member on the first plane to drop a hydrogen bomb. I have wondered if any one of them died in Vietnam. «Swap deal: you boost my reviews, I boost yours.» I imagine that they flew missions there. Regrettably, the film fails. The movie lacks visual interest, drama,...

Inserções no início do texto ou logo após o fim de uma frase (%), por tipo:

tipo_anomalia	%
manipulacao_avaliacoes	54.13
repeticao_excessiva	52.37
spam_promocional	52.23

Resultado da inspeção qualitativa.

- **Consistência:** em todas as amostras o trecho aparece íntegro, na posição indicada, e o restante do texto é idêntico ao original limpo. A reconstrução exata do texto original ao remover o trecho também é verificada automaticamente em 100% das anomalias (3.5).
- **Posição:** entre 52% e 54% das inserções ocorrem no início do texto ou logo após o fim de uma frase (metade por desenho, mais os casos aleatórios que coincidem com fim de frase); as demais caem no meio de frases (por exemplo, "the worst acting I've «...» ever seen"), como ocorreria numa injeção real.
- **Problemas observados, não corrigidos:** (1) algumas combinações de spam são agramaticais, como "subscribe to our channel for buy one get one free", porque o modelo de mensagem e a oferta são sorteados separadamente; (2) a "frase" repetida é uma janela de palavras consecutivas que pode começar no meio de uma frase ("by klaus kinski as reverend cotton is ..."), então o nome é aproximado; (3) a repetição de expressão tirada do próprio texto pode formar sequências sem sentido ("fairy godparents because basically ..."), o que é esperado em texto artificial; (4) a variação de caixa e pontuação altera a aparência de parte dos trechos (25% em minúsculas, 6% em maiúsculas). Nenhum deles invalida os critérios da etapa, e os mecanismos de geração **não foram modificados**.

3.4 Dataset final

O dataset tem **uma linha por avaliação de origem**: cada avaliação é normal ou modificada, nunca as duas. Assim a base não é duplicada, e o texto original fica compartilhado com o `DataFrame` da Fase 2.

Coluna	Conteúdo
`id_origem`, `grupo_id`, `n_copias_removidas`	vínculo com o arquivo original, grupo de quase-duplicatas e quantidade de cópias removidas
`particao_original`, `particao`	partição do IMDB (`train` / `test`) e partição final (`treino`, `validacao`, `teste`, `generalizacao`)
`sentimento`	sentimento original (apenas descritivo)
`texto_original`, `texto_modelo`	texto original e texto usado pelo classificador
`classe`, `sintetico`	rótulo 0/1 e indicador de texto sintético
`tipo_anomalia`, `variante`	categoria e mecanismo da anomalia
`trecho_inserido`, `ini_anomalia`, `fim_anomalia`, `pos_palavra`	trecho, posições em caracteres e posição em palavras

Proporção entre as classes: 50% / 50% em cada partição e sentimento. A escolha torna as métricas de treino e de validação menos dominadas por uma classe e evita decidir uma prevalência sem base real: a frequência verdadeira de spam em avaliações é desconhecida. A proporção é uma decisão de projeto. Na avaliação final, a classe 1 do teste poderá ser subamostrada para simular prevalências menores, sem alterar o treino.

```
print("Avaliações por partição, classe e sentimento:")
display(pd.crosstab([dataset["particao"], dataset["classe"]], dataset["sentimento"]).reindex(ordem, level=0))

print("Tipos de anomalia por partição:")
display(pd.crosstab(dataset["particao"], dataset["tipo_anomalia"]).loc[ordem])

print("Variantes (modelos de mensagem) por tipo, apenas treino + validação + teste:")
padrao = dataset[(dataset["classe"] == 1) & (dataset["particao"] != "generalizacao")]
display(padrao.groupby("tipo_anomalia")["variante"].nunique().to_frame("variantes distintas"))
print("Variantes inéditas (generalização):")
display(dataset[dataset["particao"] == "generalizacao"].query("classe == 1").groupby(["tipo_anomalia",
"variante"]).size().to_frame("avaliações"))

print("Trecho inserido (palavras) e posição da inserção (índice da palavra), por tipo:")
trechos = padrao.assign(palavras_trecho=padrao["trecho_inserido"].str.count(" ") + 1)
display(trechos.groupby("tipo_anomalia").agg(trecho_medio=("palavras_trecho", "mean"), trecho_min=("palavras_trecho", "min"), trecho_max=
("palavras_trecho", "max"),
                                             pos_min=("pos_palavra", "min"), pos_mediana=("pos_palavra", "median"), pos_max=("pos_palavra",
"max")))

memoria = dataset.memory_usage(deep=True)
print(f"Memória: {memoria.sum() / 1e6:.0f} MB, dos quais {memoria['texto_original'] / 1e6:.0f} MB são `texto_original` (as mesmas strings de
`df`, sem cópia).")
print(f"Colunas: {list(dataset.columns)}")
```

```
Avaliações por partição, classe e sentimento:
```

particao	classe	sentimento negativo	sentimento positivo
treino	0	5284	5299
	1	5283	5298
validacao	0	932	937
	1	932	936
teste	0	5831	5901
	1	5830	5901
generalizacao	0	300	300
	1	300	300

Tipos de anomalia por partição:

particao	tipo_anomalia manipulacao_avaliacoes	nenhuma	repeticao_excessiva	spam_promocional
treino	3527	10583	3527	3527
validacao	622	1869	623	623
teste	3910	11732	3910	3911
generalizacao	200	600	200	200

Variantes (modelos de mensagem) por tipo, apenas treino + validação + teste:

tipo_anomalia	variantes distintas
manipulacao_avaliacoes	10
repeticao_excessiva	3
spam_promocional	7

Variantes inéditas (generalização):

tipo_anomalia	variante	avaliações
manipulacao_avaliacoes	campanha	39
	multilingue	31
	pago	48
	ranking_pessoal	45
	troca_mutua	37
repeticao_excessiva	alongamento	52
	blocos	39
	enumeracao	51
	sem_sentido	58
spam_promocional	camuflado	58
	link_ofuscado	47
	multilingue	47
	patrocinado	48

Trecho inserido (palavras) e posição da inserção (índice da palavra), por tipo:

tipo_anomalia	trecho_medio	trecho_min	trecho_max	pos_min	pos_mediana	pos_max
manipulacao_avaliacoes	9.99	5	16	0	69.00	160
repeticao_excessiva	18.56	8	40	0	69.00	160
spam_promocional	12.06	6	22	0	69.00	160

```
Memória: 169 MB, dos quais 77 MB são `texto_original` (as mesmas strings de `df`, sem cópia).
Colunas: ['id_origem', 'grupo_id', 'n_copias_removidas', 'particao_original', 'particao', 'sentimento', 'texto_original', 'texto_modelo',
'classe', 'tipo_anomalia', 'variante', 'sintetico', 'trecho_inserido', 'ini_anomalia', 'fim_anomalia', 'pos_palavra']
```

Interpretação.

- **Classes (classe 0 / classe 1):** treino 10.583 / 10.581; validação 1.869 / 1.868; teste 11.732 / 11.731; generalização 600 / 600. Em cada combinação de partição e sentimento, as classes diferem em no máximo uma avaliação.
- **Tipos de anomalia:** equilibrados (por exemplo, no treino: 3.527 de cada tipo). No padrão de treino há 7 modelos de spam, 3 formas de repetição e 10 modelos de manipulação; a generalização tem 13 mecanismos inéditos.
- **Tamanho do trecho:** em média 12,1 palavras no spam, 18,6 na repetição e 10,0 na manipulação, com a posição de inserção variando de 0 a 160 (mediana 69 nos três tipos).

3.5 Validação da qualidade

A primeira célula executa 21 verificações automáticas; qualquer falha interrompe a execução com `AssertionError`. Elas cobrem identificadores, vazamento, origem das partições, rastreabilidade das exclusões, vazios, distribuição de classes, tipos e sentimentos, coerência dos rótulos, existência e posição do padrão inserido, preservação dos textos originais, isolamento da generalização e reprodutibilidade. A segunda célula

compara as classes normal e sintética para buscar **atalhos** que o modelo poderia aprender sem reconhecer o padrão; seus diagnósticos usam somente treino e validação, o que é garantido por uma asserção.

```
DETECTORES = {
    "spam_promocional": re.compile(r"www\.|https?:|h\s*t\s*t\s*p|\.
(?:com|net|org|biz|info|tv|co|xyz)\b|@\w|%|free|discount|coupon|promo|code|offer|deal|subscribe|buy|click|visit|download|win\b|bio\b|call|ema
il|whatsapp|price|membership|order|sign
up|sponsor|referral|giveaway|\bdot\b|gratis|grátis|desconto|descuento|sparen|oferta|\bad:|partnership", re.I),
    "manipulacao_avaliacoes": re.compile(r"vot|rat(?:e|ing)|stars?
|\d\s*/\s*10|helpful|score|upvote|downvote|thumbs|leaderboard|follow|nota|estrella|punkte|review-bombing|top rating|average|marks|out of
ten|boost|swap|give (?:this|it)|a 10", re.I),
}


def tem_repeticao(trecho):
    tokens = re.findall(r"\w+", trecho.lower())
    diversidade = len(set(tokens)) / max(len(tokens), 1)
    return diversidade <= 0.5 or max(Counter(tokens).values(), default=0) >= 5 or bool(re.search(r"(\w)\1{4,}", trecho.lower()))


def padrao_presente(linha):
    if linha["tipo_anomalia"] == "repeticao_excessiva":
        return tem_repeticao(linha["trecho_inserido"])
    return bool(DETECTORES[linha["tipo_anomalia"]].search(linha["trecho_inserido"]))


def remover_trecho(linha):
    t, ini, fim = linha["texto_modelo"], linha["ini_anomalia"], linha["fim_anomalia"]
    return t[:ini] + t[fim + 1:] if fim < len(t) else t[:ini - 1]


sint = dataset[dataset["classe"] == 1]
normal = dataset[dataset["classe"] == 0]
ids_df = df["particao"] + "/" + df["sentimento"].str[:3] + "/" + df["id_review"].astype(str) + "_" + df["nota"].astype(str)
originais = pd.Series(df["texto"].to_numpy(), index=ids_df.to_numpy())
p99_natural = df.loc[df["particao"] == "train", "rep_consec"].quantile(0.99)   # calculado só com o treino original
reps_palavra = sint.query("tipo_anomalia == 'repeticao_excessiva' and variante == 'palavra'")["trecho_inserido"].map(lambda t:
len(re.findall(r"\w+", t)))
cont = dataset.groupby(["particao", "sentimento", "classe"]).size().unstack()
tipos_cont = sint.groupby(["particao", "sentimento", "tipo_anomalia"]).size().unstack()
pos_pct = dataset.groupby(["particao", "classe"])["sentimento"].apply(lambda s: (s == "positivo").mean())
reconstruidos = sint.apply(remover_trecho, axis=1)
palavras_ate_fim = sint.apply(lambda l: len(l["texto_modelo"][:l["fim_anomalia"]].split()), axis=1)
dataset_2 = construir_dataset(base, SEMENTE)
dataset_3 = construir_dataset(base, SEMENTE + 1)
hash_ds = lambda d: pd.util.hash_pandas_object(d[["id_origem", "texto_modelo", "classe", "tipo_anomalia"]], index=False).sum()
fracao_unica = sint.groupby("tipo_anomalia")["trecho_inserido"].agg(lambda s: s.value_counts().iloc[0] / len(s))
chi_p = [chi2_contingency(pd.crosstab(d["classe"], d["sentimento"]))[1] for _, d in dataset.groupby("particao")]
em_treino_val = dataset["particao"].isin(["treino", "validacao"])

ger = dataset["particao"] == "generalizacao"
inseridos = dataset["classe"] == 1
ids_validos = dataset["id_origem"].str.fullmatch(r"(train|test)/(pos|neg)/\d+_\d+")
ids_treino_conflito = excluidas["ids_treino_do_grupo"].str.split(", ").explode()
sem_bordas = dataset["texto_modelo"].map(lambda t: t == t.strip() and "  " not in t and "<br" not in t).all()

verificacoes = [
    ("Identificadores válidos, únicos, presentes no IMDB carregado e coerentes com sentimento e partição original",
     bool(dataset["id_origem"].is_unique and ids_validos.all() and dataset["id_origem"].isin(originais.index).all()
          and (dataset["id_origem"].str.split("/").str[1] == dataset["sentimento"].str[:3]).all()
          and (dataset["id_origem"].str.split("/").str[0] == dataset["particao_original"]).all()), f"{len(dataset):,} linhas"),
    ("Nenhum texto equivalente (chave normalizada) se repete", dataset["texto_original"].map(chave_normalizada).is_unique, "duplicatas
removidas"),
    ("Nenhum grupo de quase-duplicatas atravessa partições", dataset.groupby("grupo_id")["particao"].nunique().max() == 1, "agrupamento por
início/fim"),
    ("Treino e validação só do treino original; teste e generalização só do teste original",
     bool(dataset.loc[em_treino_val, "particao_original"].eq("train").all() and dataset.loc[~em_treino_val,
"particao_original"].eq("test").all()), "sem transferência entre origens"),
    ("Exclusões rastreáveis: ausentes do dataset e ligadas a registros do treino presentes",
     bool(not excluidas["id_origem"].isin(dataset["id_origem"]).any() and ids_treino_conflito.isin(dataset.loc[em_treino_val,
"id_origem"]).all()), f"{len(excluidas)} exclusões"),
    ("Sem textos vazios; texto do classificador sem <br, espaços duplicados ou nas bordas",
     bool((dataset["texto_original"].str.strip() != "").all() and (dataset["texto_modelo"].str.strip() != "").all() and sem_bordas), "0
vazios"),
    ("Classes equilibradas por partição e sentimento (diferença <= 1)", bool((cont[0] - cont[1]).abs().max() <= 1), f"diferença máx. =
{(cont[0] - cont[1]).abs().max()}"),
    ("Tipos equilibrados por partição e sentimento (diferença <= 1)", bool((tipos_cont.max(axis=1) - tipos_cont.min(axis=1)).max() <= 1),
f"diferença máx. = {(tipos_cont.max(axis=1) - tipos_cont.min(axis=1)).max()}"),
    ("% de positivos igual nas duas classes (diferença < 1 p.p.)", bool((pos_pct.unstack().diff(axis=1)[1].abs() < 0.01).all()), f"p-valores
qui-quadrado: {[round(float(p), 2) for p in chi_p]}"),
    ("Rótulos coerentes: classe, sintetico, tipo e trecho inserido",
     bool(((dataset["classe"] == 1) == dataset["sintetico"]).all() and (inseridos == (dataset["tipo_anomalia"] != "nenhuma")).all() and
(inseridos == (dataset["trecho_inserido"] != "")).all()), "nenhum sintético sem trecho"),
    ("Padrão inserido existe em 100% das anomalias", bool(sint.apply(padrao_presente, axis=1).all()), f"{len(sint):,} anomalias"),
    ("Trecho registrado coincide com as posições gravadas", bool(sint.apply(lambda l: l["texto_modelo"][l["ini_anomalia"]:l["fim_anomalia"]]
== l["trecho_inserido"], axis=1).all()), "início/fim corretos"),
```

```
    ("Remover o trecho recupera o texto limpo original", bool((reconstruidos == sint["texto_original"].map(limpar)).all()), "reconstrução
exata"),
    ("Classe 0: texto do classificador = texto original limpo", bool((normal["texto_modelo"] == normal["texto_original"].map(limpar)).all()),
"sem alteração indevida"),
    ("Textos originais idênticos aos carregados na Fase 2", bool((originais.loc[dataset["id_origem"]].to_numpy() ==
dataset["texto_original"].to_numpy()).all()) and pd.util.hash_pandas_object(df["texto"], index=False).sum() == HASH_TEXTOS, "hash do df
conferido"),
    ("Anomalia termina dentro da janela segura", bool((palavras_ate_fim <= JANELA_SEGURA).all()), f"máx. = {palavras_ate_fim.max()} palavras
(janela {JANELA_SEGURA})"),
    ("Repetição de palavra supera a repetição natural (P99 do treino)", bool((reps_palavra - 1 > p99_natural).all()), f"mín.
{reps_palavra.min() - 1} pares consecutivos > P99 = {p99_natural:.0f}"),
    ("Nenhum trecho domina o seu tipo (< 2%)", bool((fracao_unica < 0.02).all()), f"maior repetição: {fracao_unica.max():.2%}"),
    ("Mecanismos e trechos da generalização ausentes do treino, validação e teste",
     bool(set(dataset.loc[ger & inseridos, "variante"]).isdisjoint(dataset.loc[~ger & inseridos, "variante"]) and set(dataset.loc[ger &
inseridos, "trecho_inserido"]).isdisjoint(dataset.loc[~ger & inseridos, "trecho_inserido"])), "variantes e trechos disjuntos"),
    ("Reprodutível com a mesma semente", hash_ds(dataset) == hash_ds(dataset_2), "hashes idênticos"),
    ("Semente diferente altera as anomalias", bool((dataset_3.loc[dataset_3["classe"] == 1, "texto_modelo"].to_numpy() !=
dataset.loc[dataset["classe"] == 1, "texto_modelo"].to_numpy()).mean() > 0.9), "> 90% dos textos mudam"),
]
resultado = pd.DataFrame(verificacoes, columns=["verificação", "ok", "detalhe"])
display(resultado)
assert resultado["ok"].all(), resultado[~resultado["ok"]]
```

	verificação	ok	detalhe
0	Identificadores válidos, únicos, presentes no ...	True	49,564 linhas
1	Nenhum texto equivalente (chave normalizada) s...	True	duplicatas removidas
2	Nenhum grupo de quase-duplicatas atravessa par...	True	agrupamento por início/fim
3	Treino e validação só do treino original; test...	True	sem transferência entre origens
4	Exclusões rastreáveis: ausentes do dataset e l...	True	12 exclusões
5	Sem textos vazios; texto do classificador sem ...	True	0 vazios
6	Classes equilibradas por partição e sentimento...	True	diferença máx. = 1
7	Tipos equilibrados por partição e sentimento (...	True	diferença máx. = 1
8	% de positivos igual nas duas classes (diferen...	True	p-valores qui-quadrado: [1.0, 1.0, 1.0, 1.0]
9	Rótulos coerentes: classe, sintetico, tipo e t...	True	nenhum sintético sem trecho
10	Padrão inserido existe em 100% das anomalias	True	24,780 anomalias
11	Trecho registrado coincide com as posições gra...	True	início/fim corretos
12	Remover o trecho recupera o texto limpo original	True	reconstrução exata
13	Classe 0: texto do classificador = texto origi...	True	sem alteração indevida
14	Textos originais idênticos aos carregados na F...	True	hash do df conferido
15	Anomalia termina dentro da janela segura	True	máx. = 199 palavras (janela 200)
16	Repetição de palavra supera a repetição natura...	True	mín. 7 pares consecutivos > P99 = 3
17	Nenhum trecho domina o seu tipo (< 2%)	True	maior repetição: 1.93%
18	Mecanismos e trechos da generalização ausentes...	True	variantes e trechos disjuntos
19	Reprodutível com a mesma semente	True	hashes idênticos
20	Semente diferente altera as anomalias	True	> 90% dos textos mudam

```
URL_RX = r"www\.|https?:|\b[\w-]+\.(?:com|net|org|biz|info|tv|co|xyz)\b"
POSITIVAS = {"great", "good", "best", "excellent", "amazing", "love", "loved", "wonderful", "perfect", "brilliant", "superb", "fantastic"}
NEGATIVAS = {"bad", "worst", "awful", "terrible", "boring", "waste", "poor", "stupid", "horrible", "dull", "disappointing", "mediocre"}


def polaridade(texto):
    tokens = tokenizar(texto)
    return 100 * (sum(t in POSITIVAS for t in tokens) - sum(t in NEGATIVAS for t in tokens)) / max(len(tokens), 1)


dev = dataset[em_treino_val].copy()    # diagnóstico apenas com treino e validação
assert set(dev["particao"]) == {"treino", "validacao"}, "o diagnóstico não pode usar teste nem generalização"
t = dev["texto_modelo"]
carac = pd.DataFrame({
    "palavras (texto do classificador)": t.str.count(" ") + 1,
    "palavras (texto original)": dev["texto_original"].str.split().str.len(),
    "caracteres": t.str.len(),
    "contém URL": t.str.contains(URL_RX, case=False),
    "dígitos": t.str.count(r"\d"),
    "exclamações": t.str.count("!"),
    "fração de letras maiúsculas": t.str.count(r"[A-Z]") / t.str.count(r"[A-Za-z]"),
    "palavra em MAIÚSCULAS (4+ letras)": t.str.contains(r"\b[A-Z]{4,}\b"),
    "caractere não ASCII": t.str.contains(r"[^\x00-\x7f]"),
    "começa com maiúscula": t.str.match(r"[A-Z]"),
    "termina com . ! ?": t.str.contains(r"[.!?]$"),
    "polaridade lexical": t.map(polaridade),
    "<br /> remanescente": t.str.contains("<br", regex=False),
}).astype(float)


def auc(coluna, mascara=None):
    m = slice(None) if mascara is None else mascara
    return roc_auc_score(dev["classe"][m], carac[coluna][m])


linhas = {c: {"média (classe 0)": carac.loc[dev["classe"] == 0, c].mean(), "média (classe 1)": carac.loc[dev["classe"] == 1, c].mean(),
              "AUC": auc(c), "AUC só positivos": auc(c, dev["sentimento"] == "positivo"), "AUC só negativos": auc(c, dev["sentimento"] ==
"negativo")}
          for c in carac.columns}
```

```
atalhos = pd.DataFrame(linhas).T
atalhos["|AUC - 0,5| >= 0,10"] = (atalhos["AUC"] - 0.5).abs() >= 0.10
display(atalhos)

print("Tipo de anomalia por sentimento (% de positivos em cada tipo):")
display((dev[dev["classe"] == 1].groupby("tipo_anomalia")["sentimento"].apply(lambda s: (s == "positivo").mean() * 100)).to_frame("% positivos"))
```

	média (classe 0)	média (classe 1)	AUC	AUC só positivos	AUC só negativos	|AUC - 0,5| >= 0,10
palavras (texto do classificador)	230.10	246.90	0.54	0.54	0.54	False
palavras (texto original)	232.37	235.68	0.50	0.50	0.50	False
caracteres	1,295.34	1,387.98	0.54	0.54	0.54	False
contém URL	0.01	0.21	0.60	0.60	0.60	False
dígitos	3.27	4.44	0.60	0.59	0.60	False
exclamações	0.98	1.42	0.55	0.56	0.55	False
fração de letras maiúsculas	0.04	0.04	0.51	0.52	0.51	False
palavra em MAIÚSCULAS (4+ letras)	0.22	0.27	0.52	0.52	0.52	False
caractere não ASCII	0.08	0.09	0.50	0.50	0.50	False
começa com maiúscula	0.93	0.93	0.50	0.50	0.50	False
termina com . ! ?	0.86	0.84	0.49	0.49	0.49	False
polaridade lexical	0.44	0.40	0.49	0.49	0.50	False
** remanescente**	0.00	0.00	0.50	0.50	0.50	False

```
Tipo de anomalia por sentimento (% de positivos em cada tipo):
```

tipo_anomalia	% positivos
manipulacao_avaliacoes	50.08
repeticao_excessiva	50.07
spam_promocional	50.07

Interpretação dos resultados.

- **Integridade:** as 21 verificações passam. Não há vazamento por `id_origem`, texto equivalente ou grupo de quase-duplicatas; treino e validação só têm avaliações do treino original, e teste e generalização só do teste original; as 12 exclusões estão ausentes do dataset e ligadas a registros do treino presentes; classes, tipos e sentimentos estão equilibrados (qui-quadrado entre classe e sentimento com p = 1,0 em todas as partições); o padrão está presente em 100% das 24.780 anomalias; os originais estão inalterados; o resultado é idêntico com a mesma semente e diferente com outra.
- **Comprimento:** a seleção não depende do tamanho original (AUC 0,50). O texto da classe 1 é em média 17 palavras mais longo (247 contra 230): cerca de 13 vêm da inserção e o restante de uma diferença casual no tamanho original das avaliações sorteadas (236 contra 232). A AUC do comprimento é 0,54.
- **Atalhos corrigidos:** na primeira versão do diagnóstico, o número de exclamações tinha AUC 0,61 e palavras em MAIÚSCULAS tinham 0,56. Os modelos de mensagem foram ajustados com variação de pontuação (metade dos "!" vira ".") e menos texto em caixa alta, resultando em AUC 0,55 e 0,52. O diagnóstico usou apenas treino e validação.
- **Atalhos remanescentes:** a presença de URL (21% das anomalias contra cerca de 1% das normais) e a contagem de dígitos têm AUC 0,60. São marcadores legítimos dos próprios padrões (links, notas "10/10"), então não foram removidos, mas um modelo poderia depender deles. A generalização inclui anomalias sem link e sem dígitos para testar isso.
- **Sentimento e formatação:** a polaridade lexical praticamente não distingue as classes (AUC 0,49), cada tipo tem cerca de 50% de avaliações positivas, e a formatação (`<br />`, caracteres não ASCII, início e fim de frase) tem AUC entre 0,49 e 0,50. Não há dependência evidente entre classe e sentimento nem entre classe e formatação.

3.6 Teste complementar de generalização

Plano. Os 1.200 grupos reservados do teste original formam a partição `generalizacao`: 600 avaliações normais e 600 com anomalias de mecanismos que **não aparecem no treino** (200 por categoria, 13 mecanismos, em ambos os sentimentos). Essa partição não será usada para treinar, ajustar hiperparâmetros, escolher limiar ou parar o treinamento. Depois do modelo final, ela será avaliada **uma vez**, para medir o recall por categoria e

por mecanismo e a taxa de falsos positivos nas 600 normais. A diferença entre o recall no teste padrão e na generalização indicará o quanto o modelo memorizou os modelos de mensagem.

Por que não é apenas troca de palavras. As variações mudam o mecanismo e não só o vocabulário: promoção escondida em relato pessoal, outros idiomas, link ofuscado, repetição em nível de caractere ou de sequências sem sentido, pedidos indiretos de nota, revelação de pagamento e campanhas coordenadas. A célula abaixo mede o quanto os trechos inéditos diferem dos do treino.

Isolamento (verificado automaticamente em 3.5). A partição usa apenas avaliações do teste original; as variantes e os trechos inéditos não aparecem em treino, validação nem teste; e o diagnóstico de atalhos recebe uma asserção que impede o uso do teste e da generalização. Os modelos de geração dos mecanismos inéditos foram escritos antes de qualquer resultado de modelo, e nenhuma decisão de ajuste pode usar esta partição.

```
def palavras_de(texto):
    return re.findall(r"\w+", texto.lower())


def trigramas(textos):
    return {tuple(p[i:i + 3]) for t in textos for p in [palavras_de(t)] for i in range(len(p) - 2)}


conhecidos = dataset[(dataset["classe"] == 1) & dataset["particao"].isin(["treino", "validacao"])]
inedito = dataset[(dataset["classe"] == 1) & (dataset["particao"] == "generalizacao")]
linhas = {}
for tipo in TIPOS:
    a, b = conhecidos.query("tipo_anomalia == @tipo")["trecho_inserido"], inedito.query("tipo_anomalia == @tipo")["trecho_inserido"]
    voc_a = {w for t in a for w in palavras_de(t)} - ENGLISH_STOP_WORDS
    voc_b = {w for t in b for w in palavras_de(t)} - ENGLISH_STOP_WORDS
    tri_b = trigramas(b)
    linhas[tipo] = {"trechos (treino+val)": len(a), "trechos (generalização)": len(b),
                    "vocabulário inédito (sem stopwords)": len(voc_b - voc_a) / len(voc_b),
                    "trigramas inéditos": len(tri_b - trigramas(a)) / max(len(tri_b), 1),
                    "variantes inéditas": inedito.query("tipo_anomalia == @tipo")["variante"].nunique()}
pd.DataFrame(linhas).T
```

	trechos (treino+val)	trechos (generalização)	vocabulário inédito (sem stopwords)	trigramas inéditos	variantes inéditas
spam_promocional	4,150.00	200.00	0.82	0.99	4.00
repeticao_excessiva	4,150.00	200.00	0.94	0.99	4.00
manipulacao_avaliacoes	4,149.00	200.00	0.77	0.99	5.00

Interpretação. Entre 77% e 94% do vocabulário dos trechos inéditos (sem stopwords) não aparece nos trechos do treino, e 99% dos trigramas de palavras são novos, em todas as categorias. Algum vocabulário comum permanece (por exemplo, "review" ou "movie") porque o contexto é o mesmo.

Limitações dos mecanismos sintéticos.

- Os mecanismos inéditos foram criados pelo próprio grupo e refletem a sua imaginação, e não anomalias reais; a novidade é relevante, mas não é total.
- São apenas 13 mecanismos, com poucos idiomas além do inglês (português, espanhol e alemão), e todos são inserções contínuas de um único trecho; spam real é mais variado e pode estar espalhado pelo texto.
- A repetição em nível de caractere e as sequências sem sentido viram termos raros após a tokenização, o que pode facilitar ou dificultar a detecção conforme o vocabulário.
- Se algum resultado desta partição influenciar decisões de ajuste, ela deixa de ser um teste inédito.

3.7 Conclusões da Etapa 03

Critérios de conclusão.

Critério	Situação
Duplicatas com tratamento documentado	424 cópias removidas, 12 registros do teste excluídos por quase-duplicata do treino, 42 grupos de quase-duplicatas preservados juntos
Partições livres de vazamento conhecido	verificado automaticamente (21 verificações); nenhum registro do teste original no treino ou na validação

Critério	Situação
Três categorias implementadas	spam promocional, repetição excessiva e manipulação de avaliações
Classes e sentimentos validados	equilibrados em todas as partições
Anomalias reproduzíveis	semente 42, resultado idêntico em duas execuções
Dataset pronto para tokenização e *padding*	coluna `texto_modelo` e rótulo `classe`
Execução sem erros até aqui	o notebook foi executado do início ao fim

Riscos e limitações.

1. **Anomalias sintéticas:** o desempenho medirá o reconhecimento dos padrões inseridos, e não a detecção de fraude real.
2. **Aprendizado de modelos de mensagem:** os trechos de treino vêm de um número finito de modelos escritos pelo grupo. A variação reduz, mas não elimina, o risco de memorização, que a partição de generalização ajuda a medir.
3. **Marcadores correlacionados:** URL e dígitos permanecem associados à classe 1 (AUC 0,60).
4. **Prevalência artificial:** 50% de anomalias não reflete a realidade; as métricas deverão ser lidas com isso em mente.
5. **Quase-duplicatas não detectadas:** o agrupamento usa início e fim de 20 tokens; paráfrases mais livres e avaliações do mesmo autor com texto diferente não são detectadas.
6. **Filmes compartilhados:** segundo a documentação do corpus, o treino e o teste originais têm filmes distintos, mas a validação é sorteada do treino e pode compartilhar filmes com ele. Como a classe é atribuída independentemente do filme, o efeito esperado sobre a tarefa é pequeno, mas não foi medido.
7. **Janela segura:** a garantia de que a anomalia fica dentro da janela depende da tokenização final; ela precisa ser conferida com `fim_anomalia` na próxima etapa.
8. **Classe 0 presumida:** pode conter spam real não identificado, como já registrado na Fase 1.

Recomendações para a próxima etapa (tokenização e *padding*).

- Usar `texto_modelo` como entrada e `classe` como rótulo; ajustar vocabulário e tokenizador **somente com a partição** `treino`.
- Tratar o comprimento máximo e o vocabulário como hipóteses a testar (256 a 400 palavras; 10.000 a 20.000 termos), validando-os na partição `validacao`.
- Truncar mantendo o **início** do texto e conferir, com `fim_anomalia` e o tokenizador escolhido, que nenhuma anomalia fica fora da janela.
- Manter a partição `generalizacao` intocada até a avaliação final.
- Ao avaliar, reportar também resultados por tipo e por variante de anomalia.

3.8 Registro da Etapa 03B — correção da política de divisão

Problema. A auditoria da Etapa 03 mostrou que, na primeira versão, os grupos de quase-duplicatas que reuniam avaliações do treino e do teste originais eram atribuídos ao treino. Com isso, **12 avaliações do teste original foram movidas para o treino**, contrariando a premissa de preservar o teste original.

Correção. Nesses grupos, preserva-se o treino e **excluem-se os registros do teste** (seção 3.1). Cada exclusão fica registrada em uma tabela com o grupo e os registros do treino correspondentes, e a divisão agora usa a partição original de cada avaliação. Foram acrescentadas asserções de origem na célula de divisão e revistas as validações, sem alterar os mecanismos de geração das anomalias.

Item	Etapa 03	Etapa 03B
Avaliações mantidas	49.576	49.564
Avaliações do teste original no treino ou na validação	12	0
Registros do teste excluídos por quase-duplicata do treino	0	12
Grupos de quase-duplicatas (2 ou mais avaliações)	48	42
treino	21.174	21.164

Item	Etapa 03	Etapa 03B
validacao	3.739	3.737
teste	23.463	23.463
generalizacao	1.200	1.200
Anomalias (classe 1)	24.787	24.780
Verificações automáticas	19	21

As partições `teste` e `generalizacao` permanecem **idênticas** às da Etapa 03 (mesmo conteúdo e mesmos rótulos, conferido por *hash*); só `treino` e `validacao` mudaram, porque deixaram de receber as 12 avaliações e os sorteios dessas partições foram refeitos. Nas validações, foram acrescentadas as verificações de origem das partições, de rastreabilidade das exclusões, de consistência dos identificadores e dos rótulos, de normalização do texto do classificador e de isolamento da generalização, e foram fundidas verificações redundantes (identificador único com validade dos identificadores; ausência de vazios com normalização do texto).

Fase 3 (continuação) — Etapa 04: pré-processamento e tokenização

Esta etapa transforma `texto_modelo` em sequências numéricas de tamanho fixo para a LSTM, com `classe` como rótulo. O vocabulário é ajustado **exclusivamente com a partição** `treino`. Nenhum modelo é treinado, e o dataset supervisionado não é alterado.

Como as decisões são tomadas. Os candidatos e as regras de escolha abaixo foram definidos **antes** de olhar os resultados. A escolha usa apenas treino e validação; teste e generalização só são vetorizados depois, para verificação e relatório.

Item	Definição
Candidatos	vocabulário de 10.000 ou 20.000 tokens (incluindo `[PAD]` e `[UNK]`); comprimento de 256 ou 400 posições
Regra para o comprimento	o menor comprimento que mantém, no máximo, 0,1% dos trechos inseridos (treino + validação) fora da janela
Regra para o vocabulário	o menor vocabulário cujo percentual de tokens desconhecidos dentro dos trechos inseridos (validação) não ultrapasse em mais de 1 ponto percentual o do maior
Dados usados na escolha	treino e validação
Dados apenas medidos	teste e generalização

4.1 Consistência das contagens da Etapa 03B

A primeira célula confere o dataset contra os números documentados na seção 3.8 (contagens por partição, cópias removidas, exclusões e origem das partições) e define os parâmetros desta etapa. Uma divergência interrompe a execução.

```
import tensorflow as tf

VOCABULARIOS = [10_000, 20_000]
COMPRIMENTOS = [256, 400]
MAX_PERDA_ANOMALIA = 0.001   # regra de escolha do comprimento: no máximo 0,1% das anomalias fora da janela
MAX_AUMENTO_UNK = 0.01       # regra de escolha do vocabulário: aceitar o menor se o UNK nos trechos subir até 1 p.p.
HASH_DATASET = pd.util.hash_pandas_object(dataset[["id_origem", "texto_modelo", "classe", "particao"]], index=False).sum()

documentado = {"treino": 21_164, "validacao": 3_737, "teste": 23_463, "generalizacao": 1_200}
tabela = pd.crosstab(dataset["particao"], dataset["classe"]).loc[ordem]
tabela.columns = ["classe 0", "classe 1"]
tabela.insert(0, "documentado (3.8)", pd.Series(documentado))
tabela["total no dataset"] = tabela["classe 0"] + tabela["classe 1"]
display(tabela)

assert tabela["total no dataset"].to_dict() == documentado, "contagens diferentes das documentadas na Etapa 03B"
assert len(df) - len(removidas) - len(excluidas) == len(dataset) == 49_564 and (len(removidas), len(excluidas)) == (424, 12)
assert dataset.loc[dataset["particao"].isin(["treino", "validacao"]), "particao_original"].eq("train").all()
assert dataset.loc[dataset["particao"].isin(["teste", "generalizacao"]), "particao_original"].eq("test").all()
pd.Series({"avaliações originais": len(df), "- cópias equivalentes removidas": len(removidas),
           "- registros do teste excluídos (quase-duplicata do treino)": len(excluidas), "= avaliações no dataset": len(dataset),
           "  treino + validação (origem: treino original)": int(tabela.loc[["treino", "validacao"], "total no dataset"].sum()),
           "  teste + generalização (origem: teste original)": int(tabela.loc[["teste", "generalizacao"], "total no dataset"].sum())},
name="quantidade").to_frame()
```

particao	documentado (3.8)	classe 0	classe 1	total no dataset
treino	21164	10583	10581	21164
validacao	3737	1869	1868	3737
teste	23463	11732	11731	23463
generalizacao	1200	600	600	1200

	quantidade
avaliações originais	50000
- cópias equivalentes removidas	424
- registros do teste excluídos (quase-duplicata do treino)	12
= avaliações no dataset	49564
treino + validação (origem: treino original)	24901
teste + generalização (origem: teste original)	24663

Interpretação. As contagens coincidem com as documentadas: 50.000 avaliações originais, menos 424 cópias equivalentes e 12 registros do teste excluídos, resultam em **49.564** avaliações. Treino e validação somam 24.901 (origem: treino original), e teste e generalização somam 24.663 (origem: teste original).

4.2 Padronização e tokenização

Ferramenta. Foi usada a camada `TextVectorization` do Keras, por quatro motivos ligados à explicabilidade futura: recebe **texto bruto**, o que permite chamá-la a partir do `classifier_fn` do LIME e da interface Gradio; a padronização é feita com operações do TensorFlow, de modo que pode ser embutida no modelo; o vocabulário é ajustado com `adapt` somente nos dados fornecidos; e o truncamento mantém o **início** do texto, com *padding* à direita, compatível com a máscara da LSTM.

Padronização (função `padronizar`):

- converte para minúsculas e remove `<br />`;
- normaliza aspas e apóstrofos tipográficos;
- **mantém contrações** como um único token (*don't*, *isn't*), além de acentos e dígitos;
- transforma `! ? % $ @ /` em **tokens próprios**, porque são sinais de spam e de manipulação (promoções, contatos, notas "10/10");
- converte as demais pontuações em espaço, de modo que um endereço vira partes como `www`, nome e `com`.

Stopwords não são removidas. As negações (*not*, *no*, *never*, *nor*, *cannot*) e as palavras funcionais fazem parte dos sinais que o modelo precisa ver, e a repetição excessiva frequentemente envolve palavras funcionais. A remoção de stopwords é citada no enunciado como exemplo de limpeza, mas aqui é uma decisão consciente do grupo.

Palavras desconhecidas recebem o índice 1 (`[UNK]`), e o *padding* usa o índice 0.

As duas células a seguir ajustam um vocabulário completo apenas com o treino, mostram um exemplo de tokenização e verificam a presença dos sinais relevantes.

```
@keras.saving.register_keras_serializable(package="cp2")
def padronizar(textos):
    """Minúsculas; !?%$@/ viram tokens próprios; apóstrofos internos e acentos são mantidos; demais pontuações viram espaço."""
    t = tf.strings.lower(textos, encoding="utf-8")
    t = tf.strings.regex_replace(t, r"<br\s*/?>", " ")
    t = tf.strings.regex_replace(t, "[’‘`´]", "'")
    t = tf.strings.regex_replace(t, r"([!?%$@/])", r" \1 ")
    t = tf.strings.regex_replace(t, r"[^\pL\pN\s'_!?%$@/]", " ")
    return tf.strings.regex_replace(t, r"(^|\s)'+|'+(\s|$)", " ")


def criar_camada(max_tokens=None, comprimento=None):
    return keras.layers.TextVectorization(max_tokens=max_tokens, standardize=padronizar, split="whitespace",
                                          output_mode="int", output_sequence_length=comprimento)
```

```
def em_lotes(textos, tamanho=1024):
    return tf.data.Dataset.from_tensor_slices(np.asarray(list(textos), dtype=object)).batch(tamanho)


P = {p: dataset[dataset["particao"] == p] for p in ordem}
completa = criar_camada()                                  # vocabulário completo, ajustado só com o treino
completa.adapt(em_lotes(P["treino"]["texto_modelo"]))
vocab_completo = [str(p) for p in completa.get_vocabulary()]


def ids_completos(textos, lote=512):
    """Sequências completas (sem truncar) como lista de arrays; 0 = padding, 1 = desconhecido, 2+ = palavras do treino."""
    saida = []
    for i in range(0, len(textos), lote):
        bloco = completa(tf.constant(textos[i:i + lote])).numpy()
        saida += [linha[:np.count_nonzero(linha)] for linha in bloco]
    return saida


ids_dev = {p: ids_completos(list(P[p]["texto_modelo"])) for p in ("treino", "validacao")}   # só treino e validação nesta fase
print(f"Vocabulário completo do treino: {len(vocab_completo) - 2:,} palavras distintas (fora [PAD] e [UNK])")
exemplo = P["treino"]["texto_modelo"].iloc[0]
print("\nTexto:", exemplo[:150], "...")
print("Tokens:", [vocab_completo[i] for i in ids_dev["treino"][0][:30]])
```

```
Vocabulário completo do treino: 77,642 palavras distintas (fora [PAD] e [UNK])

Texto: Airport '77 starts as a brand new luxury 747 plane is loaded up with valuable paintings & such belonging to rich businessman Philip
Stevens (James Ste ...
Tokens: ['airport', '77', 'starts', 'as', 'a', 'brand', 'new', 'luxury', '747', 'plane', 'is', 'loaded', 'up', 'with', 'valuable',
'paintings', 'such', 'belonging', 'to', 'rich', 'businessman', 'philip', 'stevens', 'james', 'stewart', 'who', 'is', 'flying', 'them', 'a']
```

```
contagem_tokens = np.bincount(np.concatenate(ids_dev["treino"]), minlength=len(vocab_completo))
rank = {p: i for i, p in enumerate(vocab_completo)}
sinais = ["not", "no", "never", "nor", "cannot", "don't", "isn't", "can't", "!", "?", "%", "$", "@", "/", "www", "com", "free", "vote", "10"]
display(pd.DataFrame({"ocorrências no treino": [contagem_tokens[rank[s]] if s in rank else 0 for s in sinais],
                      "posição no vocabulário": [rank.get(s, -1) for s in sinais]}, index=sinais).T)

print("Palavras do treino por frequência mínima:")
pd.Series({f"aparecem >= {k} vezes": int((contagem_tokens[2:] >= k).sum()) for k in (1, 2, 5, 10)}, name="palavras").to_frame()
```

	not	no	never	nor	cannot	don't	isn't	can't	!	?	%	$	@	/	www	com	free	vote	10
ocorrências no treino	26588	10814	5495	573	944	7460	2716	3092	25322	13981	1072	979	505	8878	1066	447	1998	1267	5035
posição no vocabulário	22	60	121	918	585	93	217	189	24	48	518	568	1033	75	522	1139	290	439	132

Palavras do treino por frequência mínima:

	palavras
aparecem >= 1 vezes	77642
aparecem >= 2 vezes	47111
aparecem >= 5 vezes	28100
aparecem >= 10 vezes	18856

Interpretação.

- O treino tem **77.642 palavras distintas**; 47.111 aparecem ao menos duas vezes, 28.100 ao menos cinco e 18.856 ao menos dez. A cauda longa de palavras raras confirma a observação da Fase 2.
- As **negações e os sinais estão no vocabulário e entre os mais frequentes**: *not* (26.588 ocorrências), *no*, *never*, *don't*, `!` (25.322), `?`, `/`, além de `%`, `$`, `@`, `www`, `com`, *free*, *vote* e `10`. Nenhum desses sinais foi descartado pela padronização.
- Os dígitos são mantidos como estão (por exemplo, "10", "555"). Isso preserva notas e telefones, ao custo de adicionar números raros ao vocabulário.

4.3 Avaliação dos vocabulários e comprimentos

Para cada combinação de vocabulário (10.000 e 20.000) e comprimento (256 e 400) são medidos, **somente com treino e validação**:

- a taxa de tokens desconhecidos, no texto inteiro e dentro dos trechos inseridos;
- o impacto do truncamento: textos truncados, tokens descartados e posições de *padding*;

- a preservação dos trechos inseridos: íntegros, cortados parcialmente ou perdidos. A posição de cada trecho em tokens é medida pelo próprio tokenizador.

Como o vocabulário é ordenado por frequência, cada vocabulário candidato é obtido a partir do completo ajustado no treino (os demais índices viram `[UNK]`); essa equivalência é verificada em 4.5. Em seguida, as regras da abertura são aplicadas.

```
def limites_tokens(sub):
    """Posição (em tokens) em que cada trecho inserido começa e termina, medida pelo próprio tokenizador."""
    a = sub[sub["classe"] == 1]
    antes = [len(x) for x in ids_completos([t[:i] for t, i in zip(a["texto_modelo"], a["ini_anomalia"])])]
    ate = [len(x) for x in ids_completos([t[:f] for t, f in zip(a["texto_modelo"], a["fim_anomalia"])])]
    return np.array(antes), np.array(ate)


def trechos_ids(ids_lista, sub, antes, ate):
    idx = np.flatnonzero(sub["classe"].to_numpy() == 1)
    return [ids_lista[i][a:b] for i, a, b in zip(idx, antes, ate)]


lim = {p: limites_tokens(P[p]) for p in ("treino", "validacao")}
segmentos = {p: trechos_ids(ids_dev[p], P[p], *lim[p]) for p in ("treino", "validacao")}
tokens_por_texto = {p: np.array([len(x) for x in ids_dev[p]]) for p in ids_dev}
pd.DataFrame({p: pd.Series(tokens_por_texto[p]).describe(percentiles=[0.5, 0.9, 0.95, 0.99]) for p in tokens_por_texto})
```

	treino	validacao
count	21,164.00	3,737.00
mean	242.71	246.28
std	174.70	174.70
min	10.00	17.00
50%	184.00	187.00
90%	469.00	470.80
95%	608.00	627.00
99%	919.00	934.64
max	2,518.00	1,527.00

```
def taxa_unk(ids_lista, V):
    todos = np.concatenate(ids_lista)
    return 100 * np.mean((todos == 1) | (todos >= V))


def unk_trechos(segs, V):
    todos = np.concatenate(segs)
    return 100 * np.mean((todos == 1) | (todos >= V))


def truncamento(n_tokens, L):
    return {"textos truncados (%)": 100 * np.mean(n_tokens > L), "tokens descartados (%)": 100 * np.maximum(n_tokens - L, 0).sum() /
n_tokens.sum(),
            "posições de padding (%)": 100 * (1 - np.minimum(n_tokens, L).sum() / (len(n_tokens) * L))}


def preservacao(antes, ate, L):
    return {"trechos íntegros (%)": 100 * np.mean(ate <= L), "trechos parciais (%)": 100 * np.mean((antes < L) & (ate > L)), "trechos
perdidos (%)": 100 * np.mean(antes >= L)}


antes_dev, ate_dev = np.concatenate([lim["treino"][0], lim["validacao"][0]]), np.concatenate([lim["treino"][1], lim["validacao"][1]])
linhas = []
for V in VOCABULARIOS:
    for L in COMPRIMENTOS:
        linhas.append({"vocabulário": V, "comprimento": L,
                       "UNK treino (%)": taxa_unk(ids_dev["treino"], V), "UNK validação (%)": taxa_unk(ids_dev["validacao"], V),
                       "UNK nos trechos, validação (%)": unk_trechos(segmentos["validacao"], V),
                       **truncamento(tokens_por_texto["validacao"], L), **preservacao(antes_dev, ate_dev, L)})
avaliacao = pd.DataFrame(linhas)
display(avaliacao.set_index(["vocabulário", "comprimento"]).T)

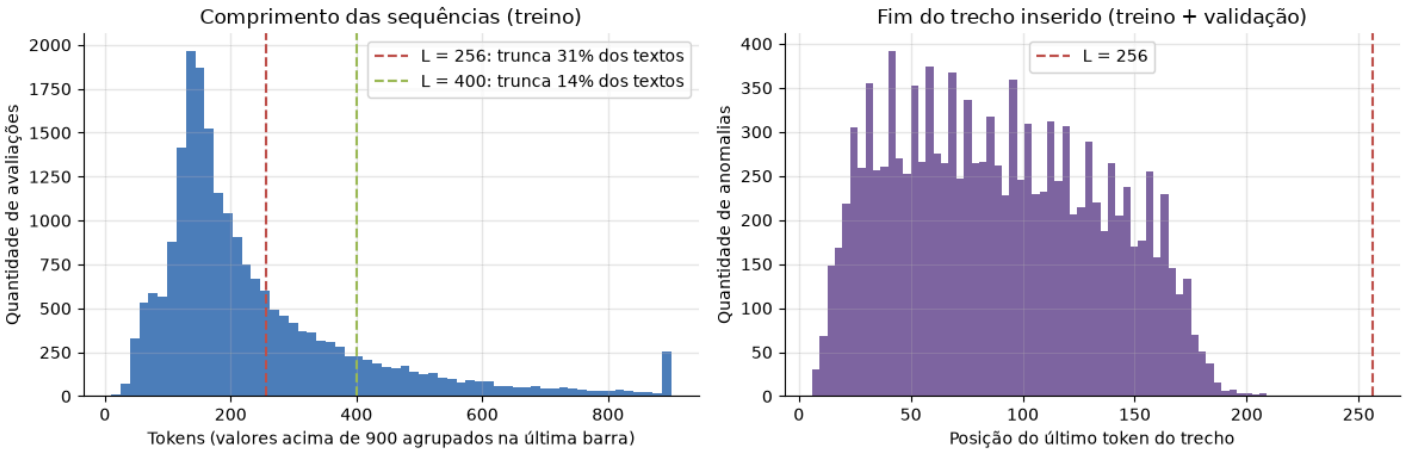
# regras decididas antes de olhar os resultados, aplicadas apenas a treino e validação
unk_ref = unk_trechos(segmentos["validacao"], max(VOCABULARIOS))
V_ESCOLHIDO = min(V for V in VOCABULARIOS if unk_trechos(segmentos["validacao"], V) - unk_ref <= 100 * MAX_AUMENTO_UNK)
L_ESCOLHIDO = min((L for L in COMPRIMENTOS if preservacao(antes_dev, ate_dev, L)["trechos perdidos (%)"] + preservacao(antes_dev, ate_dev, L)
["trechos parciais (%)"] <= 100 * MAX_PERDA_ANOMALIA), default=max(COMPRIMENTOS))
print(f"Configuração escolhida pelas regras: vocabulário = {V_ESCOLHIDO:,} | comprimento = {L_ESCOLHIDO}")
```

vocabulário	10000		20000	
comprimento	256	400	256	400
UNK treino (%)	5.51	5.51	2.57	2.57
UNK validação (%)	5.87	5.87	3.00	3.00
UNK nos trechos, validação (%)	3.74	3.74	2.65	2.65
textos truncados (%)	32.03	14.00	32.03	14.00
tokens descartados (%)	23.99	11.36	23.99	11.36
posições de padding (%)	26.88	45.43	26.88	45.43
trechos íntegros (%)	100.00	100.00	100.00	100.00
trechos parciais (%)	0.00	0.00	0.00	0.00
trechos perdidos (%)	0.00	0.00	0.00	0.00

```
Configuração escolhida pelas regras: vocabulário = 20,000 | comprimento = 256
```

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
n_treino = tokens_por_texto["treino"]
ax1.hist(np.clip(n_treino, 0, 900), bins=60, color="#4f81bd")
for L, cor in zip(COMPRIMENTOS, ("#c0504d", "#9bbb59")):
    ax1.axvline(L, color=cor, linestyle="--", label=f"L = {L}: trunca {100 * np.mean(n_treino > L):.0f}% dos textos")
ax1.set_title("Comprimento das sequências (treino)")
ax1.set_xlabel("Tokens (valores acima de 900 agrupados na última barra)")
ax1.set_ylabel("Quantidade de avaliações")
ax1.legend()

ax2.hist(ate_dev, bins=60, color="#8064a2")
ax2.axvline(COMPRIMENTOS[0], color="#c0504d", linestyle="--", label=f"L = {COMPRIMENTOS[0]}")
ax2.set_title("Fim do trecho inserido (treino + validação)")
ax2.set_xlabel("Posição do último token do trecho")
ax2.set_ylabel("Quantidade de anomalias")
ax2.legend()
plt.tight_layout()
plt.show()
```



Interpretação.

- **Comprimento:** a mediana é de 184 tokens no treino, mas a cauda é longa (P95 de 608). Com 256 posições, 32% dos textos de validação são truncados e 24% dos tokens são descartados; com 400, são 14% e 11%. Em compensação, o *padding* ocupa 27% das posições com 256 e 45% com 400.
- **Trechos inseridos:** ficam **100% íntegros** nas duas janelas. Isso decorre do desenho da Etapa 03 (a anomalia termina até a 200ª palavra) e foi confirmado com o tokenizador: o último token de um trecho está na posição 209 no treino e 206 na validação, longe do limite de 256 (a seção 4.4 inclui o teste e a generalização). Como a regra escolhe o menor comprimento que preserva os trechos, o resultado é **256**.
- **Vocabulário:** com 10.000 tokens, 5,9% dos tokens da validação são desconhecidos, contra 3,0% com 20.000. Dentro dos trechos inseridos, o desconhecido vai de 2,65% (20.000) para 3,74% (10.000), uma diferença de **1,09 ponto percentual, pouco acima do limite de 1,0** da regra. O resultado é **20.000**, mas a margem é estreita: com 10.000 a escolha seria defensável, e a decisão deve ser tratada como hipótese a revisar na modelagem, usando apenas treino e validação.

4.4 Configuração final e dados preparados

Com a configuração escolhida, a camada é ajustada novamente **apenas com o treino** e usada para vetorizar as quatro partições. `vetorizar(textos)` recebe um texto ou uma lista de textos brutos e devolve uma matriz `int32` de formato `(n, 256)`; `decodificar(ids)` faz o caminho inverso, para depuração. Os dados ficam em `X[partição]` e `y[partição]`. O teste e a generalização são medidos aqui apenas para relatório, sem influenciar a escolha.

```
camada = criar_camada(V_ESCOLHIDO, L_ESCOLHIDO)
camada.adapt(em_lotes(P["treino"]["texto_modelo"]))          # ajuste exclusivamente com o treino
vocabulario = [str(p) for p in camada.get_vocabulary()]


def vetorizar(textos, lote=1024):
    """Texto(s) bruto(s) -> matriz int32 (n, comprimento). Serve de entrada para a LSTM e para classifier_fn do LIME / Gradio."""
    textos = [textos] if isinstance(textos, str) else list(textos)
    if not textos:
        return np.zeros((0, L_ESCOLHIDO), dtype="int32")
    return np.concatenate([camada(tf.constant(textos[i:i + lote])).numpy() for i in range(0, len(textos), lote)]).astype("int32")


def decodificar(ids):
    return [vocabulario[i] for i in ids if i != 0]


X = {p: vetorizar(list(P[p]["texto_modelo"])) for p in ordem}
y = {p: P[p]["classe"].to_numpy(dtype="int8") for p in ordem}
ids_todas = {**ids_dev, **{p: ids_completos(list(P[p]["texto_modelo"])) for p in ("teste", "generalizacao")}}   # só para medir truncamento e UNK
lim.update({p: limites_tokens(P[p]) for p in ("teste", "generalizacao")})

segs = {p: trechos_ids(ids_todas[p], P[p], *lim[p]) for p in ordem}
linhas = {}
for p in ordem:
    n_tok = np.array([len(x) for x in ids_todas[p]])
    antes, ate = lim[p]
    linhas[p] = {"X": str(X[p].shape), "y": str(y[p].shape), "% classe 1": 100 * y[p].mean(), "UNK (%)": taxa_unk(ids_todas[p], V_ESCOLHIDO),
                 "UNK nos trechos (%)": unk_trechos(segs[p], V_ESCOLHIDO), **truncamento(n_tok, L_ESCOLHIDO),
                 **preservacao(antes, ate, L_ESCOLHIDO), "fim máx. do trecho (token)": int(ate.max())}
display(pd.DataFrame(linhas).T)

tipos = {p: P[p].loc[P[p]["classe"] == 1, "tipo_anomalia"].to_numpy() for p in ordem}
print("UNK dentro dos trechos inseridos (%), por tipo de anomalia:")
pd.DataFrame({p: {t: unk_trechos([s for s, tp in zip(segs[p], tipos[p]) if tp == t], V_ESCOLHIDO) for t in TIPOS} for p in ("validacao",
"teste", "generalizacao")})
```

	X	y	% classe 1	UNK (%)	UNK nos trechos (%)	textos truncados (%)	tokens descartados (%)	posições de padding (%)	trechos íntegros (%)	trechos parciais (%)	trechos perdidos (%)	fim máx. do trecho (token)
treino	(21164, 256)	(21164,)	50.00	2.57	2.16	30.69	23.88	27.83	100.00	0.00	0.00	209
validacao	(3737, 256)	(3737,)	49.99	3.00	2.65	32.03	23.99	26.88	100.00	0.00	0.00	206
teste	(23463, 256)	(23463,)	50.00	3.32	2.94	29.69	22.93	28.35	100.00	0.00	0.00	213
generalizacao	(1200, 256)	(1200,)	50.00	3.83	21.31	29.17	22.35	29.03	100.00	0.00	0.00	180

UNK dentro dos trechos inseridos (%), por tipo de anomalia:

	validacao	teste	generalizacao
spam_promocional	6.36	6.28	19.43
repeticao_excessiva	1.16	1.92	37.52
manipulacao_avaliacoes	0.00	0.00	8.95

Configuração escolhida (hipótese de trabalho, não hiperparâmetro definitivo).

Parâmetro	Valor	Justificativa
Vocabulário	20.000 (19.998 palavras + `[PAD]` + `[UNK]`)	menor UNK nos trechos inseridos na validação (2,65% contra 3,74%); regra aplicada com margem estreita
Comprimento	256	todos os trechos permanecem íntegros; menos *padding* (27% contra 45%) e menor custo do que 400
Truncamento / *padding*	mantém o início / zeros à direita	preserva os trechos (que ficam no início do texto) e é compatível com máscara
Ajuste do vocabulário	somente `treino`	evita vazamento
Desconhecidas	índice 1 (`[UNK]`)	

Dimensões e medidas por partição.

Partição	X	UNK (%)	UNK nos trechos (%)	Textos truncados (%)	Tokens descartados (%)	Trechos íntegros
treino	(21.164, 256)	2,57	2,16	30,7	23,9	100%
validacao	(3.737, 256)	3,00	2,65	32,0	24,0	100%
teste	(23.463, 256)	3,32	2,94	29,7	22,9	100%
generalizacao	(1.200, 256)	3,83	21,31	29,2	22,4	100%

- **Cobertura do vocabulário:** a taxa de desconhecidos cresce do treino (2,57%, já "vistas") para a validação (3,00%) e o teste (3,32%), como esperado para dados novos.
- **Truncamento:** descarta cerca de 23% a 24% dos tokens em todas as partições, mas **nenhum trecho inserido é perdido ou cortado**: o último token de um trecho está, no máximo, na posição 213 (209 no treino, 206 na validação, 213 no teste e 180 na generalização).
- **Generalização:** 21% dos tokens dos trechos inéditos são desconhecidos (repetição: 37,5%; spam: 19,4%; manipulação: 8,9%), contra 2% a 3% nas demais partições. Isso é esperado: os mecanismos inéditos usam outros idiomas, links ofuscados, alongamentos e sequências sem sentido. Esse valor **não** foi usado para escolher o vocabulário; ele indica que o modelo terá de reconhecer esses trechos pelo contexto e pelo padrão de repetição, e não pelas palavras.

4.5 Validações

A célula a seguir executa as verificações automáticas desta etapa (qualquer falha interrompe a execução). Entre elas há uma **sonda de vazamento**: um vocabulário é construído com tudo o que não é treino (validação, teste e generalização), e confere-se que nenhuma palavra exclusiva desses dados entrou no vocabulário final. A sonda é apenas um diagnóstico e não serve para escolher parâmetros.

```
sonda = criar_camada()      # sonda de vazamento: vocabulário de tudo o que NÃO é treino
sonda.adapt(em_lotes(pd.concat([P["validacao"], P["teste"], P["generalizacao"]])["texto_modelo"]))
exclusivas = {str(p) for p in sonda.get_vocabulary()} - set(vocab_completo)

camada_2 = criar_camada(V_ESCOLHIDO, L_ESCOLHIDO)
camada_2.adapt(em_lotes(P["treino"]["texto_modelo"]))
camada_3 = criar_camada(V_ESCOLHIDO, L_ESCOLHIDO)
camada_3.set_vocabulary(vocabulario[2:])                    # reconstrução só a partir da lista de vocabulário

val = P["validacao"]
simulado = np.zeros_like(X["validacao"])
for i, ids in enumerate(ids_dev["validacao"]):
    ids = np.where((ids == 1) | (ids >= V_ESCOLHIDO), 1, ids)[:L_ESCOLHIDO]
    simulado[i, :len(ids)] = ids
amostra = val.sample(200, random_state=SEMENTE)["texto_modelo"].tolist()
indices = [val.index.get_loc(i) for i in val.sample(200, random_state=SEMENTE).index]
nao_adulterado = pd.util.hash_pandas_object(dataset[["id_origem", "texto_modelo", "classe", "particao"]], index=False).sum() == HASH_DATASET
trechos_perdidos = int(sum(((lim[p][0] >= L_ESCOLHIDO) | (lim[p][1] > L_ESCOLHIDO)).sum() for p in ordem))
neg_simbolos = ["not", "no", "never", "nor", "cannot", "don't", "isn't", "!", "?", "%", "$", "@", "/", "www", "com"]

verificacoes = [
    ("Vocabulário contido no vocabulário do treino", set(vocabulario[2:]) <= set(vocab_completo[2:]), f"{len(vocabulario) - 2:,} palavras +
[PAD] + [UNK]"),
    ("Sonda de vazamento: palavras exclusivas de validação/teste/generalização ausentes do vocabulário", exclusivas.isdisjoint(vocabulario),
f"{len(exclusivas):,} palavras exclusivas, nenhuma incluída"),
    ("Dimensões consistentes: (n, L) int32 e índices dentro do vocabulário",
     all(X[p].shape == (len(P[p]), L_ESCOLHIDO) and X[p].dtype == np.int32 and X[p].min() >= 0 and X[p].max() < V_ESCOLHIDO for p in ordem),
f"L = {L_ESCOLHIDO}, V = {V_ESCOLHIDO:,}"),
    ("Rótulos preservados e alinhados às linhas", all((y[p] == P[p]["classe"].to_numpy()).all() and set(np.unique(y[p])) == {0, 1} for p in
ordem), "y igual a classe, na mesma ordem"),
    ("Linhas alinhadas: vetorizar um texto isolado = linha correspondente de X", bool((vetorizar(amostra) == X["validacao"][indices]).all()),
"200 textos conferidos"),
    ("Nenhuma partição misturada", sum(len(P[p]) for p in ordem) == len(dataset) and len(set().union(*[set(P[p]["id_origem"]) for p in
ordem])) == len(dataset), f"{len(dataset):,} avaliações em 4 partições disjuntas"),
    ("Truncamento mantém o início e padding é à direita (equivale à simulação)", bool((simulado == X["validacao"]).all()), "validação
conferida"),
    ("Mesmo vocabulário e mesma saída ao ajustar de novo", camada_2.get_vocabulary() == camada.get_vocabulary() and
bool((camada_2(tf.constant(amostra)).numpy() == vetorizar(amostra)).all()), "reajuste idêntico"),
    ("Reconstrução a partir da lista de vocabulário", bool((camada_3(tf.constant(amostra)).numpy() == vetorizar(amostra)).all()), "saída
idêntica"),
    ("Negações e sinais mantidos no vocabulário", all(s in vocabulario for s in neg_simbolos), ", ".join(neg_simbolos[:5]) + ", !, ?, %, $,
@, /, www, com"),
    ("Trechos inseridos: nenhuma perda ou corte parcial em nenhuma partição", trechos_perdidos == 0, f"{trechos_perdidos} trechos perdidos ou
parciais"),
    ("Dataset supervisionado não foi alterado", bool(nao_adulterado), "hash igual ao do início da etapa"),
]
resultado = pd.DataFrame(verificacoes, columns=["verificação", "ok", "detalhe"])
display(resultado)
assert resultado["ok"].all(), resultado[~resultado["ok"]]
```

	verificação	ok	detalhe
0	Vocabulário contido no vocabulário do treino	True	19,998 palavras + [PAD] + [UNK]
1	Sonda de vazamento: palavras exclusivas de val...	True	37,678 palavras exclusivas, nenhuma incluída
2	Dimensões consistentes: (n, L) int32 e índices...	True	L = 256, V = 20,000
3	Rótulos preservados e alinhados às linhas	True	y igual a classe, na mesma ordem
4	Linhas alinhadas: vetorizar um texto isolado =...	True	200 textos conferidos
5	Nenhuma partição misturada	True	49,564 avaliações em 4 partições disjuntas
6	Truncamento mantém o início e padding é à dire...	True	validação conferida
7	Mesmo vocabulário e mesma saída ao ajustar de ...	True	reajuste idêntico
8	Reconstrução a partir da lista de vocabulário	True	saída idêntica
9	Negações e sinais mantidos no vocabulário	True	not, no, never, nor, cannot, !, ?, %, $, @, /,...
10	Trechos inseridos: nenhuma perda ou corte parc...	True	0 trechos perdidos ou parciais
11	Dataset supervisionado não foi alterado	True	hash igual ao do início da etapa

Interpretação. As 12 verificações passam:

- o vocabulário usa só palavras do treino, e as 37.678 palavras exclusivas de validação, teste e generalização não entraram;
- todas as matrizes têm formato `(n, 256)`, tipo `int32` e índices dentro do vocabulário; os rótulos estão preservados e alinhados, e as quatro partições continuam disjuntas (49.564 avaliações);
- o resultado equivale à simulação a partir do vocabulário completo (a equivalência assumida em 4.3 foi confirmada, e o truncamento mantém o início);
- o reajuste produz o mesmo vocabulário e a mesma saída, e uma camada reconstruída apenas a partir da lista de vocabulário gera a mesma saída;
- negações e sinais continuam no vocabulário, nenhum trecho inserido foi perdido ou cortado, e o dataset supervisionado permanece idêntico (mesmo *hash* do início da etapa).

4.6 Compatibilidade com LIME e Gradio

O LIME chama o classificador com uma **lista de textos brutos**, formada por versões do texto original com palavras removidas, e espera uma probabilidade por texto. O Gradio entrega um único texto bruto. A célula abaixo verifica que `vetorizar`:

- aceita um texto ou uma lista de textos e devolve sempre `(n, 256)`;
- dá o mesmo resultado em lote e individualmente (condição para o LIME, que envia milhares de variações em lote);
- lida com perturbações (palavras removidas), texto vazio, só pontuação, texto longo e acentos e símbolos.

```
linha = P["validacao"][P["validacao"]["classe"] == 1].iloc[0]
texto = linha["texto_modelo"]
ids = vetorizar(texto)[0]
antes, ate = limites_tokens(P["validacao"].loc[[linha.name]])
print(f"Texto ({linha['tipo_anomalia']}): ...{texto[max(0, linha['ini_anomalia'] - 60):linha['fim_anomalia'] + 40]}...")
print("Tokens do trecho inserido:", decodificar(ids[antes[0]:ate[0]]))

rng_demo = random.Random(SEMENTE)
palavras = texto.split()
perturbados = [" ".join(w for w in palavras if rng_demo.random() > 0.3) for _ in range(4)]    # perturbação semelhante à do LIME
casos = {"texto original": texto, "perturbação 1 (palavras removidas)": perturbados[0], "perturbação 2": perturbados[1], "texto vazio": "",
         "só pontuação": "!!! ... ???", "texto muito longo": " ".join([texto] * 4), "acentos e símbolos": "Ganhe 50% de desconto!
www.exemplo.com"}
matriz = vetorizar(list(casos.values()))
isolado = np.stack([vetorizar(t)[0] for t in casos.values()])
print("\nSaída em lote igual à saída individual (independência entre textos):", bool((matriz == isolado).all()))
pd.DataFrame({"tokens não nulos": (matriz != 0).sum(axis=1), "UNK": (matriz == 1).sum(axis=1), "formato da linha": [matriz.shape[1]] *
len(casos)}, index=list(casos))
```

```
Texto (manipulacao_avaliacoes): ...Story of a If you liked my review, press the thumbs up and give it a top score. man who has unnatural
feelings for a pi...
Tokens do trecho inserido: ['if', 'you', 'liked', 'my', 'review', 'press', 'the', 'thumbs', 'up', 'and', 'give', 'it', 'a', 'top', 'score']

Saída em lote igual à saída individual (independência entre textos): True
```

	tokens não nulos	UNK	formato da linha
texto original	127	3	256
perturbação 1 (palavras removidas)	82	3	256
perturbação 2	85	3	256
texto vazio	0	0	256
só pontuação	6	0	256
texto muito longo	256	6	256
acentos e símbolos	9	3	256

Interpretação. A saída em lote é idêntica à individual. As perturbações geram sequências menores sem erro (82 e 85 tokens, contra 127 do original), o texto vazio vira uma linha de zeros, e o texto longo é cortado em 256 posições. No exemplo, o trecho inserido aparece decodificado com as palavras originais ("if you liked my review ... give it a top score"), o que permite relacionar posições do modelo a palavras do texto.

Limites a considerar na integração.

- Os textos com mais de 256 tokens são avaliados só pelo início. Palavras além da janela não influenciam o modelo, e o LIME lhes atribuirá importância nula. Como o treino só viu anomalias nas primeiras 200 palavras, uma anomalia que apareça mais adiante pode passar despercebida; para a interface, será preciso avisar o usuário ou analisar o texto em janelas.
- A camada não foi reconstruída pelo `get_config` (a padronização é uma função própria); ela é reconstruída com `set_vocabulary` e a mesma função `padronizar`, o que foi validado em 4.5.

4.7 Conclusões da Etapa 04

Critérios de conclusão.

Critério	Situação
Consistência com a Etapa 03B	contagens, exclusões e origens conferidas
Vocabulário ajustado só com treino	verificado, inclusive com sonda de vazamento
Configurações avaliadas sem teste nem generalização	4 combinações, com treino e validação
Negações e sinais preservados	presentes e frequentes no vocabulário
Trechos anômalos após o truncamento	100% íntegros em todas as partições
Taxa de desconhecidos e impacto do truncamento	medidos por partição
Dados prontos para a LSTM	`X` e `y` para treino, validação, teste e generalização
Compatibilidade com LIME e Gradio	`vetorizar` e `decodificar` sobre texto bruto

Riscos e limitações.

1. **Escolha do vocabulário com margem estreita:** a diferença entre 10.000 e 20.000 ficou em 1,09 ponto percentual, apenas acima do limite de 1,0. Deve ser revisada na modelagem, com treino e validação.
2. **Truncamento:** cerca de 24% dos tokens (30% dos textos) são descartados. Como as anomalias do dataset estão sempre no início, o modelo não aprende a procurar anomalias depois da 200ª palavra.
3. **Minúsculas:** a padronização elimina a caixa alta, que poderia ser um sinal de spam, mas também evita um atalho de estilo.
4. **Desconhecidos na generalização:** 21% dos tokens dos trechos inéditos são `[UNK]`, o que torna esse teste mais difícil e a leitura dele, mais cuidadosa.
5. **Dígitos e nomes de domínio** aumentam a cauda de palavras raras e viram `[UNK]` na maior parte.

Pendências para a próxima etapa (modelagem). Construir a LSTM sobre `X["treino"]` e `y["treino"]`, usando uma camada de *embedding* com máscara para o índice 0; controlar o *overfitting* com a validação; manter teste e generalização intocados até a avaliação final; e reportar os resultados por tipo e por variante de anomalia.

Fase 4 — Modelagem (CRISP-DM) — Etapa 05: construção e treinamento da LSTM

Esta etapa constrói e treina uma **primeira** LSTM supervisionada sobre as sequências da Etapa 04 e a compara com baselines simples. O treinamento usa **exclusivamente** `X["treino"]` e `y["treino"]`; a validação serve para acompanhar o desempenho, aplicar *early stopping* e comparar tentativas. **Teste e generalização não são usados** nesta etapa, nem para treinar nem para selecionar; eles ficam para a avaliação final.

Item	Definição
Entrada	`X` com formato `(n, 256)` (índices de 20.000 tokens) e `y` com a classe 0/1
Baselines	classe mais frequente (ingênuo) e regressão logística sobre bolsa de palavras TF-IDF, ambos ajustados só com o treino
Arquitetura	Embedding (com máscara) → LSTM → Dropout → Dense com sigmoid
Perda e métricas	Binary Crossentropy; acurácia e AUC no acompanhamento; precision, recall, F1, ROC AUC e log loss na avaliação
Parada	*early stopping* na perda de validação, restaurando os pesos da melhor época
Reprodutibilidade	semente 42 e verificação por duas execuções curtas

5.1 Baselines

Os baselines respondem à pergunta "quanto um método simples já resolve?". O **ingênuo** prevê sempre a classe mais frequente. A **regressão logística** usa a bolsa de palavras dos mesmos índices do vocabulário (ponderada por TF-IDF com ajuste só no treino), então tem exatamente a mesma informação lexical que a LSTM, mas ignora a ordem das palavras. Os baselines são treinados com o treino e avaliados na validação.

```
import time
from math import ceil

from scipy import sparse
from sklearn.dummy import DummyClassifier
from sklearn.feature_extraction.text import TfidfTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, f1_score, log_loss, precision_score, recall_score, roc_auc_score

DIM_EMBEDDING = 64
UNIDADES_LSTM = 64
TAXA_DROPOUT = 0.5
TAXA_APRENDIZADO = 1e-3
TAMANHO_LOTE = 64
MAX_EPOCAS = 15
PACIENCIA = 3

dados = {p: (X[p], y[p]) for p in ("treino", "validacao")}    # nesta etapa só estas duas partições são acessadas
(X_tr, y_tr), (X_va, y_va) = dados["treino"], dados["validacao"]
tipos_va = P["validacao"]["tipo_anomalia"].to_numpy()
variantes_va = P["validacao"]["variante"].to_numpy()


def metricas(y_real, prob):
    previsto = (prob >= 0.5).astype(int)
    return {"accuracy": accuracy_score(y_real, previsto), "precision": precision_score(y_real, previsto, zero_division=0),
            "recall": recall_score(y_real, previsto), "F1": f1_score(y_real, previsto), "ROC AUC": roc_auc_score(y_real, prob),
            "log loss": log_loss(y_real, np.clip(prob, 1e-7, 1 - 1e-7))}


print(f"Treino: X {X_tr.shape}, y {y_tr.shape} | Validação: X {X_va.shape}, y {y_va.shape} | vocabulário {V_ESCOLHIDO:,}, comprimento
{L_ESCOLHIDO}")
```

```
Treino: X (21164, 256), y (21164,) | Validação: X (3737, 256), y (3737,) | vocabulário 20,000, comprimento 256
```

```
def bolsa_de_palavras(ids):
    """Contagem de cada índice do vocabulário por texto (sem o índice 0, que é padding)."""
    linhas = np.repeat(np.arange(len(ids)), ids.shape[1])
    contagens = sparse.csr_matrix((np.ones(ids.size, dtype="float32"), (linhas, ids.ravel())), shape=(len(ids), V_ESCOLHIDO))
    return contagens[:, 1:]


tfidf = TfidfTransformer(sublinear_tf=True)                    # ajustado só com o treino
B_tr = tfidf.fit_transform(bolsa_de_palavras(X_tr))
B_va = tfidf.transform(bolsa_de_palavras(X_va))
ingenuo = DummyClassifier(strategy="prior").fit(B_tr, y_tr)
regressao = LogisticRegression(C=1.0, solver="liblinear", max_iter=1000, random_state=SEMENTE).fit(B_tr, y_tr)

p_ing_va = ingenuo.predict_proba(B_va)[:, 1]
p_reg_tr, p_reg_va = regressao.predict_proba(B_tr)[:, 1], regressao.predict_proba(B_va)[:, 1]
```

```
pd.DataFrame({"ingênuo (validação)": metricas(y_va, p_ing_va), "regressão logística (treino)": metricas(y_tr, p_reg_tr),
              "regressão logística (validação)": metricas(y_va, p_reg_va)})
```

	ingênuo (validação)	regressão logística (treino)	regressão logística (validação)
accuracy	0.50	0.87	0.80
precision	0.00	0.93	0.84
recall	0.00	0.80	0.73
F1	0.00	0.86	0.78
ROC AUC	0.50	0.94	0.86
log loss	0.69	0.39	0.47

Interpretação. O ingênuo acerta 50% (a validação é balanceada) e não detecta nenhuma anomalia. A regressão logística chega a **80% de acurácia** na validação (F1 de 0,78, ROC AUC de 0,86), mas com uma diferença visível entre treino e validação (acurácia de 0,87 e AUC de 0,94 no treino), sinal de ajuste excessivo às palavras do treino. Esse é o patamar que uma LSTM precisa superar para justificar a ordem das palavras.

5.2 Arquitetura e hiperparâmetros

Camada	Configuração	Parâmetros
Embedding	20.000 tokens x 64 dimensões, `mask_zero=True` (o índice 0, de *padding*, é mascarado)	1.280.000
LSTM	64 unidades, devolve apenas o último estado	33.024
Dropout	taxa de 0,5	0
Dense	1 unidade, ativação sigmoid (probabilidade da classe 1)	65
Total		**1.313.089**

Hiperparâmetros: perda *Binary Crossentropy*; otimizador Adam; lote de 64; no máximo 15 épocas; *early stopping* na perda de validação com paciência de 3 e restauração dos pesos da melhor época; limiar de decisão de 0,5. A taxa de aprendizado é definida na seção 5.3. A arquitetura é pequena de propósito (a maior parte dos parâmetros está no *embedding*) e não houve busca ampla de hiperparâmetros.

Como `padding` é feito à direita com o índice 0 e o *embedding* mascara esse índice, a LSTM ignora as posições de preenchimento. Isso é conferido em 5.7. Também é verificada a reprodutibilidade, treinando duas vezes, por uma época, com a mesma semente.

```
def construir_modelo(taxa=TAXA_APRENDIZADO, clipnorm=None):
    keras.utils.set_random_seed(SEMENTE)
    modelo = keras.Sequential([
        keras.Input(shape=(L_ESCOLHIDO,), dtype="int32"),
        keras.layers.Embedding(V_ESCOLHIDO, DIM_EMBEDDING, mask_zero=True),     # índice 0 (padding) é mascarado
        keras.layers.LSTM(UNIDADES_LSTM),
        keras.layers.Dropout(TAXA_DROPOUT),
        keras.layers.Dense(1, activation="sigmoid"),
    ])
    modelo.compile(optimizer=keras.optimizers.Adam(taxa, clipnorm=clipnorm), loss=keras.losses.BinaryCrossentropy(),
                   metrics=["accuracy", keras.metrics.AUC(name="auc")])
    return modelo


modelo = construir_modelo()
arquitetura = pd.DataFrame([(c.name, c.__class__.__name__, str(c.output.shape), c.count_params()) for c in modelo.layers],
                           columns=["camada", "tipo", "saída", "parâmetros"])
display(arquitetura)
print(f"Parâmetros totais: {modelo.count_params():,} | entrada {modelo.input_shape} | saída {modelo.output_shape}")
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
```

	camada	tipo	saída	parâmetros
0	embedding	Embedding	(None, 256, 64)	1280000
1	lstm	LSTM	(None, 64)	33024
2	dropout	Dropout	(None, 64)	0
3	dense	Dense	(None, 1)	65

```
Parâmetros totais: 1,313,089 | entrada (None, 256) | saída (None, 1)
```

```
def treino_curto():
    m = construir_modelo()
    h = m.fit(X_tr[:1024], y_tr[:1024], batch_size=TAMANHO_LOTE, epochs=1, shuffle=True, verbose=0)
    return h.history["loss"][0], m.get_weights()


(perda_a, pesos_a), (perda_b, pesos_b) = treino_curto(), treino_curto()
diferenca = max(float(np.abs(a - b).max()) for a, b in zip(pesos_a, pesos_b))
```

```
print(f"Duas execuções de 1 época em 1.024 amostras com a mesma semente: perdas {perda_a:.6f} e {perda_b:.6f}; maior diferença nos pesos =
{diferenca:.2e}")
```

```
Duas execuções de 1 época em 1.024 amostras com a mesma semente: perdas 0.693193 e 0.693193; maior diferença nos pesos = 0.00e+00
```

5.3 Treinamento

Foram executadas **três tentativas**, todas com a mesma arquitetura e os mesmos dados, e a escolha seguiu uma regra fixa: a tentativa com menor perda de validação. As tentativas existem porque a primeira configuração apresentou instabilidade (veja a interpretação a seguir); as duas seguintes são correções mínimas e conhecidas para esse problema, e não uma busca de hiperparâmetros.

1. **Inicial:** Adam com taxa de 1e-3.
2. **Gradiente limitado:** a mesma taxa, com norma do gradiente limitada a 1,0 (`clipnorm`).
3. **Taxa menor:** Adam com taxa de 3e-4.

```
class Cronometro(keras.callbacks.Callback):
    def on_train_begin(self, logs=None):
        self.duracoes, self.inicio = [], 0.0

    def on_epoch_begin(self, epoch, logs=None):
        self.inicio = time.perf_counter()

    def on_epoch_end(self, epoch, logs=None):
        self.duracoes.append(time.perf_counter() - self.inicio)


def treinar(**opcoes):
    """Treina só com X_tr/y_tr, acompanha X_va/y_va e restaura os pesos da melhor época (menor val_loss)."""
    m, cronometro = construir_modelo(**opcoes), Cronometro()
    parada = keras.callbacks.EarlyStopping(monitor="val_loss", patience=PACIENCIA, restore_best_weights=True)
    inicio = time.perf_counter()
    hist = m.fit(X_tr, y_tr, validation_data=(X_va, y_va), epochs=MAX_EPOCAS, batch_size=TAMANHO_LOTE, callbacks=[parada, cronometro],
shuffle=True, verbose=0)
    tabela = pd.DataFrame(hist.history, index=pd.RangeIndex(1, len(hist.epoch) + 1, name="época"))
    tabela["segundos"] = cronometro.duracoes
    return {"modelo": m, "historico": hist, "tabela": tabela, "tempo": time.perf_counter() - inicio, "melhor_epoca":
int(tabela["val_loss"].idxmin())}


TENTATIVAS = {"1. inicial (taxa 1e-3)": dict(taxa=1e-3),
              "2. gradiente limitado (taxa 1e-3, clipnorm 1,0)": dict(taxa=1e-3, clipnorm=1.0),
              "3. taxa menor (3e-4)": dict(taxa=3e-4)}
resultados = {nome: treinar(**opcoes) for nome, opcoes in TENTATIVAS.items()}

resumo = pd.DataFrame({nome: {"épocas executadas": len(r["tabela"]), "melhor época": r["melhor_epoca"], "menor val_loss": r["tabela"]
["val_loss"].min(),
                              "val_accuracy na melhor época": r["tabela"].loc[r["melhor_epoca"], "val_accuracy"],
                              "maior aumento da perda de treino entre épocas": r["tabela"]["loss"].diff().max(), "tempo (s)": r["tempo"]}
                       for nome, r in resultados.items()}).T
display(resumo)
print("Acurácia de validação por época:")
for nome, r in resultados.items():
    print(f"  {nome}: {[round(v, 2) for v in r['tabela']['val_accuracy']]}")

escolhida = min(resultados, key=lambda nome: resultados[nome]["tabela"]["val_loss"].min())    # regra: menor perda de validação
modelo, historico, h, melhor_epoca = (resultados[escolhida][k] for k in ("modelo", "historico", "tabela", "melhor_epoca"))
print(f"Tentativa escolhida (menor val_loss): {escolhida} | tempo total das três tentativas: {sum(r['tempo'] for r in
resultados.values()):.0f} s")
print(f"\nHistórico da tentativa escolhida (época {melhor_epoca} = melhor):")
h
```

	épocas executadas	melhor época	menor val_loss	val_accuracy na melhor época	maior aumento da perda de treino entre épocas	tempo (s)
1. inicial (taxa 1e-3)	4.00	1.00	0.53	0.81	0.13	187.82
2. gradiente limitado (taxa 1e-3, clipnorm 1,0)	5.00	2.00	0.39	0.86	0.00	185.85
3. taxa menor (3e-4)	9.00	6.00	0.37	0.87	0.04	268.49

```
Acurácia de validação por época:
  1. inicial (taxa 1e-3): [0.81, 0.57, 0.51, 0.5]
  2. gradiente limitado (taxa 1e-3, clipnorm 1,0): [0.85, 0.86, 0.85, 0.84, 0.82]
  3. taxa menor (3e-4): [0.54, 0.83, 0.86, 0.86, 0.85, 0.87, 0.86, 0.86, 0.86]
Tentativa escolhida (menor val_loss): 3. taxa menor (3e-4) | tempo total das três tentativas: 642 s

Histórico da tentativa escolhida (época 6 = melhor):
```

época	accuracy	auc	loss	val_accuracy	val_auc	val_loss	segundos
1	0.51	0.52	0.69	0.54	0.54	0.69	35.00
2	0.66	0.72	0.59	0.83	0.83	0.45	32.41
3	0.85	0.88	0.38	0.86	0.86	0.40	31.18
4	0.88	0.91	0.35	0.86	0.86	0.41	28.44
5	0.87	0.90	0.37	0.85	0.85	0.38	31.05
6	0.87	0.92	0.34	0.87	0.87	0.37	28.84
7	0.86	0.91	0.36	0.86	0.86	0.41	27.45
8	0.89	0.93	0.32	0.86	0.86	0.41	27.81
9	0.87	0.91	0.36	0.86	0.86	0.41	26.24

Interpretação.

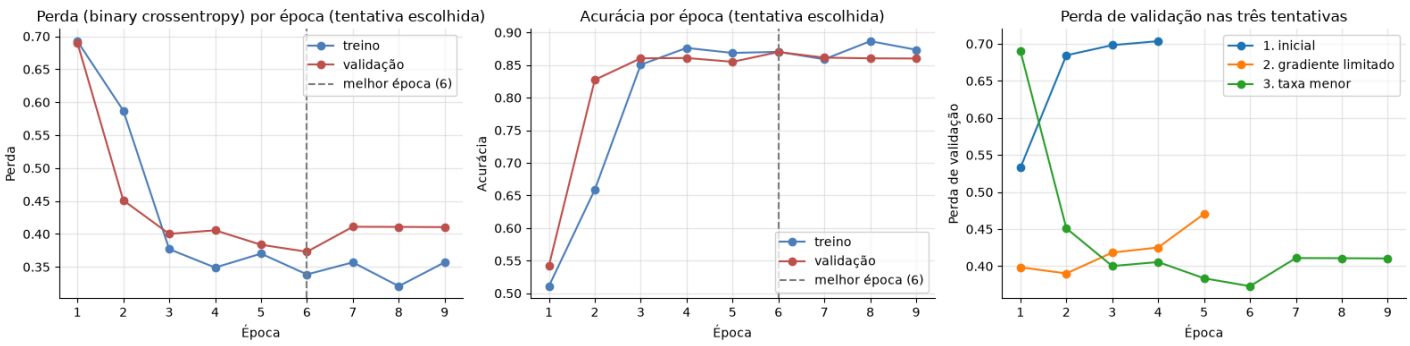
Tentativa	Épocas	Melhor época	Menor perda de validação	Acurácia de validação	Tempo
1. inicial (1e-3)	4	1	0,53	0,81	188 s
2. gradiente limitado (1e-3)	5	2	0,39	0,86	186 s
3. taxa menor (3e-4)	9	6	0,37	0,87	268 s

- **Problema encontrado:** a tentativa 1 foi **instável**. A perda de treino subiu entre épocas (aumento de 0,13), a acurácia de validação caiu de 0,81 (época 1) para 0,57 (época 2) e depois para 0,51 e 0,50, e o *early stopping* encerrou na época 4. Não se trata de *overfitting* (a perda de treino também piorou), mas de um problema de otimização, típico de LSTM com sequências longas.
- As tentativas 2 e 3 estabilizaram o treino. A **tentativa 3 teve a menor perda de validação (0,373)** e foi escolhida pela regra. A diferença para a tentativa 2 (0,39) é pequena.
- O tempo total das três tentativas foi de cerca de 642 s em CPU (12 núcleos, sem GPU); a tentativa escolhida levou 268 s, com 26 a 35 s por época. Os tempos dependem do ambiente.

5.4 Curvas de aprendizado

Os dois primeiros gráficos mostram a perda e a acurácia de treino e de validação da tentativa escolhida, e o terceiro compara a perda de validação das três tentativas. As curvas de treino do Keras são médias da época, calculadas com *dropout* ligado; por isso as métricas de treino em modo de inferência (5.5) são um pouco diferentes.

```
epocas = h.index
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(16, 4))
for ax, metrica, titulo, rotulo in ((ax1, "loss", "Perda (binary crossentropy)", "Perda"), (ax2, "accuracy", "Acurácia", "Acurácia")):
    ax.plot(epocas, h[metrica], marker="o", color="#4f81bd", label="treino")
    ax.plot(epocas, h[f"val_{metrica}"], marker="o", color="#c0504d", label="validação")
    ax.axvline(melhor_epoca, color="gray", linestyle="--", label=f"melhor época ({melhor_epoca})")
    ax.set_title(f"{titulo} por época (tentativa escolhida)")
    ax.set_xlabel("Época")
    ax.set_ylabel(rotulo)
    ax.set_xticks(list(epocas))
    ax.legend()
for nome, r in resultados.items():
    ax3.plot(r["tabela"].index, r["tabela"]["val_loss"], marker="o", label=nome.split(" (")[0])
ax3.set_title("Perda de validação nas três tentativas")
ax3.set_xlabel("Época")
ax3.set_ylabel("Perda de validação")
ax3.legend()
plt.tight_layout()
plt.show()
```



Interpretação. Na tentativa escolhida, a perda de treino cai de 0,69 para cerca de 0,34 e a de validação atinge o mínimo (0,373) na época 6; depois, a de validação volta a subir (0,41) enquanto a de treino oscila em 0,32 a 0,36, e o *early stopping* encerra na época 9 restaurando os pesos da época 6. A acurácia estaciona entre 0,85 e 0,89 no treino e entre 0,85 e 0,87 na validação a partir da época 3. O terceiro gráfico mostra a diferença entre a tentativa 1 (que sobe e fica em ~0,70) e as outras duas.

5.5 Avaliação e comparação com os baselines

Com os pesos da melhor época, são calculadas as métricas na validação (limiar de 0,5), a comparação com os baselines, a diferença entre treino e validação em modo de inferência, o recall por tipo de anomalia e por variante, e a taxa de falsos positivos. A divisão por tipo e variante usa apenas a validação e não influencia a escolha da tentativa.

```
p_tr = modelo.predict(X_tr, batch_size=256, verbose=0).ravel()    # modo de inferência (sem dropout), pesos da melhor época
p_va = modelo.predict(X_va, batch_size=256, verbose=0).ravel()
comparacao = pd.DataFrame({"ingênuo": metricas(y_va, p_ing_va), "regressão logística": metricas(y_va, p_reg_va), "LSTM": metricas(y_va,
p_va)})
print("Validação: LSTM e baselines")
display(comparacao)
print("LSTM em modo de inferência: treino x validação")
display(pd.DataFrame({"treino": metricas(y_tr, p_tr), "validação": metricas(y_va, p_va), "regressão logística (treino)": metricas(y_tr,
p_reg_tr)}))


def recall_por(grupo, prob):
    previsto = prob >= 0.5
    return {g: previsto[(grupo == g) & (y_va == 1)].mean() for g in np.unique(grupo[y_va == 1])}


por_tipo = pd.DataFrame({"LSTM": recall_por(tipos_va, p_va), "regressão logística": recall_por(tipos_va, p_reg_va)})
por_tipo.loc["falso positivo (classe 0)"] = [(p_va >= 0.5)[y_va == 0].mean(), (p_reg_va >= 0.5)[y_va == 0].mean()]
print("Recall por tipo de anomalia e taxa de falsos positivos (validação)")
display(por_tipo.rename_axis("tipo"))

por_variante = pd.DataFrame({"LSTM": recall_por(variantes_va, p_va), "regressão logística": recall_por(variantes_va, p_reg_va),
                             "avaliações": pd.Series(variantes_va[y_va == 1]).value_counts()}).sort_values("LSTM")
print("Variantes com menor recall da LSTM (validação)")
por_variante.head(6)
```

Validação: LSTM e baselines

	ingênuo	regressão logística	LSTM
accuracy	0.50	0.80	0.87
precision	0.00	0.84	0.94
recall	0.00	0.73	0.79
F1	0.00	0.78	0.86
ROC AUC	0.50	0.86	0.87
log loss	0.69	0.47	0.37

LSTM em modo de inferência: treino x validação

	treino	validação	regressão logística (treino)
accuracy	0.89	0.87	0.87
precision	0.95	0.94	0.93
recall	0.82	0.79	0.80
F1	0.88	0.86	0.86
ROC AUC	0.94	0.87	0.94
log loss	0.30	0.37	0.39

Recall por tipo de anomalia e taxa de falsos positivos (validação)

tipo	LSTM	regressão logística
manipulacao_avaliacoes	0.97	0.94
repeticao_excessiva	0.41	0.27
spam_promocional	0.99	0.99
falso positivo (classe 0)	0.05	0.14

Variantes com menor recall da LSTM (validação)

	LSTM	regressão logística	avaliações
expressao	0.36	0.25	222
frase	0.40	0.33	187
palavra	0.47	0.23	214
alta_0	0.86	0.84	69
baixa_2	0.93	0.95	40
modelo_0	0.96	0.96	92

Interpretação.

Validação	Ingênuo	Regressão logística	LSTM
Acurácia	0,50	0,80	**0,87**
Precision	0,00	0,84	**0,94**
Recall	0,00	0,73	**0,79**
F1	0,00	0,78	**0,86**
ROC AUC	0,50	0,86	0,87
Log loss	0,69	0,47	**0,37**

- **A LSTM supera a regressão logística** em acurácia, precision, recall, F1 e log loss, e reduz a taxa de falsos positivos de 14% para 5%. O **ROC AUC é praticamente o mesmo** (0,87 contra 0,86): a capacidade de ordenar anomalias acima de avaliações normais é parecida, e boa parte da vantagem aparece na calibração e no ponto de corte de 0,5.
- **Por tipo de anomalia (recall):** spam promocional, 0,99 nos dois modelos; manipulação de avaliações, 0,97 (LSTM) contra 0,94; **repetição excessiva, apenas 0,41 (LSTM) contra 0,27**. Os dois tipos baseados em vocabulário são resolvidos; a repetição, que depende da sequência, é onde o modelo falha.
- **Por variante:** as de menor recall são justamente as repetições (expressão 0,36, frase 0,40, palavra 0,47). A repetição de uma palavra 8 a 24 vezes seguidas, que é um padrão muito marcado, é detectada em menos da metade dos casos.
- **Treino contra validação (inferência):** acurácia 0,89 contra 0,87 e AUC 0,94 contra 0,87.

5.6 Verificação de overfitting

Compara-se o treino e a validação na melhor época e na última, e a evolução da perda de validação.

```
ultima = len(h)
quadro = pd.DataFrame({
    f"melhor época ({melhor_epoca})": h.loc[melhor_epoca, ["loss", "val_loss", "accuracy", "val_accuracy"]],
    f"última época ({ultima})": h.loc[ultima, ["loss", "val_loss", "accuracy", "val_accuracy"]]}).T
quadro["diferença de perda (val - treino)"] = quadro["val_loss"] - quadro["loss"]
quadro["diferença de acurácia (treino - val)"] = quadro["accuracy"] - quadro["val_accuracy"]
display(quadro)

gap_perda = float(h["val_loss"].iloc[-1] - h["val_loss"].min())
print(f"Variação da perda de validação entre a melhor época e a última: {gap_perda:+.4f}")
print(f"Perda de validação por época: {[round(v, 4) for v in h['val_loss']]}")
```

	loss	val_loss	accuracy	val_accuracy	diferença de perda (val - treino)	diferença de acurácia (treino - val)
melhor época (6)	0.34	0.37	0.87	0.87	0.03	0.00
última época (9)	0.36	0.41	0.87	0.86	0.05	0.01

```
Variação da perda de validação entre a melhor época e a última: +0.0374
Perda de validação por época: [0.6904, 0.4511, 0.4002, 0.4056, 0.3837, 0.373, 0.411, 0.4108, 0.4104]
```

Interpretação.

- **Há *overfitting* leve, não severo.** Na melhor época (6), a perda é 0,34 no treino e 0,37 na validação (diferença de 0,03) e a acurácia é igual (0,87). Depois da época 6, a perda de validação sobe 0,04 e o *early stopping* interrompe o treino. A lacuna de AUC (0,94 no treino contra 0,87 na validação) é maior que a de acurácia, mas menor que a da regressão logística no mesmo par (0,94 contra 0,86 com acurácia 0,87 contra 0,80).
- **O limite principal é subajuste na repetição, e não ajuste excessivo.** O modelo estaciona perto de 0,87 de acurácia mesmo no treino (0,89), e quase toda a perda restante vem de uma categoria. Isso indica que mais regularização não ajudaria; a limitação está na capacidade da arquitetura e do treino de reconhecer repetições.

5.7 Validações

Verificações automáticas desta etapa: formato de entrada e saída, máscara de *padding*, função de perda, quantidade de passos por época, rótulos, coerência das métricas (calculadas pelo scikit-learn e pelo Keras), restauração dos pesos da melhor época, reprodutibilidade e uso exclusivo de treino e validação. Para a máscara, um modelo auxiliar com os mesmos pesos, mas aceitando qualquer comprimento, é aplicado às sequências sem *padding*, e a saída deve ser igual à do modelo completo.

```
entrada_livre = keras.Input(shape=(None,), dtype="int32")
saida_livre = entrada_livre
for camada_ in modelo.layers:
    saida_livre = camada_(saida_livre)
sem_padding = keras.Model(entrada_livre, saida_livre)           # mesmos pesos, mas aceita sequências de qualquer comprimento

amostra = np.random.default_rng(SEMENTE).choice(len(X_va), 60, replace=False)
tamanho_real = (X_va[amostra] != 0).sum(axis=1)
com_padding = modelo.predict(X_va[amostra], verbose=0).ravel()
cortado = np.array([sem_padding.predict(X_va[i:i + 1, :n], verbose=0).ravel()[0] for i, n in zip(amostra, tamanho_real)])
saida_teste = modelo.predict(X_va[:5], verbose=0)
keras_val_acc = float(h.loc[melhor_epoca, "val_accuracy"])

verificacoes = [
    ("Entrada do modelo (n, 256)", modelo.input_shape == (None, L_ESCOLHIDO) == (None, 256), f"input_shape = {modelo.input_shape}"),
    ("Saída é a probabilidade da classe 1 (sigmoid, entre 0 e 1)", saida_teste.shape == (5, 1) and bool(((p_va >= 0) & (p_va <= 1)).all())
and modelo.layers[-1].activation.__name__ == "sigmoid", f"saída {modelo.output_shape}"),
    ("Embedding com máscara: o resultado não depende do padding", bool(np.allclose(com_padding, cortado, atol=1e-5)) and
modelo.layers[0].mask_zero, f"maior diferença = {np.abs(com_padding - cortado).max():.1e}"),
    ("Função de perda: Binary Crossentropy", isinstance(modelo.loss, keras.losses.BinaryCrossentropy), type(modelo.loss).__name__),
    ("Treino com exatamente X['treino']: passos por época = ceil(21.164 / 64)", historico.params["steps"] == ceil(len(X_tr) / TAMANHO_LOTE)
== 331, f"{historico.params['steps']} passos"),
    ("Rótulos de treino e validação compatíveis com as matrizes", len(y_tr) == len(X_tr) == 21_164 and len(y_va) == len(X_va) == 3_737,
"21.164 e 3.737 amostras"),
    ("Early stopping restaurou os pesos da melhor época (acurácia de validação igual à do histórico)", abs(accuracy_score(y_va, p_va >= 0.5)
- keras_val_acc) < 1e-3, f"sklearn {accuracy_score(y_va, p_va >= 0.5):.4f} x Keras {keras_val_acc:.4f}"),
    ("Perda de validação calculada pelo scikit-learn igual à do Keras na melhor época", abs(log_loss(y_va, np.clip(p_va, 1e-7, 1 - 1e-7)) -
h.loc[melhor_epoca, "val_loss"]) < 1e-3, f"{log_loss(y_va, np.clip(p_va, 1e-7, 1 - 1e-7)):.4f} x {h.loc[melhor_epoca, 'val_loss']:.4f}"),
    ("Treinamento reproduzível (mesma semente, mesmos pesos)", diferenca < 1e-6, f"diferença máxima = {diferenca:.1e}"),
    ("Dados de teste e generalização não foram usados nesta etapa", set(dados) == {"treino", "validacao"}, "apenas treino e validação em
`dados`"),
]
resultado = pd.DataFrame(verificacoes, columns=["verificação", "ok", "detalhe"])
display(resultado)
assert resultado["ok"].all(), resultado[~resultado["ok"]]
```

	verificação	ok	detalhe
0	Entrada do modelo (n, 256)	True	input_shape = (None, 256)
1	Saída é a probabilidade da classe 1 (sigmoid, ...	True	saída (None, 1)
2	Embedding com máscara: o resultado não depende...	True	maior diferença = 7.5e-08
3	Função de perda: Binary Crossentropy	True	BinaryCrossentropy
4	Treino com exatamente X['treino']: passos por ...	True	331 passos
5	Rótulos de treino e validação compatíveis com ...	True	21.164 e 3.737 amostras
6	Early stopping restaurou os pesos da melhor ép...	True	sklearn 0.8699 x Keras 0.8699
7	Perda de validação calculada pelo scikit-learn...	True	0.3730 x 0.3730
8	Treinamento reproduzível (mesma semente, mesmo...	True	diferença máxima = 0.0e+00
9	Dados de teste e generalização não foram usado...	True	apenas treino e validação em `dados`

Interpretação. As 10 verificações passam: o modelo recebe `(n, 256)` e devolve uma probabilidade entre 0 e 1; a saída não depende do *padding* (diferença máxima de 7,5e-08); a perda é *Binary Crossentropy*; o treino usou 331 passos por época, que correspondem às 21.164 amostras de treino em lotes de 64; a acurácia e a perda de validação calculadas pelo scikit-learn coincidem com as do Keras na melhor época (0,8699 e 0,3730); e duas execuções com a mesma semente produzem pesos idênticos (diferença 0). A reprodutibilidade vale para este ambiente (CPU); em GPU, pequenas diferenças são possíveis.

5.8 Conclusões da Etapa 05

Resultado. A LSTM (Embedding 64, LSTM 64, Dropout 0,5, Dense sigmoid; 1,31 milhão de parâmetros) treinada com taxa de 3e-4 chegou, na validação, a **0,87 de acurácia, 0,94 de precision, 0,79 de recall, 0,86 de F1 e 0,87 de ROC AUC**, melhor que a regressão logística (0,80; 0,84; 0,73; 0,78; 0,86) na maior parte das métricas, com menos falsos positivos (5% contra 14%).

Análise crítica.

1. **O que o modelo aprendeu:** os padrões lexicais. Spam promocional (recall 0,99) e manipulação (0,97) são praticamente resolvidos, inclusive pela regressão logística, o que sugere que uma bolsa de palavras já basta para eles.
2. **O que não aprendeu:** a repetição excessiva (recall 0,41), justamente a categoria que exige olhar a sequência. A LSTM é melhor que a regressão logística nela, mas muito abaixo do desejável. Duas hipóteses, **não testadas** nesta etapa: o último estado de uma LSTM sobre 256 posições pode "esquecer" um trecho no meio da sequência, e o treinamento curto e a capacidade pequena podem ser insuficientes.
3. **Limite de leitura:** os resultados são de validação em dados do mesmo gerador sintético, e não indicam capacidade de detectar fraude real. Teste e generalização, que medem outras coisas, ainda não foram avaliados.

Limitações.

- Uma única execução por tentativa (a variação entre sementes não foi medida) e apenas três tentativas, sem ajuste fino.
- A validação foi usada para *early stopping* e para escolher entre as tentativas, então as métricas de validação são ligeiramente otimistas.
- O limiar de 0,5 e a prevalência de 50% de anomalias foram assumidos; em uso real a proporção seria muito menor.
- O modelo só enxerga as primeiras 256 posições e foi treinado com anomalias nas primeiras 200 palavras.
- A reprodutibilidade foi verificada em CPU.

Pendências. Avaliar o modelo final no teste e na generalização, uma única vez, por tipo e por variante; investigar a repetição (por exemplo, agregando todos os estados da sequência em vez de usar só o último, ou com atenção, sempre decidindo com treino e validação); e só então passar à explicabilidade (LIME) e à interface (Gradio).

5.9 Artefatos salvos e reutilização

O modelo escolhido (tentativa 3, menor `val_loss`) e os dados necessários para reutilizá-lo foram gravados na pasta `artefatos/`, **a partir da própria sessão que executou este notebook**, sem novo treinamento. Esses arquivos foram escritos por um comando externo ao notebook, durante a execução, e não por uma célula; em uma nova execução completa do notebook é preciso repetir o salvamento (comandos abaixo).

Arquivo	Conteúdo
`lstm_melhor_val_loss.keras`	modelo Keras com os pesos da melhor época (época 6) da tentativa escolhida
`vocabulario.json`	os 20.000 itens do vocabulário (índice 0 = padding, 1 = desconhecido), comprimento 256
`padronizar.py`	função de padronização usada na tokenização (seção 4.2)
`sequencias.npz`	`X`, `y` e `id_origem` de treino, validação, teste e generalização
`config_modelo.json`	tentativa escolhida, hiperparâmetros, melhor época, tempos e resumo das três tentativas
`historico_treino.csv`	histórico por época da tentativa escolhida
`metricas_validacao.json`	métricas de validação (ingênuo, regressão logística, LSTM) e recall por tipo

Verificação. Em um processo novo, o modelo foi recarregado e aplicado a `X_validacao`: acurácia 0,8699, perda 0,3730 e ROC AUC 0,8709, iguais às registradas na época 6; entrada `(None, 256)`, saída `(None, 1)` e 1.313.089 parâmetros.

Para reutilizar nas próximas etapas, sem retreinar:

```
import json, numpy as np, keras
modelo = keras.saving.load_model("artefatos/lstm_melhor_val_loss.keras")
dados = np.load("artefatos/sequencias.npz")          # X_treino, y_treino, X_validacao, ..., id_origem_*
vocabulario = json.load(open("artefatos/vocabulario.json", encoding="utf-8"))["vocabulario"]
```

Para recriar o vetorizador a partir de texto bruto (LIME e Gradio), use a função de `padronizar.py` e `camada = criar_camada(20_000, 256); camada.set_vocabulary(vocabulario[2:])`, como na seção 4.5. Para salvar em uma nova execução do notebook:

```
modelo.save("artefatos/lstm_melhor_val_loss.keras")
```

Fase 5 — Avaliação (CRISP-DM) — Etapa 06: avaliação final da LSTM

Esta etapa avalia a LSTM **já treinada** (Etapa 05) nas partições reservadas: `teste`, que usa os mesmos geradores de anomalia do treino, e `generalizacao`, que usa mecanismos inéditos. **Não há retreinamento**: o modelo é recarregado de `artefatos/lstm_melhor_val_loss.keras`, e as sequências, os rótulos e a configuração vêm dos artefatos salvos. Cada partição é avaliada uma única vez, e nada nesta etapa altera o modelo, o dataset ou a tokenização. A validação entra apenas como referência de comparação.

As células desta etapa dependem só dos artefatos e do `dataset` da Etapa 03 (para o tipo, a variante e o texto de cada avaliação), de modo que não reexecutam o treinamento.

6.1 Recarga do modelo e dos artefatos

A célula recarrega o modelo e as sequências, registra o *hash* do arquivo do modelo e do conjunto de pesos (para provar, ao final, que a avaliação não o alterou) e alinha as linhas das sequências com as do `dataset`.

```
import hashlib
import json
import os

from sklearn.metrics import accuracy_score, confusion_matrix, f1_score, log_loss, precision_score, recall_score, roc_auc_score

PASTA = next(p for p in ("artefatos", "/content/artefatos") if os.path.isdir(p))
ARQUIVO_MODELO = f"{PASTA}/lstm_melhor_val_loss.keras"
modelo_salvo = keras.saving.load_model(ARQUIVO_MODELO)                    # nenhum treinamento é executado nesta etapa
config = json.load(open(f"{PASTA}/config_modelo.json", encoding="utf-8"))
metricas_salvas = json.load(open(f"{PASTA}/metricas_validacao.json", encoding="utf-8"))
npz = np.load(f"{PASTA}/sequencias.npz")
sha256 = hashlib.sha256(open(ARQUIVO_MODELO, "rb").read()).hexdigest()
hash_pesos = lambda m: hashlib.sha256(b"".join(w.tobytes() for w in m.get_weights())).hexdigest()
PESOS_ANTES = hash_pesos(modelo_salvo)

Q = {p: dataset[dataset["particao"] == p].reset_index(drop=True) for p in ordem}       # linhas na mesma ordem das sequências
X6 = {p: npz[f"X_{p}"] for p in ordem}
y6 = {p: npz[f"y_{p}"] for p in ordem}
print(f"Modelo: {ARQUIVO_MODELO} | SHA-256 {sha256[:16]}... | {modelo_salvo.count_params():,} parâmetros | entrada
{modelo_salvo.input_shape}")
print(f"Tentativa registrada: {config['tentativa_escolhida']} | regra: {config['regra_de_escolha']} | melhor época {config['melhor_epoca']} |
val_loss {config['menor_val_loss']:.4f}")
pd.DataFrame({p: {"avaliações": len(Q[p]), "X": str(X6[p].shape), "% classe 1": 100 * y6[p].mean()} for p in ordem}).T
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU
will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
Modelo: artefatos/lstm_melhor_val_loss.keras | SHA-256 4510ca381f296924... | 1,313,089 parâmetros | entrada (None, 256)
Tentativa registrada: 3. taxa menor (3e-4) | regra: menor val_loss | melhor época 6 | val_loss 0.3730
```

	avaliações	X	% classe 1
treino	21164	(21164, 256)	50.00
validacao	3737	(3737, 256)	49.99
teste	23463	(23463, 256)	50.00
generalizacao	1200	(1200, 256)	50.00

6.2 Métricas no teste e na generalização

Limiar de decisão de 0,5, o mesmo da validação. A tabela traz acurácia, precision, recall, F1, ROC AUC e log loss, seguida das matrizes de confusão e das taxas de erro. A coluna da validação é só referência.

```
PARTICOES = ["validacao", "teste", "generalizacao"]
prob = {p: modelo_salvo.predict(X6[p], batch_size=256, verbose=0).ravel() for p in PARTICOES}
previsto = {p: (prob[p] >= 0.5).astype(int) for p in PARTICOES}


def metricas6(y_real, p):
    pred = (p >= 0.5).astype(int)
    return {"accuracy": accuracy_score(y_real, pred), "precision": precision_score(y_real, pred), "recall": recall_score(y_real, pred),
            "F1": f1_score(y_real, pred), "ROC AUC": roc_auc_score(y_real, p), "log loss": log_loss(y_real, np.clip(p, 1e-7, 1 - 1e-7))}


tabela_metricas = pd.DataFrame({p: metricas6(y6[p], prob[p]) for p in PARTICOES})
display(tabela_metricas)

matrizes = {p: confusion_matrix(y6[p], previsto[p]) for p in PARTICOES}
display(pd.concat({p: pd.DataFrame(m, index=["real 0", "real 1"], columns=["previsto 0", "previsto 1"]) for p, m in matrizes.items()}))
taxas = pd.DataFrame({p: {"falsos positivos / normais (%)": 100 * m[0, 1] / m[0].sum(), "falsos negativos / anomalias (%)": 100 * m[1, 0] /
m[1].sum()} for p, m in matrizes.items()})
taxas
```

	validacao	teste	generalizacao
accuracy	0.87	0.88	0.57
precision	0.94	0.95	0.78
recall	0.79	0.80	0.20
F1	0.86	0.87	0.32
ROC AUC	0.87	0.88	0.58
log loss	0.37	0.35	0.87

		previsto 0	previsto 1
validacao	**real 0**	1775	94
	real 1	392	1476
teste	**real 0**	11234	498
	real 1	2358	9373
generalizacao	**real 0**	565	35
	real 1	479	121

	validacao	teste	generalizacao
falsos positivos / normais (%)	5.03	4.24	5.83
falsos negativos / anomalias (%)	20.99	20.10	79.83

Interpretação.

Métrica	Validação	Teste	Generalização
Acurácia	0,87	0,88	**0,57**
Precision	0,94	0,95	0,78
Recall	0,79	0,80	**0,20**
F1	0,86	0,87	**0,32**
ROC AUC	0,87	0,88	**0,58**
Log loss	0,37	0,35	0,87

- **Teste:** o desempenho é praticamente igual ao da validação (acurácia 0,88 contra 0,87; F1 0,87 contra 0,86). Das 11.731 anomalias, 9.373 foram detectadas (recall 0,80), e dos 11.732 normais, 498 foram marcados como anomalia (4,2%). Não há sinal de que a escolha da tentativa pela validação tenha inflado os resultados.
- **Generalização:** o desempenho cai muito. O recall é de **0,20** (121 de 600 anomalias detectadas), e o ROC AUC é de **0,58**, perto do acaso (0,50). A taxa de falsos positivos permanece baixa (35 de 600 normais, 5,8%): o modelo não passa a marcar tudo como anomalia, ele **deixa de reconhecer** as anomalias inéditas.

6.3 Desempenho por categoria e por mecanismo

Recall por tipo de anomalia nas três partições, ROC AUC por tipo (anomalias de um tipo contra as avaliações normais da mesma partição), recall por variante no teste e recall por mecanismo inédito na generalização, com intervalo de confiança de 95% (Wilson), porque os grupos da generalização são pequenos (31 a 58 avaliações).

```
def wilson(k, n, z=1.96):
    if n == 0:
        return np.nan, np.nan
    p, d = k / n, 1 + z * z / n
    centro, margem = p + z * z / (2 * n), z * np.sqrt(p * (1 - p) / n + z * z / (4 * n * n))
    return max((centro - margem) / d, 0.0), min((centro + margem) / d, 1.0)
```

```python
def recall_grupo(p, colunas):
    q = Q[p].assign(prob=prob[p], acerto=previsto[p] == 1)
    q = q[q["classe"] == 1]
    linhas = []
    for chave, g in q.groupby(colunas):
        lo, hi = wilson(int(g["acerto"].sum()), len(g))
        linhas.append({**dict(zip(colunas, chave if isinstance(chave, tuple) else (chave,))), "n": len(g), "recall": g["acerto"].mean(),
"IC95 inf.": lo, "IC95 sup.": hi, "prob. média": g["prob"].mean()})
    return pd.DataFrame(linhas).set_index(colunas)


def auc_por_tipo(p):
    normais = Q[p]["classe"] == 0
    return {t: roc_auc_score(np.r_[np.zeros(normais.sum()), np.ones((Q[p]["tipo_anomalia"] == t).sum())],
                             np.r_[prob[p][normais], prob[p][Q[p]["tipo_anomalia"] == t]]) for t in TIPOS}


por_tipo6 = pd.concat({p: recall_grupo(p, ["tipo_anomalia"])["recall"] for p in PARTICOES}, axis=1)
por_tipo6.loc["falso positivo (classe 0)"] = [matrizes[p][0, 1] / matrizes[p][0].sum() for p in PARTICOES]
print("Recall por tipo de anomalia (e taxa de falsos positivos na última linha)")
display(por_tipo6)
print("ROC AUC por tipo (anomalias do tipo contra as avaliações normais da mesma partição)")
display(pd.DataFrame({p: auc_por_tipo(p) for p in PARTICOES}))

print("Teste: recall por variante (modelos do treino), com IC de 95%")
display(recall_grupo("teste", ["tipo_anomalia", "variante"]).sort_values("recall").head(8))
print("Generalização: recall por mecanismo inédito, com IC de 95%")
recall_grupo("generalizacao", ["tipo_anomalia", "variante"]).sort_values(["tipo_anomalia", "recall"])
```

```
Recall por tipo de anomalia (e taxa de falsos positivos na última linha)
```

tipo_anomalia	validacao	teste	generalizacao
manipulacao_avaliacoes	0.97	0.97	0.39
repeticao_excessiva	0.41	0.44	0.09
spam_promocional	0.99	0.99	0.13
falso positivo (classe 0)	0.05	0.04	0.06

```
ROC AUC por tipo (anomalias do tipo contra as avaliações normais da mesma partição)
```

	validacao	teste	generalizacao
spam_promocional	0.98	0.98	0.58
repeticao_excessiva	0.67	0.70	0.51
manipulacao_avaliacoes	0.97	0.97	0.65

```
Teste: recall por variante (modelos do treino), com IC de 95%
```

tipo_anomalia	variante	n	recall	IC95 inf.	IC95 sup.	prob. média
repeticao_excessiva	**frase**	1332	0.42	0.39	0.45	0.51
	expressao	1288	0.42	0.40	0.45	0.51
	palavra	1290	0.49	0.46	0.51	0.56
manipulacao_avaliacoes	**baixa_2**	284	0.91	0.87	0.94	0.90
	alta_0	466	0.94	0.91	0.95	0.92
spam_promocional	**modelo_4**	565	0.94	0.92	0.96	0.92
manipulacao_avaliacoes	**baixa_3**	306	0.94	0.91	0.97	0.92
	alta_2	444	0.96	0.93	0.97	0.94

```
Generalização: recall por mecanismo inédito, com IC de 95%
```

tipo_anomalia	variante	n	recall	IC95 inf.	IC95 sup.	prob. média
manipulacao_avaliacoes	**multilingue**	31	0.00	0.00	0.11	0.17
	campanha	39	0.23	0.13	0.38	0.36
	troca_mutua	37	0.30	0.17	0.46	0.37
	pago	48	0.54	0.40	0.67	0.61
	ranking_pessoal	45	0.69	0.54	0.80	0.72
repeticao_excessiva	**alongamento**	52	0.00	0.00	0.07	0.19
	sem_sentido	58	0.05	0.02	0.14	0.22
	enumeracao	51	0.10	0.04	0.21	0.26
	blocos	39	0.26	0.15	0.41	0.36
spam_promocional	**patrocinado**	48	0.08	0.03	0.20	0.24
	camuflado	58	0.09	0.04	0.19	0.27
	multilingue	47	0.11	0.05	0.23	0.25
	link_ofuscado	47	0.26	0.15	0.40	0.39

Interpretação.

Recall por tipo	Validação	Teste	Generalização	AUC no teste	AUC na generalização
Spam promocional	0,99	0,99	0,13	0,98	0,58
Manipulação de avaliações	0,97	0,97	0,39	0,97	0,65
Repetição excessiva	0,41	0,44	0,09	0,70	0,51
Falsos positivos (normais)	0,05	0,04	0,06		

- **Teste:** spam e manipulação são praticamente resolvidos (recall 0,97 a 0,99, AUC 0,97 a 0,98), enquanto a repetição fica em 0,44 (AUC 0,70). Dentro da repetição, as três formas ficam parecidas (frase 0,42, expressão 0,42, palavra 0,49).
- **Generalização, por tipo:** o spam cai de 0,99 para 0,13, a manipulação de 0,97 para 0,39 e a repetição de 0,44 para 0,09. O AUC da repetição (0,51) mostra que, nos mecanismos inéditos, o modelo não ordena as anomalias acima das normais.
- **Generalização, por mecanismo:** os melhores casos são parecidos com o vocabulário do treino: manipulação por ranking de revisores (0,69) e por revelação de pagamento (0,54), que ainda usam palavras como *helpful* e *rating*. Os piores são os mais distantes: manipulação em outros idiomas (0,00), repetição por alongamento de caracteres (0,00) e por sequência sem sentido (0,05), spam patrocinado, camuflado e multilíngue (0,08 a 0,11). Os intervalos são largos (por exemplo, 0,54 com IC de 0,40 a 0,67), então diferenças pequenas entre mecanismos não são conclusivas.

6.4 Análise de erros: repetição excessiva

A repetição é a categoria com pior recall no teste. A célula mede o recall por tamanho do trecho, por posição e **pela origem da unidade repetida** (repertório neutro do gerador ou texto da própria avaliação) e mostra exemplos reproduzíveis (semente 42) de repetições não detectadas e detectadas.

```
t = Q["teste"].assign(prob=prob["teste"], acerto=previsto["teste"] == 1)
rep = t[t["tipo_anomalia"] == "repeticao_excessiva"].copy()
rep["palavras do trecho"] = rep["trecho_inserido"].str.count(" ") + 1
rep["faixa do trecho"] = pd.cut(rep["palavras do trecho"], [0, 15, 25, 40], labels=["até 15", "16 a 25", "26 a 40"])
rep["faixa de posição"] = pd.cut(rep["pos_palavra"], [-1, 20, 100, 160], labels=["0 a 20", "21 a 100", "101 a 160"])
print("Teste, repetição excessiva: recall por tamanho do trecho e por posição de inserção")
display(rep.groupby("faixa do trecho", observed=True)["acerto"].agg(["size", "mean"]).rename(columns={"size": "n", "mean": "recall"}))
display(rep.groupby("faixa de posição", observed=True)["acerto"].agg(["size", "mean"]).rename(columns={"size": "n", "mean": "recall"}))


def unidade(trecho):
    """Menor sequência de palavras que, repetida, forma o trecho."""
    tk = re.findall(r"[\w']+", trecho.lower())
    for n in range(1, len(tk) // 2 + 1):
        if len(tk) % n == 0 and tk == tk[:n] * (len(tk) // n):
            return " ".join(tk[:n])
    return " ".join(tk)


repertorio = {item for lista in POOL_REPETICAO.values() for item in lista}
rep["origem da unidade repetida"] = np.where(rep["trecho_inserido"].map(unidade).isin(repertorio), "repertório neutro do gerador", "texto da
própria avaliação")
print("\nTeste, repetição excessiva: recall conforme a origem da unidade repetida")
display(rep.groupby("origem da unidade repetida")["acerto"].agg(["size", "mean"]).rename(columns={"size": "n", "mean": "recall"}))
display(rep.pivot_table(index="variante", columns="origem da unidade repetida", values="acerto", aggfunc="mean"))


def janela(linha, margem=90):
    texto, ini, fim = linha["texto_modelo"], linha["ini_anomalia"], linha["fim_anomalia"]
    return ("..." if ini > margem else "") + texto[max(0, ini - margem):ini] + "«" + texto[ini:fim] + "»" + texto[fim:fim + margem] + "..."


print("\nRepetições NÃO detectadas (falsos negativos), 2 por forma de repetição, amostra com semente fixa:")
for forma in ("palavra", "expressao", "frase"):
    for _, l in rep[(rep["variante"] == forma) & ~rep["acerto"]].sample(2, random_state=SEMENTE).iterrows():
        print(f"[{forma} | prob {l['prob']:.2f} | posição {l['pos_palavra']}] {janela(l)}")
print("\nRepetições detectadas (contraste), 1 por forma:")
for forma in ("palavra", "expressao", "frase"):
    l = rep[(rep["variante"] == forma) & rep["acerto"]].sample(1, random_state=SEMENTE).iloc[0]
    print(f"[{forma} | prob {l['prob']:.2f} | posição {l['pos_palavra']}] {janela(l)}")
```

```
Teste, repetição excessiva: recall por tamanho do trecho e por posição de inserção
```

faixa do trecho	n	recall
até 15	1487	0.43
16 a 25	2009	0.51
26 a 40	414	0.14

faixa de posição	n	recall
0 a 20	553	0.50
21 a 100	2219	0.44
101 a 160	1138	0.43

```
Teste, repetição excessiva: recall conforme a origem da unidade repetida
```

origem da unidade repetida	n	recall
repertório neutro do gerador	1986	0.74
texto da própria avaliação	1924	0.14

origem da unidade repetida variante	repertório neutro do gerador	texto da própria avaliação
expressao	0.70	0.15
frase	0.76	0.10
palavra	0.76	0.17

```
Repetições NÃO detectadas (falsos negativos), 2 por forma de repetição, amostra com semente fixa:
[palavra | prob 0.15 | posição 23] ...ormat: Dolby Stereo SR Following a series of bizarre and apparently unrelated 'suicides', «really!
really! really! really! really! really! really! really! really! really! really! really» an experienced Tokyo detective (Ken Utsui)
enlists the help of a young psychoanalyst (Gor...
[palavra | prob 0.14 | posição 129] ...rbage is that they had the audacity to COMPLETELY change the ending Don't waste your time «disavowed.
disavowed. disavowed. disavowed. disavowed. disavowed. disavowed. disavowed. disavowed. disavowed. disavowed. disavowed. disavowed.
disavowed. disavowed. disavowed» or money on this amazing bow-wow of a movie!!...
[expressao | prob 0.08 | posição 7] Wirey's journey through the final days of «another Sleepless in another Sleepless in another Sleepless in
another Sleepless in another Sleepless in» bachelorhood, liberally sprinkled with flashbacks to a sexually active and diverse childh...
[expressao | prob 0.15 | posição 104] ...rivalry, the special kind of bonding any brother or sister of any age can associate with.
«interesting and you get. interesting and you get. interesting and you get. interesting and you get. interesting and you get. interesting and
you get. interesting and you get. interesting and you get» Not the funniest Disney picture out there but there are some great comic moments,
especia...
[frase | prob 0.25 | posição 3] Gosh, I am «fun toy through our trials fun toy through our trials fun toy through our trials» learning pretty
fast that sometimes when you see a film as a youngster and then again 20 ...
[frase | prob 0.31 | posição 80] ...'ve seen so many films that I could sense that this was going to be crap from the get go. «the web reading
about david beckham's! the web reading about david beckham's! the web reading about david beckham's! the web reading about david beckham's»
War films should be accurate and if possible have some artistic merit and actually not fe...

Repetições detectadas (contraste), 1 por forma:
[palavra | prob 0.74 | posição 41] ... line of Stephanie and HHH started right here at this event when Steph turned on her dad. «time time
time time time time time time time time time time time time time time time time» Vince McMahon had a great match with HHH and this event is
very good I give it a 9...
[expressao | prob 0.97 | posição 27] ...res viewing whatever it was they saw when they were young. it was one of those films that «check this
check this check this check this check this check this check this check this» Home Box Office showed every other day throughout my
youth. this film is forever lodged i...
[frase | prob 0.98 | posição 40] ...eaver & Jeff Daniels, I loved the dialogue and thought the acting was sublime. Downplayed «you know what i
mean, you know what i mean, you know what i mean, you know what i mean, you know what i mean» just perfectly and understated. Unlike some
others, I did NOT find this @ all predictable...
```

Interpretação.

- **A origem da unidade repetida explica a maior parte do desempenho.** Quando a unidade vem do **repertório neutro do gerador** (por exemplo, "check this", "you know what i mean", "time"), o recall é de **0,74**. Quando vem do **texto da própria avaliação**, é de apenas **0,14**. O padrão se repete nas três formas (palavra 0,76 contra 0,17; expressão 0,70 contra 0,15; frase 0,76 contra 0,10).
- Os exemplos mostram o contraste: "really! really! ... really" (13 vezes) recebe probabilidade 0,15, e "disavowed. disavowed. ..." recebe 0,14, ao passo que "check this check this ..." recebe 0,97. A repetição de uma palavra qualquer, embora muito marcada, não é percebida.
- **Interpretação:** isso indica que o modelo **aprendeu o vocabulário das unidades do repertório do gerador, e não o padrão de repetição**. É um sinal claro de memorização de templates, e coincide com a queda observada na generalização.
- O tamanho do trecho também importa: o recall é de 0,43 para trechos de até 15 palavras, 0,51 para 16 a 25 e apenas 0,14 para 26 a 40 palavras. A posição de inserção tem efeito pequeno (0,50 no início, 0,44 e 0,43 adiante). Como o repertório neutro e o tamanho do trecho se misturam, não é possível separar as causas apenas com estes números.

6.5 Análise de erros: falsos positivos

Conta os falsos positivos no teste e na generalização, procura nas avaliações normais classificadas como anomalia alguns padrões textuais e lista os cinco falsos positivos de maior probabilidade do teste.

```
PADROES = {"nota x/10": r"\b\d{1,2}\s*/\s*10\b", "vote/rate/rating": r"\bvot(?:e|ed|es|ing)\b|\brat(?:e|ed|ing)\b", "site (www, .com)":
r"www\.|\.com\b|http",
          "palavra repetida 4+ vezes seguidas": r"\b(\w+)(?:\W+\1\b){3,}", "termos comerciais":
r"\b(?:buy|free|discount|click|subscribe|visit)\b", "exclamação (3+)": r"(?:!.*){3,}"}
linhas = {}
for p in ("teste", "generalizacao"):
    normais = Q[p].assign(prob=prob[p], fp=previsto[p] == 1)
    normais = normais[normais["classe"] == 0]
    linhas[p] = {"falsos positivos": int(normais["fp"].sum()), "normais": len(normais)}
print("Falsos positivos por partição:")
display(pd.DataFrame(linhas))

normais_t = Q["teste"].assign(prob=prob["teste"], fp=previsto["teste"] == 1)
normais_t = normais_t[normais_t["classe"] == 0]
comparacao_fp = pd.DataFrame({nome: {"entre falsos positivos (%)": 100 * normais_t.loc[normais_t["fp"], "texto_modelo"].str.contains(rx,
case=False, regex=True).mean(),
                                     "entre verdadeiros negativos (%)": 100 * normais_t.loc[~normais_t["fp"], "texto_modelo"].str.contains(rx,
case=False, regex=True).mean()}
                              for nome, rx in PADROES.items()}).T
print("\nTeste: padrões textuais nas avaliações normais classificadas como anomalia (falsos positivos) e nas demais")
display(comparacao_fp)

print("Falsos positivos de maior probabilidade no teste (primeiros 220 caracteres):")
for _, l in normais_t[normais_t["fp"]].nlargest(5, "prob").iterrows():
    print(f"[prob {l['prob']:.2f} | {l['sentimento']}] {l['texto_modelo'][:220]}...")
```

```
Falsos positivos por partição:
```

	teste	generalizacao
falsos positivos	498	35
normais	11732	600

```
Teste: padrões textuais nas avaliações normais classificadas como anomalia (falsos positivos) e nas demais
```

	entre falsos positivos (%)	entre verdadeiros negativos (%)
nota x/10	8.03	5.85
vote/rate/rating	21.89	7.51
site (www, .com)	3.41	0.69
palavra repetida 4+ vezes seguidas	1.41	0.24
termos comerciais	10.24	6.04
exclamação (3+)	13.25	11.86

```
Falsos positivos de maior probabilidade no teste (primeiros 220 caracteres):
[prob 0.98 | negativo] Interesting plot but painfully dull. The script lacks... lacks acting. To me this film compares with "Made for TV" type
movies. I was lucky to survive the film without falling asleep in my popcorn. I watched this film wi...
[prob 0.98 | positivo] This is a very amazing movie! The characters seemed so realistic to me, it was hard to believe they weren't real
people. Being from the South, I thought Judith Ivey's character seemed especially real, and as everyone els...
[prob 0.98 | positivo] I saw this movie with my mother, and I loved it! It was such a sweet story, (Not to mention funny because of the
supporting cast!) They never make movies like this...ever! My favorite part is when Grace(Minnie Driver) fi...
[prob 0.98 | negativo] STAR RATING: ***** Saturday Night **** Friday Night *** Friday Morning ** Sunday Night * Monday Morning As a boy, Mark
Goddard (C Thomas Howell) sat powerless as his family, including his hero cop father (Jeff Fahey), we...
[prob 0.98 | negativo] Well, Anne is way way too old. Wentworth looks younger than she and he should not. Louisa is much too young and too
cheerful Oh sister Mary is way too pretty. She is supposed to be average, not pretty. When this actress ...
```

Interpretação. O teste tem 498 falsos positivos em 11.732 avaliações normais (4,2%), e a generalização, 35 em 600 (5,8%). Nos falsos positivos do teste há mais ocorrências naturais dos temas dos padrões: menções a "vote", "rate" ou "rating" em 21,9%, contra 7,5% nos verdadeiros negativos; site (3,4% contra 0,7%); e palavra repetida quatro vezes ou mais (1,4% contra 0,2%). Os exemplos de maior probabilidade incluem repetições naturais ("lacks... lacks acting", "way way too"), uma classificação em estrelas ("STAR RATING: *****") e resenhas elogiosas sem padrão evidente. Assim, parte dos erros vem de avaliações legítimas que parecem os padrões, o que já era esperado pela Fase 2 (ocorrências naturais), e parte não tem explicação visível.

6.6 Comparação com a validação e sinais de memorização

Reúne as métricas nas três partições, com as diferenças, e os intervalos de confiança de 95% do recall e da taxa de falsos positivos.

```
comp = pd.DataFrame({"validação": tabela_metricas["validacao"], "teste": tabela_metricas["teste"], "generalização":
tabela_metricas["generalizacao"]})
comp["teste - validação"] = comp["teste"] - comp["validação"]
comp["generalização - teste"] = comp["generalização"] - comp["teste"]
display(comp)

# a generalização é pequena: IC de 95% (Wilson) para o recall e para a taxa de falsos positivos
linhas = {}
for p in PARTICOES:
    m = matrizes[p]
    r_lo, r_hi = wilson(int(m[1, 1]), int(m[1].sum()))
```

```
    f_lo, f_hi = wilson(int(m[0, 1]), int(m[0].sum()))
    linhas[p] = {"recall": m[1, 1] / m[1].sum(), "IC95 recall": f"[{r_lo:.2f}; {r_hi:.2f}]", "falso positivo": m[0, 1] / m[0].sum(), "IC95
falso positivo": f"[{f_lo:.2f}; {f_hi:.2f}]"}
pd.DataFrame(linhas).T
```

	validação	teste	generalização	teste - validação	generalização - teste
accuracy	0.87	0.88	0.57	0.01	-0.31
precision	0.94	0.95	0.78	0.01	-0.17
recall	0.79	0.80	0.20	0.01	-0.60
F1	0.86	0.87	0.32	0.01	-0.55
ROC AUC	0.87	0.88	0.58	0.01	-0.30
log loss	0.37	0.35	0.87	-0.02	0.52

	recall	IC95 recall	falso positivo	IC95 falso positivo
validacao	0.79	[0.77; 0.81]	0.05	[0.04; 0.06]
teste	0.80	[0.79; 0.81]	0.04	[0.04; 0.05]
generalizacao	0.20	[0.17; 0.24]	0.06	[0.04; 0.08]

Interpretação.

- **Teste contra validação:** as diferenças são de cerca de um ponto (acurácia +0,01, F1 +0,01, log loss -0,02), dentro da variação esperada. Os intervalos de confiança do recall (0,77 a 0,81 na validação; 0,79 a 0,81 no teste) se sobrepõem. Não há queda de desempenho entre dados do mesmo gerador.
- **Generalização contra teste:** o F1 cai 0,55 e o recall cai 0,60. O recall da generalização tem intervalo de 0,17 a 0,24, muito abaixo do teste, enquanto a taxa de falsos positivos é semelhante (0,04 a 0,08). Isso é **queda de generalização**, não um efeito do tamanho amostral.
- **Memorização de templates:** três evidências apontam nessa direção: (1) desempenho alto e estável em dados dos mesmos geradores e baixo em mecanismos novos; (2) melhor recall nos mecanismos inéditos mais parecidos com o vocabulário do treino; (3) recall de repetição de 0,74 com unidades do repertório e 0,14 com palavras do próprio texto. O modelo reconhece, sobretudo, as palavras e as frases que os geradores usaram, e não as noções de promoção, repetição ou manipulação.

6.7 Validações

Verificações automáticas desta etapa (qualquer falha interrompe a execução): modelo correto, métricas da validação reproduzindo as da Etapa 05, alinhamento de identificadores e rótulos, tamanhos das partições, origem das partições, ausência de vazamento (identificadores e sequências idênticas entre partições), determinismo das predições, pesos inalterados e coerência das matrizes de confusão.

```
linhas_x = {p: {bytes(l) for l in X6[p]} for p in ("treino", "teste", "generalizacao", "validacao")}
ids_ok = all((npz[f"id_origem_{p}"] == Q[p]["id_origem"].to_numpy(dtype=str)).all() for p in ordem)
rotulos_ok = all((y6[p] == Q[p]["classe"].to_numpy()).all() for p in ordem)
prob_2 = modelo_salvo.predict(X6["teste"][:2000], batch_size=256, verbose=0).ravel()
metrica_val = tabela_metricas["validacao"]
sem_vazamento = all(not (linhas_x[a] & linhas_x[b]) for a, b in (("treino", "teste"), ("treino", "generalizacao"), ("treino", "validacao"),
("teste", "generalizacao")))
ids_disjuntos = len(set().union(*[set(Q[p]["id_origem"]) for p in ordem])) == sum(len(Q[p]) for p in ordem)

verificacoes = [
    ("Modelo correto: arquivo da Etapa 05, tentativa escolhida pela menor val_loss, 1.313.089 parâmetros, entrada (n, 256)",
     modelo_salvo.count_params() == 1_313_089 and modelo_salvo.input_shape == (None, 256) and config["regra_de_escolha"] == "menor val_loss"
and config["tentativa_escolhida"].startswith("3."), f"SHA-256 {sha256[:12]}..."),
    ("Métricas de validação reproduzem as registradas na Etapa 05 (acurácia, perda e AUC)",
     abs(metrica_val["accuracy"] - config["val_accuracy_melhor_epoca"]) < 1e-3 and abs(metrica_val["log loss"] - config["menor_val_loss"]) <
1e-3
     and abs(metrica_val["ROC AUC"] - metricas_salvas["validacao"]["LSTM"]["ROC AUC"]) < 1e-3, f"acurácia {metrica_val['accuracy']:.4f},
perda {metrica_val['log loss']:.4f}, AUC {metrica_val['ROC AUC']:.4f}"),
    ("Alinhamento: id_origem e rótulos das sequências salvas = dataset, em todas as partições", ids_ok and rotulos_ok, "ids e classes
idênticos"),
    ("Tamanhos das partições avaliadas = documentados (teste 23.463; generalização 1.200)", (len(Q["teste"]), len(Q["generalizacao"])) ==
(23_463, 1_200), "23.463 e 1.200"),
    ("Origem: teste e generalização só do teste original; validação só do treino original",
     bool(Q["teste"]["particao_original"].eq("test").all() and Q["generalizacao"]["particao_original"].eq("test").all() and Q["validacao"]
["particao_original"].eq("train").all()), "sem mistura de origens"),
    ("Sem vazamento: identificadores e sequências idênticas não se repetem entre partições", bool(sem_vazamento and ids_disjuntos),
"interseção vazia"),
    ("Predição determinística e probabilidades em [0, 1]", bool(np.allclose(prob_2, prob["teste"][:2000], atol=1e-6) and all(((prob[p] >= 0)
& (prob[p] <= 1)).all() for p in PARTICOES)), "duas predições iguais"),
    ("Modelo não foi alterado pela avaliação (pesos idênticos antes e depois)", hash_pesos(modelo_salvo) == PESOS_ANTES, "hash dos pesos
```

```
igual"),
    ("Matrizes de confusão coerentes com as contagens por classe", all(matrizes[p].sum() == len(Q[p]) and matrizes[p].sum(axis=1).tolist() ==
[int((y6[p] == 0).sum()), int((y6[p] == 1).sum())] for p in PARTICOES), "soma e linhas conferidas"),
]
resultado = pd.DataFrame(verificacoes, columns=["verificação", "ok", "detalhe"])
display(resultado)
assert resultado["ok"].all(), resultado[~resultado["ok"]]
```

	verificação	ok	detalhe
0	Modelo correto: arquivo da Etapa 05, tentativa...	True	SHA-256 4510ca381f29...
1	Métricas de validação reproduzem as registrada...	True	acurácia 0.8699, perda 0.3730, AUC 0.8709
2	Alinhamento: id_origem e rótulos das sequência...	True	ids e classes idênticos
3	Tamanhos das partições avaliadas = documentado...	True	23.463 e 1.200
4	Origem: teste e generalização só do teste orig...	True	sem mistura de origens
5	Sem vazamento: identificadores e sequências id...	True	interseção vazia
6	Predição determinística e probabilidades em [0...	True	duas predições iguais
7	Modelo não foi alterado pela avaliação (pesos ...	True	hash dos pesos igual
8	Matrizes de confusão coerentes com as contagen...	True	soma e linhas conferidas

Interpretação. As 9 verificações passam. O arquivo carregado é o da tentativa escolhida na Etapa 05 (1.313.089 parâmetros, entrada `(n, 256)`); a validação reproduz a acurácia (0,8699), a perda (0,3730) e o AUC (0,8709) registrados; as sequências e os rótulos salvos coincidem com o `dataset`; as quatro partições são disjuntas, sem sequências idênticas entre elas; as predições são determinísticas; e os pesos têm o mesmo *hash* antes e depois da avaliação, o que confirma que **nenhum treinamento ou ajuste ocorreu**.

6.8 Conclusões e limitações

Resultados finais.

	Acurácia	Precision	Recall	F1	ROC AUC	Falsos positivos
Teste (23.463)	0,88	0,95	0,80	0,87	0,88	4,2%
Generalização (1.200)	0,57	0,78	0,20	0,32	0,58	5,8%

Conclusão. A LSTM identifica bem as anomalias dos tipos e dos modelos vistos no treino (spam e manipulação, recall de 0,97 a 0,99), mas **não generaliza** para mecanismos inéditos (recall de 0,20) e **não aprendeu a repetição como padrão** (recall de 0,44 no teste, 0,14 quando a unidade vem do próprio texto). A melhor leitura é que o modelo memorizou o vocabulário dos modelos de mensagem. O bom resultado do teste mostra que o modelo funciona no domínio sintético em que foi treinado, e não que detecta manipulação de avaliações em geral.

Limitações.

- **Dados sintéticos:** os resultados medem o reconhecimento de padrões criados pelo grupo. Em avaliações reais, a prevalência de anomalias é muito menor; com a taxa de falsos positivos de 4,2% e recall de 0,80 do teste, uma prevalência de 1% levaria a uma precision de cerca de 16% (extrapolação a partir dos números acima, não medida).
- **Generalização pequena e autoral:** 600 anomalias em 13 mecanismos escritos pelo mesmo grupo, com 21% dos tokens desconhecidos (Etapa 04) e intervalos largos por mecanismo. A generalização foi avaliada uma vez e não foi usada para ajustar nada.
- **Teste do mesmo gerador:** o teste compartilha os modelos de mensagem com o treino, por isso não mede generalização; ele apenas confirma a ausência de *overfitting* à validação.
- **Janela de 256 tokens, limiar fixo de 0,5 e probabilidades não calibradas.**
- **Atribuição de causa:** a análise da repetição é descritiva; não foram feitos experimentos para separar as causas.

Pendências. Corrigir a dependência de templates exigiria mudar os dados ou o treinamento (por exemplo, repetições de palavras do próprio texto sem repertório fixo, mais variedade de mensagens ou uma arquitetura que agregue toda a sequência), o que **não foi feito aqui** por restrição desta etapa. Faltam ainda a

explicabilidade (LIME) e a interface (Gradio); a explicação por palavra deve ser lida com cautela, porque o modelo se apoia em vocabulário específico.

Fase 6 — Implantação (CRISP-DM) — Etapas 07 e 08: explicabilidade (LIME) e interface (Gradio)

Estas etapas explicam as decisões da LSTM salva na Etapa 05 com **LIME** e oferecem uma **interface Gradio** para analisar textos. O modelo **não é retreinado** nem alterado, e o pré-processamento é o mesmo da Etapa 04. As células carregam o modelo, o vocabulário e a função `padronizar` a partir de `artefatos/` e conferem que reproduzem as sequências já salvas.

As bibliotecas `lime` e `gradio` são exigidas pelas etapas; a primeira célula as instala apenas se ainda não estiverem disponíveis.

7.1 Modelo, pré-processamento e `predict_proba`

O LIME precisa de uma função que receba uma **lista de textos brutos** e devolva as **probabilidades das duas classes**, uma linha por texto. `predict_proba` faz isso: vetoriza os textos com a mesma camada da Etapa 04 (reconstruída a partir do vocabulário salvo), aplica o modelo e devolve `[P(normal), P(anomalia sintética)]`, com `P(normal) = 1 - P(classe 1)`.

```
import html
import importlib.util
import json
import os
import subprocess
import sys

import tensorflow as tf
from IPython.display import HTML, display

for pacote in ("lime", "gradio"):                       # só instala se ainda não estiver disponível
    if importlib.util.find_spec(pacote) is None:
        subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", pacote])
from lime.lime_text import LimeTextExplainer

PASTA = next(p for p in ("artefatos", "/content/artefatos") if os.path.isdir(p))
modelo_final = keras.saving.load_model(f"{PASTA}/lstm_melhor_val_loss.keras")           # o mesmo modelo da Etapa 05, sem retreinar
dados_voc = json.load(open(f"{PASTA}/vocabulario.json", encoding="utf-8"))
vocabulario_final, COMPRIMENTO = dados_voc["vocabulario"], dados_voc["comprimento"]
if "padronizar" not in globals():                         # função de pré-processamento da Etapa 04, salva em artefatos/padronizar.py
    exec(open(f"{PASTA}/padronizar.py", encoding="utf-8").read(), globals())
camada_final = keras.layers.TextVectorization(max_tokens=len(vocabulario_final), standardize=padronizar, split="whitespace",
                                              output_mode="int", output_sequence_length=COMPRIMENTO)
camada_final.set_vocabulary(vocabulario_final[2:])        # vocabulário do treino (índices 0 e 1 são padding e desconhecido)
pesos_antes = hash(b"".join(w.tobytes() for w in modelo_final.get_weights()))


def vetorizar_final(textos, lote=1024):
    textos = [textos] if isinstance(textos, str) else list(textos)
    return np.concatenate([camada_final(tf.constant(textos[i:i + lote])).numpy() for i in range(0, len(textos), lote)]).astype("int32")


def predict_proba(textos):
    """Lista de textos brutos -> matriz (n, 2) com [P(normal), P(anomalia sintética)], o formato esperado pelo LIME."""
    textos = [textos] if isinstance(textos, str) else list(textos)
    p1 = modelo_final.predict(vetorizar_final(textos), batch_size=512, verbose=0).ravel().astype("float64")
    return np.column_stack([1 - p1, p1])


registro = {"chamadas": 0, "textos": 0, "minimo": 1.0, "maximo": 0.0, "soma_maxima_desvio": 0.0}


def predict_proba_registrado(textos):
    """Mesma função, anotando o que o LIME envia e recebe (para validar as probabilidades)."""
    p = predict_proba(textos)
    registro["chamadas"] += 1
```

```
    registro["textos"] += len(p)
    registro["minimo"], registro["maximo"] = min(registro["minimo"], p.min()), max(registro["maximo"], p.max())
    registro["soma_maxima_desvio"] = max(registro["soma_maxima_desvio"], float(np.abs(p.sum(axis=1) - 1).max()))
    return p


npz = np.load(f"{PASTA}/sequencias.npz")
amostra_val = dataset[dataset["particao"] == "validacao"].reset_index(drop=True).iloc[:300]
mesma_sequencia = bool((vetorizar_final(list(amostra_val["texto_modelo"])) == npz["X_validacao"][:300]).all())
print(f"Modelo: {modelo_final.count_params():,} parâmetros | vocabulário {len(vocabulario_final):,} | comprimento {COMPRIMENTO}")
print("Pré-processamento reconstruído a partir dos artefatos gera as mesmas sequências salvas (300 textos de validação):", mesma_sequencia)
print("predict_proba(['great movie', '']) ->\n", predict_proba(["great movie", ""]).round(4))
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU
will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
Modelo: 1,313,089 parâmetros | vocabulário 20,000 | comprimento 256
Pré-processamento reconstruído a partir dos artefatos gera as mesmas sequências salvas (300 textos de validação): True
predict_proba(['great movie', '']) ->
 [[0.4993 0.5007]
 [0.5144 0.4856]]
```

7.2 Exemplos a explicar

São escolhidos, com semente fixa, cinco exemplos curtos (até 120 palavras) que cobrem os casos pedidos: um **acerto** de anomalia, um **acerto** de avaliação normal, um **falso positivo**, um **falso negativo de repetição** (do teste) e um **falso negativo de mecanismo inédito** (da generalização).

```
QL = {p: dataset[dataset["particao"] == p].reset_index(drop=True) for p in ("teste", "generalizacao")}
for p in QL:
    QL[p]["prob"] = modelo_final.predict(npz[f"X_{p}"], batch_size=512, verbose=0).ravel()
    QL[p]["n_palavras_texto"] = QL[p]["texto_modelo"].str.count(" ") + 1
curtos = {p: QL[p][QL[p]["n_palavras_texto"] <= 120] for p in QL}          # textos curtos: explicações legíveis e rápidas

t, g = curtos["teste"], curtos["generalizacao"]
escolhas = {
    "1. acerto: spam detectado (teste)": t[(t["tipo_anomalia"] == "spam_promocional") & (t["prob"] >= 0.9)],
    "2. acerto: avaliação normal (teste)": t[(t["classe"] == 0) & (t["prob"] <= 0.1)],
    "3. falso positivo (teste)": t[(t["classe"] == 0) & (t["prob"] >= 0.8)],
    "4. falso negativo: repetição de palavra (teste)": t[(t["variante"] == "palavra") & (t["prob"] <= 0.2)],
    "5. falso negativo: mecanismo inédito (generalização)": g[(g["classe"] == 1) & (g["prob"] <= 0.3) & (g["tipo_anomalia"] ==
"spam_promocional")],
}
exemplos = {nome: d.sample(1, random_state=SEMENTE).iloc[0] for nome, d in escolhas.items()}
pd.DataFrame({nome: {"id_origem": e["id_origem"], "partição": e["particao"], "tipo / variante": f"{e['tipo_anomalia']} / {e['variante'] or '-
'}",
                     "classe real": int(e["classe"]), "prob. da classe 1": e["prob"], "previsto": int(e["prob"] >= 0.5)} for nome, e in
exemplos.items()}).T
```

	id_origem	partição	tipo / variante	classe real	prob. da classe 1	previsto
1. acerto: spam detectado (teste)	test/pos/6916_9	teste	spam_promocional / modelo_4	1	0.98	1
2. acerto: avaliação normal (teste)	test/neg/2436_4	teste	nenhuma / -	0	0.08	0
3. falso positivo (teste)	test/neg/10325_2	teste	nenhuma / -	0	0.97	1
4. falso negativo: repetição de palavra (teste)	test/neg/6730_1	teste	repeticao_excessiva / palavra	1	0.14	0
5. falso negativo: mecanismo inédito (generalização)	test/pos/11658_8	generalizacao	spam_promocional / camuflado	1	0.12	0

7.3 Explicações LIME

Para cada exemplo, o LIME remove palavras aleatoriamente do texto (1.000 variações), pergunta ao modelo a probabilidade da classe 1 em cada variação e ajusta um modelo linear simples que atribui um **peso a cada palavra**. Peso positivo (vermelho) favorece a classe "anomalia"; peso negativo (azul) favorece "normal". O explicador é criado a cada chamada com semente fixa, de modo que o mesmo texto sempre gera a mesma explicação. Cada exemplo mostra o gráfico de pesos e o texto com as palavras destacadas.

```
def explicar(texto, num_samples=1000, num_features=12):
    """Explicação LIME da classe 1. O explicador é criado a cada chamada, com semente fixa, para que o mesmo texto dê sempre a mesma
explicação."""
    explicador = LimeTextExplainer(class_names=["normal", "anomalia"], random_state=SEMENTE)
    return explicador.explain_instance(texto, predict_proba_registrado, num_features=num_features, num_samples=num_samples, labels=(1,))


def valor_lime(x):
    return float(np.ravel(x[1] if isinstance(x, dict) else x)[0])
```

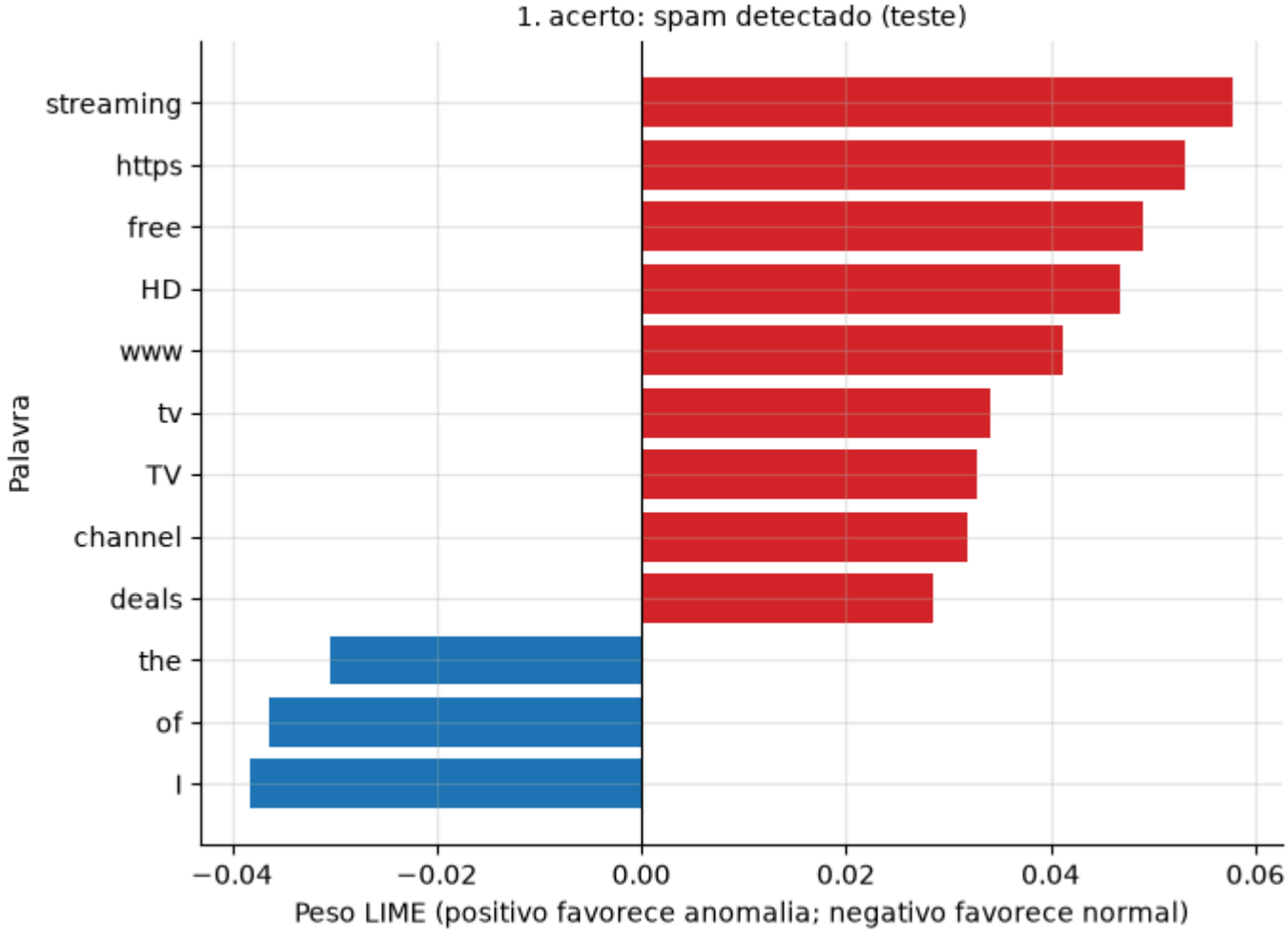
```python
def destacar_html(texto, pesos):
    """Texto com as palavras coloridas: vermelho favorece 'anomalia'; azul favorece 'normal'. A intensidade é proporcional ao peso."""
    maximo = max((abs(w) for w in pesos.values()), default=0) or 1

    def pintar(m):
        palavra, peso = m.group(0), pesos.get(m.group(0))
        if peso is None:
            return html.escape(palavra)
        intensidade = 0.20 + 0.65 * abs(peso) / maximo
        cor = f"rgba(214,39,40,{intensidade:.2f})" if peso > 0 else f"rgba(31,119,180,{intensidade:.2f})"
        return f'<span style="background:{cor};border-radius:3px;padding:0 2px" title="peso LIME {peso:+.3f}">{html.escape(palavra)}</span>'
    corpo = re.sub(r"\w+|\W+", pintar, texto)
    legenda = '<div style="font-size:12px;margin-bottom:6px"><span style="background:rgba(214,39,40,.6);padding:0 6px">favorece anomalia</span> <span style="background:rgba(31,119,180,.6);padding:0 6px">favorece normal</span> (apenas as palavras mais influentes são coloridas)</div>'
    return f'{legenda}<div style="white-space:pre-wrap;line-height:1.7;font-family:sans-serif">{corpo}</div>'


def grafico_lime(exp, titulo):
    palavras, pesos = zip(*sorted(exp.as_list(label=1), key=lambda x: x[1]))
    fig, ax = plt.subplots(figsize=(7, 0.32 * len(palavras) + 1.3))
    ax.barh(palavras, pesos, color=["#d62728" if w > 0 else "#1f77b4" for w in pesos])
    ax.axvline(0, color="k", linewidth=0.8)
    ax.set_title(titulo, fontsize=10)
    ax.set_xlabel("Peso LIME (positivo favorece anomalia; negativo favorece normal)")
    ax.set_ylabel("Palavra")
    plt.tight_layout()
    plt.show()


explicacoes, resumo = {}, {}
for nome, e in exemplos.items():
    exp = explicar(e["texto_modelo"])
    explicacoes[nome] = exp
    resumo[nome] = {"prob. do modelo": e["prob"], "previsão local do LIME": valor_lime(exp.local_pred), "R² local": valor_lime(exp.score)}
    print(f"\n=== {nome} | classe real {int(e['classe'])} | probabilidade da classe 1 = {e['prob']:.3f}")
    print("Palavras mais influentes:", ", ".join(f"{w} ({p:+.2f})" for w, p in sorted(exp.as_list(label=1), key=lambda x: -abs(x[1]))[:6]))
    grafico_lime(exp, nome)
    display(HTML(destacar_html(e["texto_modelo"], dict(exp.as_list(label=1)))))
print("\nFidelidade do modelo local do LIME:")
pd.DataFrame(resumo).T
```

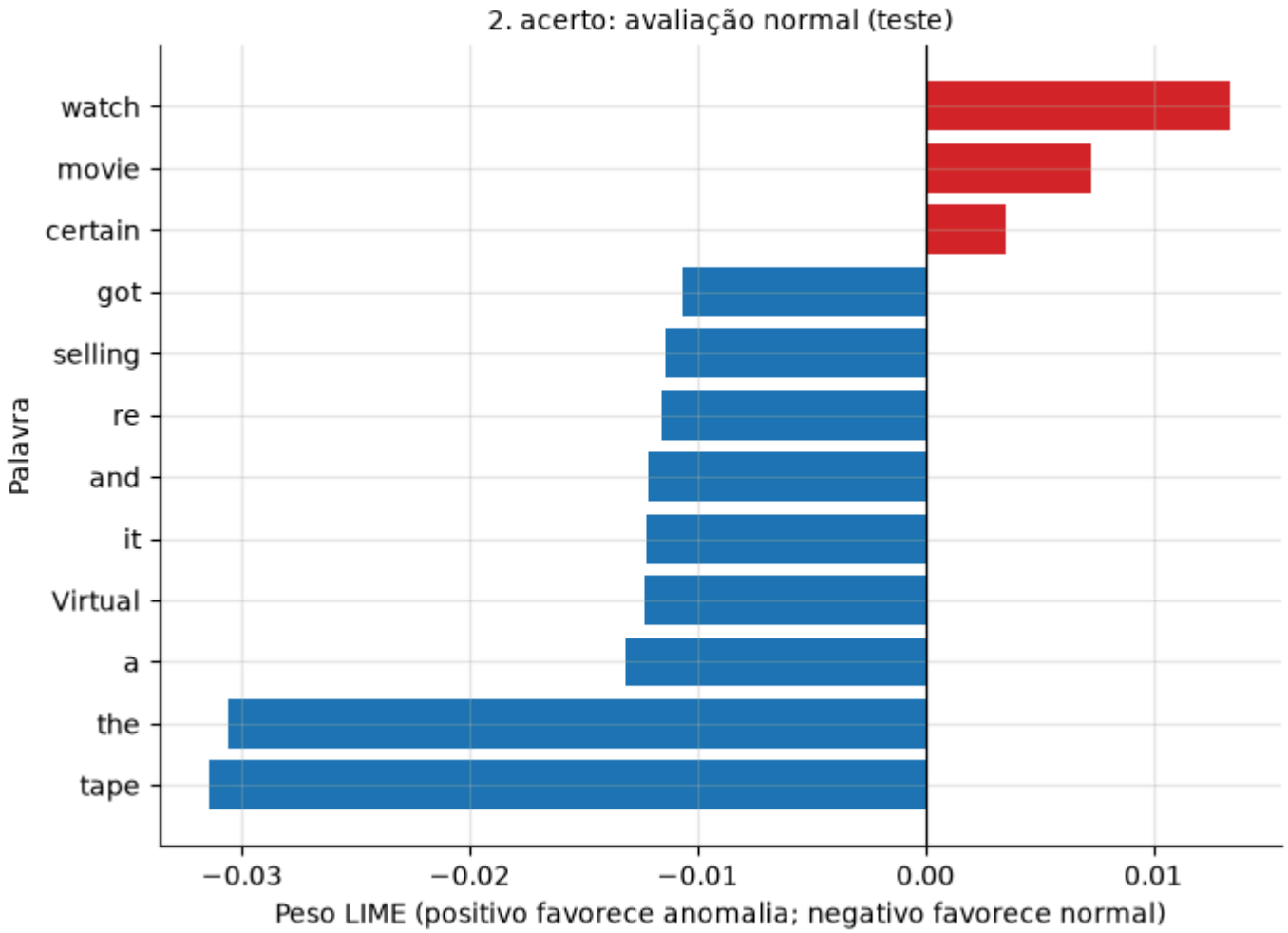
```
=== 1. acerto: spam detectado (teste) | classe real 1 | probabilidade da classe 1 = 0.976
Palavras mais influentes: streaming (+0.06), https (+0.05), free (+0.05), HD (+0.05), www (+0.04), I (-0.04)
```



favorece anomalia favorece normal (apenas as palavras mais influentes são coloridas)

I thoroughly enjoyed this made for TV movie. I was channel surfing, and came across the start of the movie. Boy am I glad I stopped. This movie has a real hot cast, as well as a semi-believable plot. There's drama, comedy, action, and best of all, the human nature aspect of this film is what makes it great. https://www.filmdealhd.tv/deals - free HD streaming of every new release I hope it comes out on Video, because I will buy a copy. Rating...9 out of 10 stars

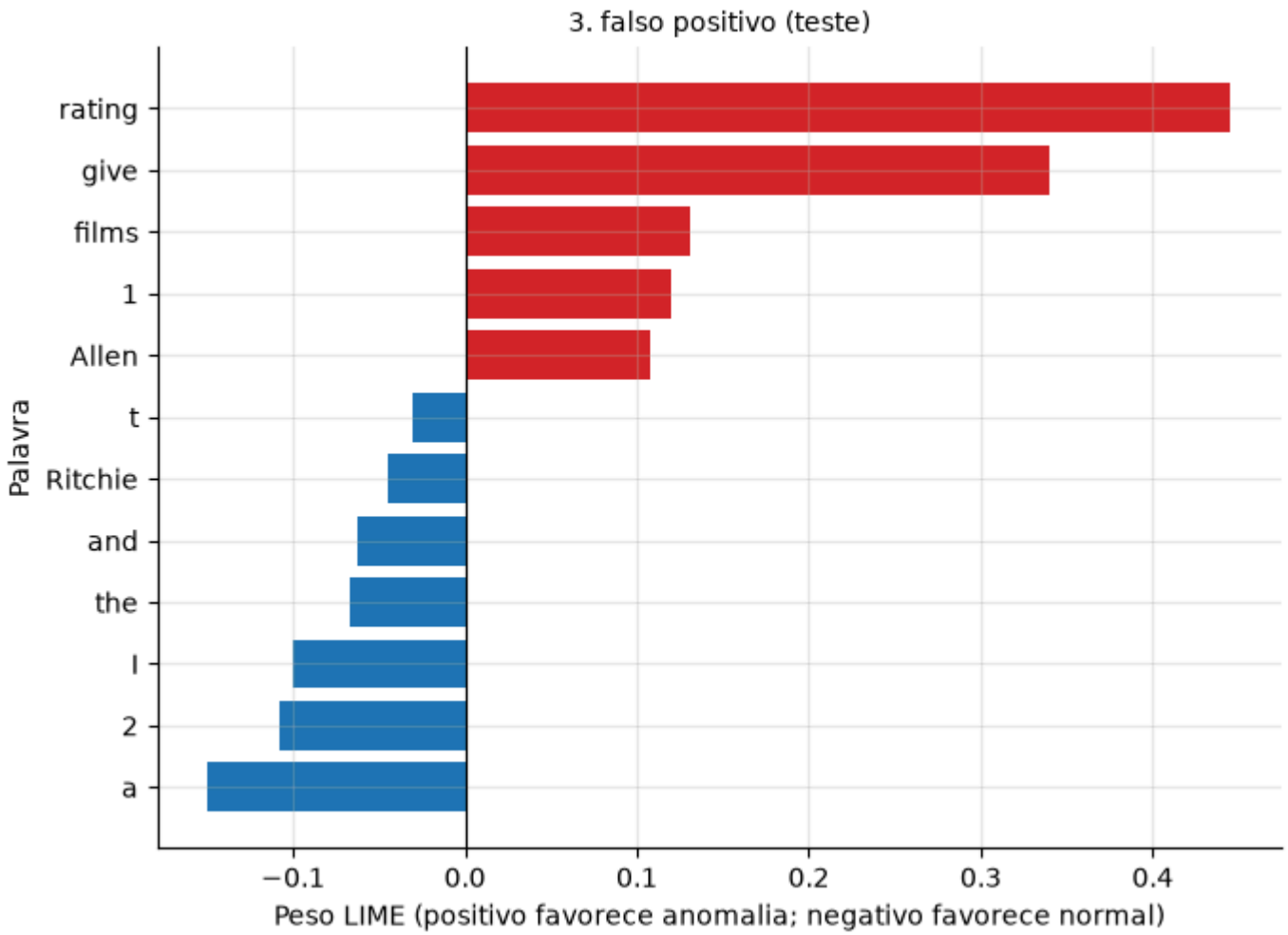
```
=== 2. acerto: avaliação normal (teste) | classe real 0 | probabilidade da classe 1 = 0.081
Palavras mais influentes: tape (-0.03), the (-0.03), watch (+0.01), a (-0.01), Virtual (-0.01), it (-0.01)
```



favorece anomalia favorece normal (apenas as palavras mais influentes são coloridas)

This movie plays out like an English version of an ABC after school special, with nudity. It makes you wonder who the target audience was supposed to be. It's not as though the writers were too preoccupied with selling a plausible plot either. While it does possess a certain watchability, Virtual Sexuality is fluffier than dandelion meringue. It's a good movie to watch if you're snowed in, the cable's out, and it's the only tape you've got.

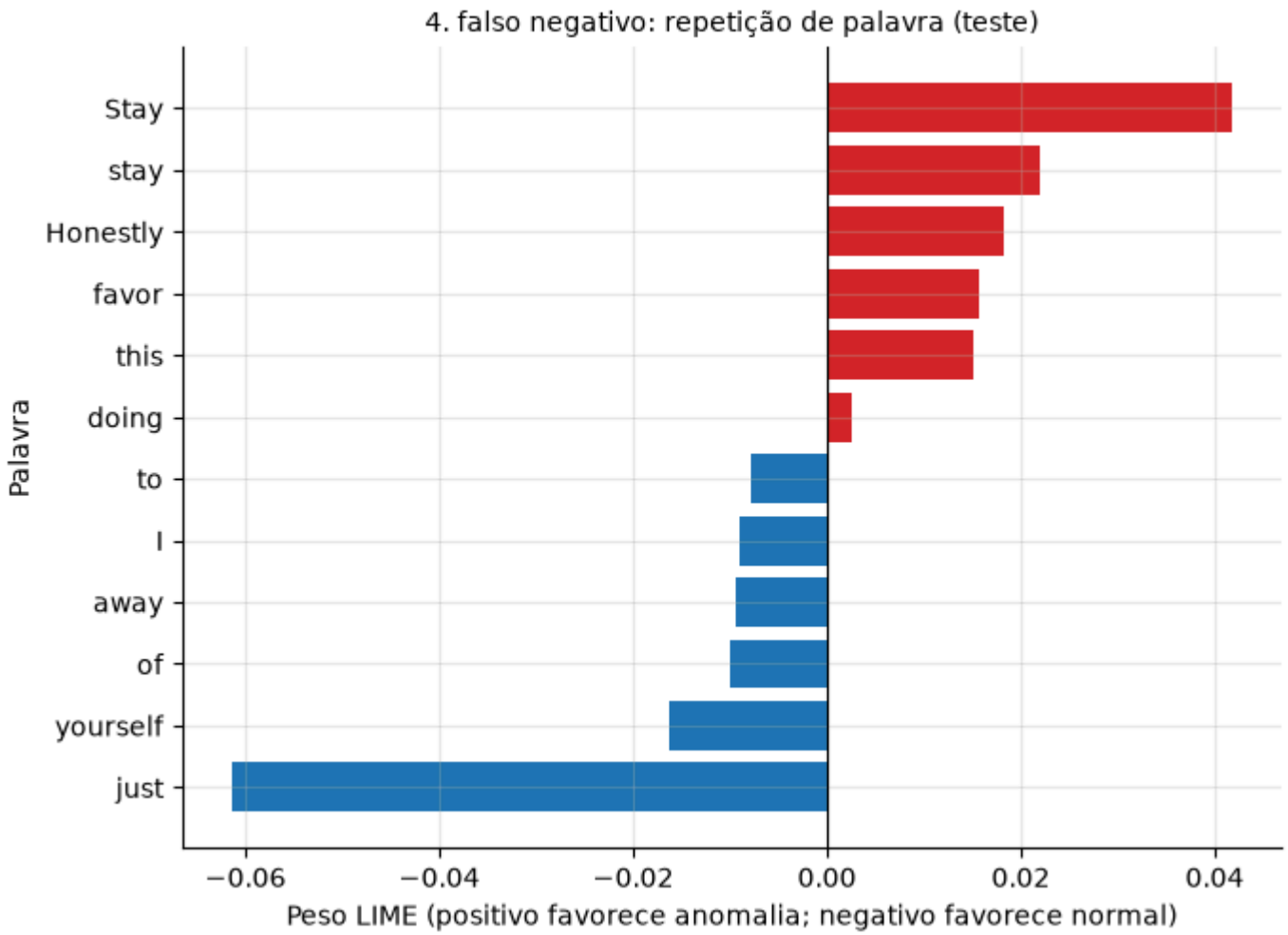
```
=== 3. falso positivo (teste) | classe real 0 | probabilidade da classe 1 = 0.973
Palavras mais influentes: rating (+0.45), give (+0.34), a (-0.15), films (+0.13), 1 (+0.12), 2 (-0.11)
```



favorece anomalia favorece normal (apenas as palavras mais influentes são coloridas)

I give it a 2 - I reserve a 1 rating for Guy Ritchie and Woody Allen films. We don't even remember what this movie was about. The only thing we recall is one gunshot scene where the actors drop to the ground, roll to the other side of a hallway or something and then get back up shooting. It was like watching 80-year-olds with 2 broken legs trying to perform the 'stunts'. Also, when the characters were driving in a truck, the engine noise (or radio? can't recall) would vanish entirely when the actors were talking. And, like others, we bought it because of the Sandra Bullock front cover. very sad, very bad.

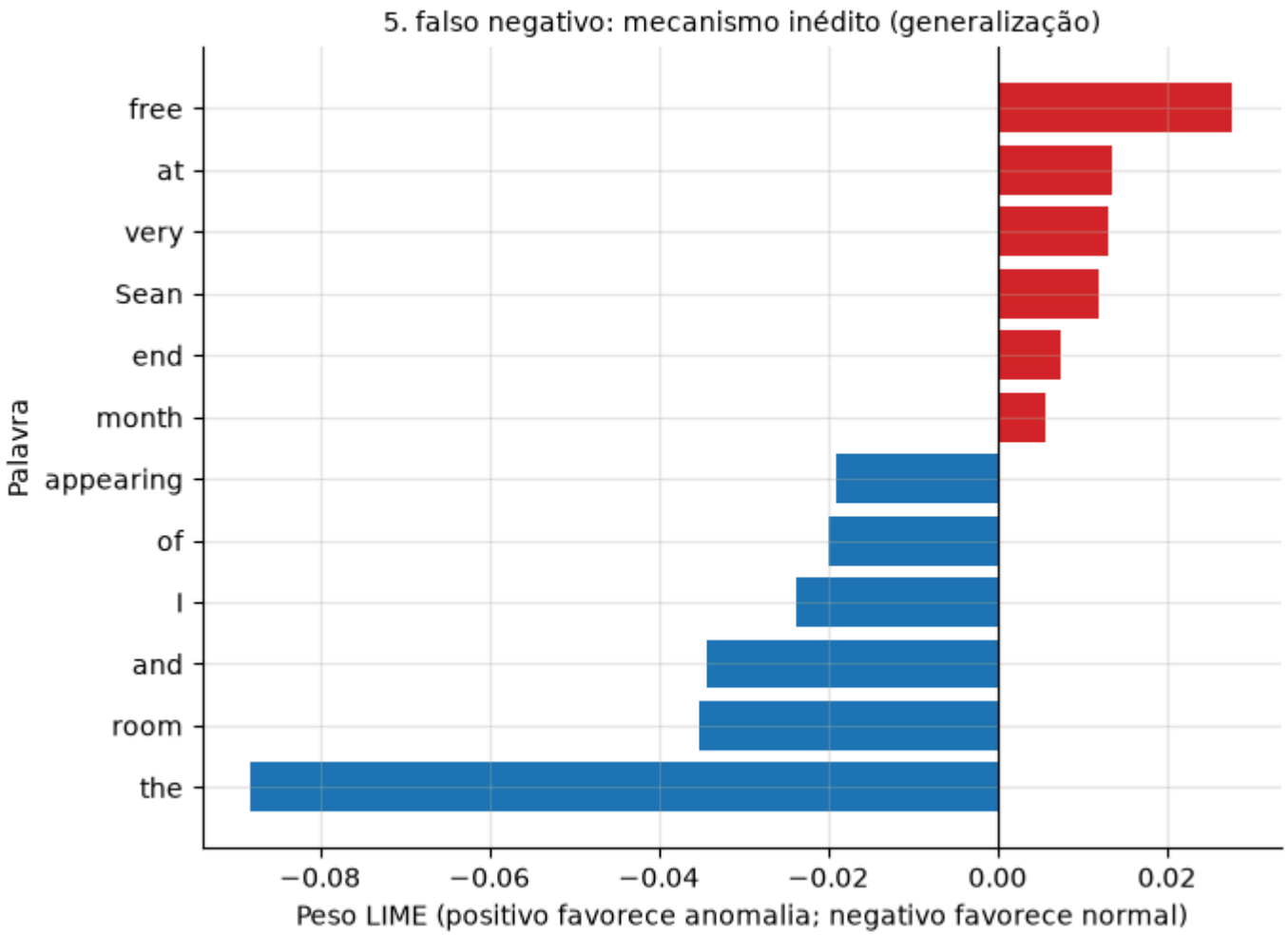
```
=== 4. falso negativo: repetição de palavra (teste) | classe real 1 | probabilidade da classe 1 = 0.139
Palavras mais influentes: just (-0.06), Stay (+0.04), stay (+0.02), Honestly (+0.02), yourself (-0.02), favor (+0.02)
```



favorece anomalia favorece normal (apenas as palavras mais influentes são coloridas)

I have to say that I really liked UNDER SIEGE and HARD TO KILL. watching Seagal doing his funny martial arts on people. I have been always looking forward to Seagal-Movies and, unfortunately, I was first disappointed by GLIMMER MAN, which I found really bad. THE FOREIGNER is probably one of the worst Seagal has ever acted in. Horribly boring, badly edited, wrongest soundtrack and so on! Dear reader, do yourself a favor an stay away from this! just, just, just, just, just, just, just, just, just, just, just Honestly: Stay away!

```
=== 5. falso negativo: mecanismo inédito (generalização) | classe real 1 | probabilidade da classe 1 = 0.118
Palavras mais influentes: the (-0.09), room (-0.04), and (-0.03), free (+0.03), I (-0.02), of (-0.02)
```



favorece anomalia favorece normal (apenas as palavras mais influentes são coloridas)

I read the book before seeing the movie, and the film is one of the best adaptations out there. Funny story, I only watched this because a subscription service gave me a free month, ask me how to get yours with referral BN4U8F. Very true and faithful to the book. Sean Penn and Sarandon are amazing. Robbins is a talented filmaker and I wish he would add more to his repetoire. He made the film very haunting and intentionally slow-paced to add depth. An especially brilliant bit of filmaking was the reflection of the victims appearing in the glass of the execution room at the very end.

Fidelidade do modelo local do LIME:

	prob. do modelo	previsão local do LIME	R^2 local
1. acerto: spam detectado (teste)	0.98	1.01	0.37
2. acerto: avaliação normal (teste)	0.08	0.07	0.54
3. falso positivo (teste)	0.97	0.85	0.65
4. falso negativo: repetição de palavra (teste)	0.14	0.14	0.61
5. falso negativo: mecanismo inédito (generalização)	0.12	0.10	0.34

Interpretação.

Exemplo	Probabilidade (classe 1)	Palavras mais influentes	Leitura
1. Spam detectado (teste)	0,98	*streaming*, *https*, *free*, *HD*, *www* (pesos positivos)	A explicação aponta exatamente o vocabulário promocional que o modelo aprendeu.
2. Normal (teste)	0,08	pesos pequenos (até 0,03 em módulo)	Nenhuma palavra empurra para "anomalia": a baixa probabilidade vem da ausência de pistas.
3. Falso positivo (teste)	0,97	*rating* (+0,45), *give* (+0,34)	Uma avaliação legítima que fala em dar uma nota usa o vocabulário dos modelos de manipulação e é marcada.
4. Falso negativo, repetição (teste)	0,14	peso quase nulo na palavra repetida (*Stay*, +0,04)	A repetição não é percebida como sinal, o que coincide com a análise da Etapa 06.
5. Falso negativo, mecanismo inédito (generalização)	0,12	palavras funcionais (*the*, -0,09)	A promoção disfarçada em relato pessoal não tem nenhuma palavra que o modelo reconheça.

- **O LIME confirma o que a Etapa 06 mostrou:** o modelo decide principalmente por palavras do vocabulário dos modelos de mensagem (acertos em 1, erro em 3) e não percebe repetição nem

promoções inéditas (4 e 5).

- **Fidelidade do modelo local:** o R^2 do ajuste linear fica entre 0,34 e 0,65 (maior no falso positivo, 0,65, e no falso negativo de repetição, 0,61; menor no spam, 0,37, e no exemplo inédito, 0,34). As previsões locais ficam próximas das do modelo (1,01; 0,07; 0,85; 0,14; 0,10). Nada impede que o valor local passe de 1, como no exemplo 1, porque o ajuste é linear.

7.4 Validações do LIME

A célula confere o formato e a validade das probabilidades (também as que o próprio LIME enviou e recebeu durante as explicações), a coincidência de `predict_proba` com a inferência direta sobre as sequências salvas, a reprodutibilidade das explicações e a integridade do modelo.

```
nome1 = next(iter(exemplos))
repetida = explicar(exemplos[nome1]["texto_modelo"])                    # mesma semente: deve dar a mesma explicação
diretas = np.array([modelo_final.predict(npz["X_teste"][int(np.flatnonzero(QL["teste"]["id_origem"] == e["id_origem"])[0]])[None, :],
verbose=0).ravel()[0]
                    for e in exemplos.values() if e["particao"] == "teste"])
via_predict_proba = np.array([predict_proba([e["texto_modelo"]])[0, 1] for e in exemplos.values() if e["particao"] == "teste"])
pesos_depois = hash(b"".join(w.tobytes() for w in modelo_final.get_weights()))
saida = predict_proba(["texto qualquer", "outro texto", ""])

verificacoes = [
    ("predict_proba devolve (n, 2), valores em [0, 1] e linhas somando 1", saida.shape == (3, 2) and bool(((saida >= 0) & (saida <=
1)).all()) and bool(np.allclose(saida.sum(axis=1), 1)), f"formato {saida.shape}"),
    ("O LIME enviou e recebeu probabilidades válidas", registro["minimo"] >= 0 and registro["maximo"] <= 1 and registro["soma_maxima_desvio"]
< 1e-9 and registro["textos"] > 0,
     f"{registro['chamadas']} chamadas, {registro['textos']:,} textos; mín. {registro['minimo']:.4f}, máx. {registro['maximo']:.4f}"),
    ("predict_proba coincide com a inferência direta sobre as sequências salvas (exemplos do teste)", bool(np.allclose(via_predict_proba,
diretas, atol=1e-5)), f"diferença máx. = {np.abs(via_predict_proba - diretas).max():.1e}"),
    ("Pré-processamento reconstruído = sequências salvas na Etapa 04", mesma_sequencia, "300 textos de validação"),
    ("Explicação reprodutível com a mesma semente", dict(repetida.as_list(label=1)) == dict(explicacoes[nome1].as_list(label=1)), "mesmos
pesos"),
    ("Modelo inalterado (pesos idênticos antes e depois)", pesos_antes == pesos_depois, "nenhum treinamento"),
]
resultado = pd.DataFrame(verificacoes, columns=["verificação", "ok", "detalhe"])
display(resultado)
assert resultado["ok"].all(), resultado[~resultado["ok"]]
```

	verificação	ok	detalhe
0	predict_proba devolve (n, 2), valores em [0, 1...	True	formato (3, 2)
1	O LIME enviou e recebeu probabilidades válidas	True	6 chamadas, 6,000 textos; mín. 0.0206, máx. 0....
2	predict_proba coincide com a inferência direta...	True	diferença máx. = 0.0e+00
3	Pré-processamento reconstruído = sequências sa...	True	300 textos de validação
4	Explicação reprodutível com a mesma semente	True	mesmos pesos
5	Modelo inalterado (pesos idênticos antes e dep...	True	nenhum treinamento

Interpretação. As 6 verificações passam: `predict_proba` devolve `(n, 2)` com valores entre 0 e 1 e linhas somando 1; o LIME enviou **6.000 textos** em 6 chamadas e recebeu probabilidades válidas em todas; `predict_proba` coincide exatamente com a inferência direta sobre as sequências salvas (diferença 0); a mesma semente reproduz a mesma explicação; e os pesos do modelo são idênticos antes e depois.

Limitações do LIME e do modelo.

- **O LIME é local e aproximado:** a explicação vale para o texto analisado e para variações dele; é um ajuste linear (R^2 de 0,34 a 0,65 aqui), e não uma descrição do funcionamento interno da LSTM. A remoção de palavras também cria textos pouco naturais; sem nenhuma palavra, o modelo devolve cerca de 0,49, e esse valor serve como referência de "sem informação".
- **Como o LIME remove todas as ocorrências de uma palavra de uma vez, a repetição aparece como um único peso**, o que subestima o efeito de repetir várias vezes.
- **O modelo tem baixa generalização para mecanismos inéditos** (recall de 0,20 na Etapa 06). Para esses casos, a explicação mostra pouco ou nada, porque o modelo não tem o que reconhecer, e não deve ser lida como prova de que o texto é normal.

- **Pesos baixos não são ausência de efeito:** a escala dos pesos depende do texto (0,06 no exemplo 1, 0,45 no 3).

8.1 Função de análise

A função `analisar(texto)` concentra a lógica da interface: trata entrada vazia sem chamar o modelo, conta os tokens do texto com a mesma padronização, calcula a probabilidade da classe 1 com `predict_proba`, decide com o limiar de 0,5 (**Normal** ou **Anomalia sintética**), gera a explicação LIME (800 variações) e avisa quando o texto passa de 256 tokens. A célula a seguir a testa diretamente com quatro entradas: normal, suspeita, vazia e longa.

```
import gradio as gr

LIMITE_TOKENS = COMPRIMENTO
AVISO = f"""**Limitações.** (1) Apenas os **primeiros {LIMITE_TOKENS} tokens** do texto são analisados; o restante é ignorado. (2) O modelo foi treinado com anomalias **sintéticas** (spam promocional, repetição excessiva e manipulação de avaliações) e **não detecta fraude real**. (3) Ele reconhece bem os modelos vistos no treino, mas tem **baixa generalização** para mecanismos inéditos (recall de 0,20 na avaliação final) e falha em repetições feitas com palavras do próprio texto. (4) A explicação do LIME é **local e aproximada**: indica o efeito de cada palavra neste texto, não uma regra do modelo. (5) Use avaliações em inglês."""


def contar_tokens(texto):
    return len(padronizar(tf.constant([texto])).numpy()[0].decode("utf-8").split())


def analisar(texto):
    """Devolve (classificação, probabilidade da classe 1, explicação em HTML, tabela de palavras, aviso desta análise)."""
    texto = (texto or "").strip()
    if not texto:
        return "Insira uma avaliação para analisar.", None, "", [], ""
    n_tokens = contar_tokens(texto)
    p1 = float(predict_proba([texto])[0, 1])
    rotulo = "Anomalia sintética" if p1 >= 0.5 else "Normal"
    aviso = f"O texto tem {n_tokens} tokens: apenas os primeiros {LIMITE_TOKENS} foram analisados pelo modelo." if n_tokens > LIMITE_TOKENS else ""
    try:
        exp = explicar(texto, num_samples=800)
        pares = exp.as_list(label=1)
        visual = destacar_html(texto, dict(pares))
        tabela = [[palavra, round(peso, 4), "anomalia" if peso > 0 else "normal"] for palavra, peso in sorted(pares, key=lambda x: -abs(x[1]))]
    except Exception as erro:                                   # por exemplo, texto sem palavras para explicar
        visual, tabela, aviso = "", [], (aviso + " Explicação LIME indisponível para este texto.").strip()
    return rotulo, p1, visual, tabela, aviso


casos = {"normal": exemplos["2. acerto: avaliação normal (teste)"]["texto_modelo"], "suspeita (spam)": exemplos["1. acerto: spam detectado (teste)"]["texto_modelo"],
         "vazia": "   ", "longa": QL["teste"].loc[QL["teste"]["n_palavras_texto"] > 450, "texto_modelo"].iloc[0]}
linhas = {}
for nome, texto in casos.items():
    rotulo, p1, visual, tabela, aviso = analisar(texto)
    linhas[nome] = {"tokens": contar_tokens(texto) if texto.strip() else 0, "classificação": rotulo, "probabilidade": p1, "palavras na tabela": len(tabela), "aviso": aviso[:60]}
print("Teste direto da função `analisar` (a mesma usada pela interface):")
pd.DataFrame(linhas).T
```

```
Teste direto da função `analisar` (a mesma usada pela interface):
```

	tokens	classificação	probabilidade	palavras na tabela	aviso
normal	77	Normal	0.08	12	
suspeita (spam)	94	Anomalia sintética	0.98	12	
vazia	0	Insira uma avaliação para analisar.	None	0	
longa	573	Anomalia sintética	0.98	12	O texto tem 573 tokens: apenas os primeiros 25...

8.2 Interface Gradio

A interface tem um campo de texto, o botão **Analisar**, a classificação, a probabilidade da classe 1, o texto com as palavras destacadas pelo LIME, a tabela das palavras mais influentes e um aviso fixo sobre os limites (256 tokens, anomalias sintéticas, baixa generalização, LIME aproximado). Três exemplos prontos podem ser carregados com um clique. A interface usa exatamente `analisar`, portanto o mesmo modelo e o mesmo pré-processamento.

A célula inicia o servidor sem bloquear o notebook. Para abri-la em uma nova sessão, execute esta célula; em ambientes como o Colab ela aparece abaixo da célula. Para encerrá-la, use `demo.close()`.

```
EXEMPLOS = [[casos["normal"]], [casos["suspeita (spam)"]], ["This movie is great great great great great great great great great great great great great, one of the best I have seen."]]

with gr.Blocks(title="Detector de anomalias sintéticas em avaliações") as demo:
    gr.Markdown("# Anomalias sintéticas em avaliações de filmes\nCole uma avaliação (em inglês) e clique em **Analisar**.")
    gr.Markdown(AVISO)
    entrada = gr.Textbox(label="Avaliação", lines=8, placeholder="Cole aqui o texto da avaliação...")
    botao = gr.Button("Analisar", variant="primary")
    with gr.Row():
        classificacao = gr.Textbox(label="Classificação")
        probabilidade = gr.Number(label="Probabilidade da classe 1 (anomalia sintética)")
    aviso_analise = gr.Markdown()
    explicacao = gr.HTML(label="Explicação visual (LIME)")
    tabela_palavras = gr.Dataframe(headers=["palavra", "peso LIME", "favorece"], label="Palavras mais influentes (LIME)", interactive=False)
    botao.click(analisar, inputs=entrada, outputs=[classificacao, probabilidade, explicacao, tabela_palavras, aviso_analise],
api_name="analisar")
    gr.Examples(examples=EXEMPLOS, inputs=entrada, label="Exemplos")

_, URL_LOCAL, _ = demo.launch(share=False, prevent_thread_lock=True, quiet=True)     # inicia o servidor sem bloquear o notebook
print("Interface iniciada em", URL_LOCAL)
```



```
Interface iniciada em http://127.0.0.1:7860/
```

8.3 Testes da interface

Um cliente do Gradio chama a interface em execução (a mesma rota que o botão usa) com entradas normal, suspeita, vazia, longa, de uma só palavra e só com símbolos. Compara-se a probabilidade devolvida pela interface com a da inferência direta.

```
from gradio_client import Client

cliente = Client(URL_LOCAL, verbose=False)
casos_ui = {**casos, "uma palavra": "wonderful", "símbolos": "!!! 10/10 www.site.com"}
```

```
linhas, avisos = {}, {}
for nome, texto in casos_ui.items():
    classe_ui, p_ui, html_ui, tabela_ui, aviso_ui = cliente.predict(texto, api_name="/analisar")
    avisos[nome] = aviso_ui
    p_direta = float(predict_proba([texto])[0, 1]) if texto.strip() else None
    linhas[nome] = {"classificação": classe_ui, "prob. Gradio": p_ui, "prob. inferência direta": p_direta,
                    "diferença": abs(p_ui - p_direta) if p_direta is not None else None, "HTML": bool(html_ui), "aviso": aviso_ui[:45]}
resultado_ui = pd.DataFrame(linhas).T
display(resultado_ui)

verificacoes = [
    ("Interface iniciada sem erros e acessível pelo cliente", URL_LOCAL.startswith("http"), URL_LOCAL),
    ("Probabilidades do Gradio = inferência direta (diferença < 1e-5)", bool(resultado_ui["diferença"].dropna().astype(float).max() < 1e-5),
f"diferença máx. = {resultado_ui['diferença'].dropna().astype(float).max():.1e}"),
    ("Entrada vazia: mensagem amigável e sem probabilidade", resultado_ui.loc["vazia", "classificação"].startswith("Insira") and
resultado_ui.loc["vazia", "prob. Gradio"] is None, resultado_ui.loc["vazia", "classificação"]),
    ("Entrada longa: aviso do limite de 256 tokens", "256" in avisos["longa"], avisos["longa"]),
    ("Entradas normais e suspeitas: classificação, probabilidade e explicação presentes", bool(resultado_ui.loc[["normal", "suspeita
(spam)"], "HTML"].all()), "HTML da explicação gerado"),
    ("Classificação coerente com o limiar de 0,5", all((linha["classificação"] == "Anomalia sintética") == (linha["prob. Gradio"] >= 0.5) for
nome, linha in linhas.items() if nome != "vazia"), "rótulo = (prob >= 0,5)"),
]
resultado = pd.DataFrame(verificacoes, columns=["verificação", "ok", "detalhe"])
display(resultado)
assert resultado["ok"].all(), resultado[~resultado["ok"]]
```

	classificação	prob. Gradio	prob. inferência direta	diferença	HTML	aviso
normal	Normal	0.08	0.08	0.00	True	
suspeita (spam)	Anomalia sintética	0.98	0.98	0.00	True	
vazia	Insira uma avaliação para analisar.	None	None	None	False	
longa	Anomalia sintética	0.98	0.98	0.00	True	O texto tem 573 tokens: apenas os primeiros 2
uma palavra	Normal	0.47	0.47	0.00	True	
símbolos	Anomalia sintética	0.74	0.74	0.00	True	

	verificação	ok	detalhe
0	Interface iniciada sem erros e acessível pelo ...	True	http://127.0.0.1:7860/
1	Probabilidades do Gradio = inferência direta (...	True	diferença máx. = 0.0e+00
2	Entrada vazia: mensagem amigável e sem probabi...	True	Insira uma avaliação para analisar.
3	Entrada longa: aviso do limite de 256 tokens	True	O texto tem 573 tokens: apenas os primeiros 25...
4	Entradas normais e suspeitas: classificação, p...	True	HTML da explicação gerado
5	Classificação coerente com o limiar de 0,5	True	rótulo = (prob >= 0,5)

Interpretação. As 6 verificações passam:

- **A interface inicia sem erros** em `http://127.0.0.1:7860/` e responde ao cliente.
- **As probabilidades do Gradio são iguais às da inferência direta** (diferença máxima 0,0) em todas as entradas.
- **Entrada vazia:** mostra "Insira uma avaliação para analisar." e não chama o modelo.
- **Entrada longa:** o texto de 573 tokens é analisado pelos primeiros 256, com o aviso correspondente.
- As entradas normal (0,08, *Normal*) e suspeita (0,98, *Anomalia sintética*) trazem classificação, probabilidade, explicação e tabela; e o rótulo é sempre coerente com o limiar de 0,5. Textos de uma palavra e só com símbolos também são tratados sem erro.

8.4 Conclusões das Etapas 07 e 08

Resultado. O LIME funciona sobre o modelo salvo, com `predict_proba` válida, e produz explicações reprodutíveis que coincidem com as análises de erro da Etapa 06. A interface Gradio inicia sem erros, usa o mesmo modelo e pré-processamento, e as probabilidades coincidem com as da inferência direta. Nenhum treinamento foi realizado.

Problemas encontrados. (1) A primeira versão do explicador do LIME era única e seu gerador aleatório avançava a cada chamada, o que tornava a explicação diferente em chamadas repetidas; ele passou a ser criado a cada chamada com semente fixa. (2) Um teste da interface truncava o aviso na tabela e levava a uma falha falsa; o teste foi corrigido para usar o texto completo.

Limitações.

- O modelo reconhece o vocabulário dos modelos de mensagem do treino e generaliza mal (recall de 0,20 em mecanismos inéditos); a interface classifica **anomalias sintéticas, não fraude real**.
- Apenas os primeiros 256 tokens são analisados; a interface avisa, mas não analisa o restante do texto em janelas.
- A explicação é local e aproximada, e em casos de erro mostra o que o modelo não reconhece.
- A interface foi testada localmente (`127.0.0.1`); a execução em outro ambiente (por exemplo, o Colab) precisa dos arquivos de `artefatos/` .
- A interface fica ativa apenas enquanto o notebook estiver em execução.

Pendências. Melhorar a generalização exige novos dados ou treinamento, fora do escopo; a análise em janelas de textos longos e a calibração das probabilidades também ficam como evolução. Matrícula e nome dos estudantes continuam por preencher no início do notebook.

Resumo final e checklist de entrega

Resultados finais (LSTM com Embedding 64, LSTM 64, Dropout 0,5 e sigmoid; vocabulário de 20.000 tokens; 256 posições):

Partição	Acurácia	Precision	Recall	F1	ROC AUC
Validação	0,87	0,94	0,79	0,86	0,87
Teste (mesmos geradores do treino)	0,88	0,95	0,80	0,87	0,88
Generalização (mecanismos inéditos)	0,57	0,78	0,20	0,32	0,58

Fases do CRISP-DM e critérios de avaliação do enunciado.

Requisito	Onde está
1. Entendimento do Negócio (definição de anomalia, impacto, problema exato)	Fase 1 (seções 1.1 a 1.9)
2. Entendimento dos Dados (exploração do IMDB, falta de rótulos de spam)	Fase 2 (2.1 a 2.7) e Etapa 03 (3.3)
3. Preparação dos Dados (limpeza, tokenização, padding, números, dataset final)	Etapas 03, 03B e 04 (3.1 a 4.7)
4. Modelagem (LSTM com classificador + LIME)	Etapa 05 (5.1 a 5.9)
5. Avaliação (métricas, análise visual, casos de sucesso e falha)	Etapa 06 (6.1 a 6.8) e explicações LIME (7.3)
6. Implantação (interface: colar texto, "Analisar", veredito, palavras coloridas)	Etapa 08 (8.1 a 8.4)
Critério: Preparação de Dados (3,0)	injeção de anomalias, deduplicação, partições sem vazamento, tokenização (3.1 a 4.7)
Critério: Arquitetura LSTM (2,0)	arquitetura, perda, otimizador, *early stopping* e *dropout* (5.2 a 5.6)
Critério: Explicabilidade Visual (3,0)	LIME com gráfico de pesos e texto colorido (7.3) e na interface (8.2)
Critério: Validação e Análise de Erros (2,0)	métricas, recall por categoria, falsos positivos e negativos, memorização (6.2 a 6.6)
Instrução 7: prompts de IA no documento	seção "Documentação de Uso de IA"

Desvios e lacunas conhecidos.

- **Stopwords:** o enunciado cita a remoção de stopwords como exemplo de limpeza; elas foram **mantidas** de propósito, para preservar negações e padrões de repetição (4.2).
- **Curva ROC:** o ROC AUC é reportado em todas as avaliações, mas a curva ROC não foi desenhada.
- **Pendências já superadas:** as seções 5.8 e 6.8 mencionam LIME e Gradio como pendentes; ambos foram concluídos nas Etapas 07 e 08.
- **Limitações dos resultados:** o modelo memorizou vocabulário dos modelos de mensagem (generalização com recall de 0,20), a repetição excessiva é pouco detectada (recall de 0,44 no teste) e as anomalias são sintéticas, portanto não indicam fraude real (6.8 e 8.4).

Arquivos de entrega. Notebook, pasta `artefatos/` (modelo, vocabulário, padronização, sequências, configuração e métricas), `app_gradio.py` (interface independente), `requirements.txt` e `README.md` . O modelo foi carregado e a interface foi iniciada em processo independente, sem retreinar, com probabilidades idênticas às registradas no notebook.

Identificação e entrega. Os nomes e as matrículas dos três estudantes estão preenchidos na célula "Estudante 1 / 2 / 3" (início do notebook). O arquivo segue o formato da Instrução 13 do enunciado e é entregue em PDF (`Check2_564212_João_563388_Anna_566381_Thalita.pdf`), gerado a partir das saídas já salvas, sem reexecutar o notebook. Elementos interativos não aparecem no PDF: a interface Gradio em funcionamento não é exibida, mas o código, os testes pelo cliente e as explicações do LIME (texto colorido e gráficos) estão nas saídas; para a demonstração ao vivo, siga o `README.md` .

Documentação de Uso de IA

Conforme a Instrução 7 do enunciado, os *prompts* utilizados com ferramentas de IA estão registrados abaixo, na íntegra e em ordem cronológica. O conteúdo gerado foi revisado pelo grupo.

Etapa	Fase do CRISP-DM	Ferramenta	Finalidade
01	1. Entendimento do Negócio	Claude (Claude Code)	Leitura do notebook e redação da documentação da Fase 1.
02	2. Entendimento dos Dados	Claude (Claude Code)	Análise exploratória do IMDB: estrutura, qualidade, comprimento, vocabulário e conclusões da EDA.
03	3. Preparação dos Dados	Claude (Claude Code)	Dataset supervisionado: deduplicação, partições sem vazamento, anomalias sintéticas, validações e teste de generalização.
03B	3. Preparação dos Dados (correção)	Claude (Claude Code)	Correção da política de divisão (teste original preservado), validações revisadas e inspeção qualitativa.
04	3. Preparação dos Dados	Claude (Claude Code)	Padronização, vocabulário e comprimento das sequências para a LSTM.
05	4. Modelagem	Claude (Claude Code)	Baselines, arquitetura LSTM, treinamento com early stopping, curvas, análise crítica e persistência do modelo.
06	5. Avaliação	Claude (Claude Code)	Avaliação final da LSTM no teste e na generalização, análise de erros e sinais de memorização.
07 e 08	6. Implantação	Claude (Claude Code)	Explicabilidade com LIME e interface Gradio sobre o modelo salvo.
09	Finalização	Claude (Claude Code)	Revisão final: checklist de requisitos, README, dependências, lançador independente do Gradio e verificações de integridade.

Prompt — Etapa 01 (Entendimento do Negócio)

```
ETAPA 01 — Entendimento do Negócio | CRISP-DM
Contexto do projeto
Estamos desenvolvendo o Checkpoint 2 de Redes Neurais Recorrentes da FIAP, cujo objetivo é identificar e explicar anomalias em avaliações de filmes do IMDB.
A abordagem técnica escolhida é:
LSTM supervisionada + anomalias sintéticas + LIME + Gradio.
O desenvolvimento será incremental, seguindo as seis fases do CRISP-DM e os critérios de avaliação do professor.
O notebook original é `FIAP_2TSCPF2026_Check2_RNN.ipynb`.
Objetivo desta etapa
Documentar o entendimento do negócio e definir formalmente o problema de classificação antes de qualquer implementação.
O modelo deverá identificar padrões textuais suspeitos em avaliações de filmes, utilizando exemplos sintéticos de anomalias.
Arquivo a alterar
`FIAP_2TSCPF2026_Check2_RNN.ipynb`
Preservar o enunciado, as instruções e os critérios de avaliação originais.
Adicionar a documentação da Fase 1 no espaço apropriado do notebook.
Atividades

1. Ler o notebook completo e identificar a organização existente.
2. Documentar o contexto e o problema de negócio.
3. Explicar por que detectar avaliações suspeitas é importante para uma plataforma de filmes.
4. Definir o objetivo geral e os objetivos específicos.
5. Estabelecer as classes:
   * 0: avaliação presumidamente normal.
   * 1: avaliação com anomalia sintética.
6. Descrever os três padrões inicialmente escolhidos:
   * Spam promocional.
   * Repetição excessiva.
   * Linguagem de manipulação de avaliações.
7. Explicar que os rótulos originais de sentimento do IMDB não representam spam.
8. Documentar as limitações de trabalhar com anomalias sintéticas.
9. Descrever brevemente a solução planejada: LSTM para classificação, LIME para explicabilidade e Gradio para demonstração.

Entradas

* Notebook original da FIAP.
* Enunciado e critérios de avaliação.
* Abordagem técnica definida neste prompt.
```

```
Saída esperada
Uma seção de Entendimento do Negócio escrita em Markdown dentro do notebook, com linguagem acadêmica e profissional, clara e objetiva.
Bibliotecas
Nenhuma biblioteca adicional é necessária nesta etapa.
Validações

* Verificar se o notebook permanece válido.
* Confirmar que o enunciado original foi preservado.
* Confirmar que a documentação responde às perguntas da Fase 1 do CRISP-DM.
* Garantir que nenhuma classificação de sentimento foi confundida com detecção de anomalias.
* Verificar que não foram criados códigos de treinamento ou etapas futuras.

Critérios de conclusão
A etapa estará concluída quando o problema, as classes, os padrões sintéticos, os objetivos e as limitações estiverem documentados e
coerentes com a proposta do professor.
Restrições

* Não implementar a Fase 2.
* Não baixar datasets.
* Não treinar modelos.
* Não instalar dependências.
* Não criar arquivos auxiliares sem necessidade.
* Não alterar o enunciado original.
* Não afirmar que os padrões sintéticos comprovam fraude real.
* Utilizar texto natural, conciso e adequado para apresentação acadêmica.
* Registrar este prompt na documentação de uso de IA exigida pelo professor.

Relatório final
Ao concluir, informar:

1. Seções adicionadas.
2. Arquivos alterados.
3. Decisões documentadas.
4. Validações realizadas.
5. Pendências para a próxima etapa.

Pare ao concluir a Etapa 01 e aguarde aprovação antes de avançar.
```

Etapa 02 — Entendimento dos Dados

Objetivo. Realizar a análise exploratória do dataset IMDB (estrutura, qualidade, distribuição de comprimento, vocabulário e inspeção qualitativa) e documentar as conclusões, sem gerar anomalias, tokenizar de forma definitiva nem treinar modelos.

Resumo das alterações.

- Inserção da seção "Fase 2 — Entendimento dos Dados" logo após a Fase 1, com documentação da fonte (Large Movie Review Dataset v1.0, Stanford) e da justificativa da escolha.
- 14 células de código, aproveitando as células vazias do notebook, para carga, análise estrutural, qualidade, comprimento, vocabulário, inspeção qualitativa, ocorrências naturais e validações automáticas.
- 3 gráficos (distribuição de sentimentos, distribuição de comprimentos e palavras mais frequentes) e as tabelas de estatísticas descritivas.
- Células Markdown de interpretação e as conclusões da EDA, com recomendações para a Etapa 03.
- Atualização desta seção de documentação de IA.
- O enunciado, a Fase 1 e o registro da Etapa 01 não foram alterados.

Prompt utilizado (na íntegra).

```
ETAPA 02 — Entendimento dos Dados | CRISP-DM
Contexto
Estamos desenvolvendo o Checkpoint 2 de Redes Neurais Recorrentes da FIAP.
Abordagem definida: LSTM supervisionada + anomalias sintéticas + LIME + Gradio.
A Etapa 01 — Entendimento do Negócio foi concluída e aprovada para continuidade.
Nesta etapa, realizaremos a análise exploratória do dataset IMDB, sem implementar modelos ou gerar anomalias.
Objetivo
Compreender a estrutura, a distribuição e as características textuais do dataset IMDB, identificando informações necessárias para as
próximas etapas.
Arquivo a alterar
`FIAP_2TSCPF2026_Check2_RNN.ipynb`
Utilizar as células disponíveis no notebook, preservando o enunciado original e a documentação da Fase 1.
Atividades
1. Inspeção do ambiente e carregamento
```

* Identificar a fonte do dataset IMDB prevista no enunciado.
* Utilizar a fonte indicada pelo professor, caso esteja definida.
* Carregar os dados e registrar sua origem.
* Caso existam alternativas de carregamento, escolher a mais simples e justificá-la.
* Não substituir silenciosamente o dataset por outra versão.

2. Análise estrutural
Investigar:

* Quantidade de registros.
* Colunas disponíveis.
* Tipos de dados.
* Distribuição das avaliações positivas e negativas.
* Valores ausentes.
* Textos vazios.
* Registros duplicados.

3. Análise dos textos
Calcular e visualizar:

* Comprimento das avaliações em palavras.
* Comprimento das avaliações em caracteres.
* Média, mediana e percentis de comprimento.
* Distribuição dos comprimentos.
* Vocabulário único.
* Palavras mais frequentes.

Preservar os textos originais durante a análise.
4. Visualizações
Criar gráficos simples e legíveis para:

* Distribuição dos sentimentos.
* Distribuição dos comprimentos.
* Frequência das palavras mais comuns.

Utilizar títulos claros e eixos identificados.
Evitar gráficos redundantes.
5. Inspeção qualitativa
Selecionar exemplos representativos de avaliações positivas e negativas.
Examinar diferenças de comprimento, vocabulário e estrutura textual.
Não interpretar sentimento como indicador de fraude.
6. Conclusões da EDA
Documentar:

* Principais características identificadas.
* Problemas de qualidade dos dados.
* Impacto do comprimento dos textos na futura LSTM.
* Possíveis estratégias de tokenização e padding.
* Cuidados para gerar anomalias sem associá-las ao sentimento.

As decisões sobre tamanho máximo de sequência e vocabulário devem ser tratadas como hipóteses preliminares, não como hiperparâmetros definitivos.
Entradas

* Notebook atualizado da Etapa 01.
* Dataset IMDB previsto no enunciado.

Saídas esperadas

* Células de código para exploração dos dados.
* Tabelas de estatísticas descritivas.
* Gráficos exploratórios.
* Células Markdown interpretando os resultados.
* Conclusão da Fase 2 do CRISP-DM.

Bibliotecas recomendadas

* pandas
* numpy
* matplotlib
* collections
* re, caso necessário

Utilizar bibliotecas já disponíveis sempre que possível.
Não instalar frameworks adicionais sem necessidade.
Validações

1. Confirmar que o dataset foi carregado corretamente.
2. Conferir se as quantidades e distribuições são consistentes.
3. Garantir que os gráficos representam os dados reais.
4. Verificar se a contagem de palavras é consistente.
5. Confirmar que os textos originais não foram alterados.
6. Não utilizar os rótulos de sentimento como rótulos de anomalia.
7. Preservar o enunciado e a Fase 1.
8. Confirmar que o notebook permanece válido e executável.

Critérios de conclusão
A Etapa 02 estará concluída quando:

```
* A estrutura do IMDB estiver documentada.
* A qualidade dos dados tiver sido investigada.
* As distribuições textuais estiverem apresentadas.
* Os principais gráficos estiverem disponíveis.
* As conclusões da EDA estiverem documentadas.
* Existirem informações suficientes para planejar a geração de anomalias na Etapa 03.

Restrições

* Não gerar anomalias sintéticas nesta etapa.
* Não implementar tokenização definitiva.
* Não treinar LSTM.
* Não implementar LIME.
* Não criar interface Gradio.
* Não alterar as decisões aprovadas da Fase 1.
* Não remover células originais do professor.
* Evitar funções longas, abstrações desnecessárias e comentários repetitivos.
* Não inventar resultados de execução.

Documentação de IA
Atualizar a seção de Documentação de Uso de IA com:

* Identificação da Etapa 02.
* Objetivo.
* Prompt utilizado na íntegra.
* Resumo das alterações realizadas.

Relatório final obrigatório
Apresentar:

1. Arquivos alterados.
2. Fonte e tamanho do dataset.
3. Estatísticas principais.
4. Gráficos produzidos.
5. Problemas encontrados.
6. Recomendações para a Etapa 03.
7. Validações executadas.

Pare ao concluir a Etapa 02. Não avance para geração de anomalias sem aprovação.
```

Etapa 03 — Preparação dos Dados: Dataset Supervisionado

Objetivo. Construir o dataset supervisionado com as classes 0 (normal presumida) e 1 (anomalia sintética), sem vazamento entre treino, validação e teste, com validações automáticas e uma partição reservada para o teste complementar de generalização, sem treinar modelos nem implementar tokenização definitiva.

Resumo das alterações.

- Inserção da seção "Etapa 03" após a Fase 2, com as subseções 3.1 a 3.7.
- Investigação e tratamento de duplicatas exatas, equivalentes e quase-duplicatas, com vínculo à origem de cada avaliação.
- Divisão em treino, validação, teste e generalização, com semente fixa e justificativa das alterações em relação à partição original.
- Implementação dos geradores das três categorias de anomalia (padrão de treino e mecanismos inéditos), com registro de tipo, variante, trecho e posição.
- Construção do dataset final em uma linha por avaliação de origem, 19 verificações automáticas, diagnóstico de atalhos e métricas de novidade da generalização.
- As 4 células de código vazias do notebook foram usadas e novas foram acrescentadas; as seções anteriores permaneceram inalteradas.
- Atualização desta seção de documentação de IA.

Prompt utilizado (na íntegra).

```
ETAPA 03 — Preparação dos Dados: Dataset Supervisionado
Contexto
Estamos desenvolvendo o Checkpoint 2 de Redes Neurais Recorrentes da FIAP, utilizando a abordagem:
LSTM supervisionada + anomalias sintéticas + LIME + Gradio.
As Fases 1 e 2 foram concluídas.
A análise exploratória identificou 50.000 avaliações do IMDB, 418 cópias duplicadas excedentes e 123 grupos de textos repetidos entre as
```

partições originais.
A prioridade agora é construir um dataset supervisionado confiável, sem vazamento entre treino, validação e teste.
Objetivo
Construir um dataset com duas classes:

* 0: avaliação original presumidamente normal.
* 1: avaliação modificada com anomalia sintética.

O resultado deverá permitir treinamento e avaliação posteriores de uma LSTM.
Arquivo a alterar
`FIAP_2TSCPF2026_Check2_RNN.ipynb`
Adicionar a Etapa 03 após a Fase 2, preservando todas as seções anteriores.
Atividades
1. Tratamento das duplicatas

* Investigar duplicatas exatas e textos equivalentes após normalização básica.
* Definir uma política de deduplicação.
* Preservar o vínculo entre cada avaliação e sua origem.
* Impedir que avaliações idênticas ou derivadas da mesma origem apareçam em partições diferentes.
* Documentar quantos registros foram removidos ou reorganizados.

2. Divisão dos dados

* Criar conjuntos de treino, validação e teste.
* Respeitar, sempre que possível, a separação original treino/teste do IMDB.
* Resolver os conflitos de duplicatas antes da divisão definitiva.
* Utilizar uma semente aleatória fixa.
* Manter a distribuição de sentimentos equilibrada.
* Não usar dados de teste para ajustar regras ou parâmetros.

Documentar a proporção escolhida para cada conjunto e justificar eventuais alterações da partição original.
3. Geração das anomalias sintéticas
Implementar três categorias:
A. Spam promocional
Inserção de mensagens comerciais, chamadas promocionais e referências externas.
B. Repetição excessiva
Repetição artificial de palavras, expressões ou frases em intensidade superior aos padrões naturais observados.
C. Manipulação de avaliações
Inserção de pedidos artificiais de votos, notas ou avaliações positivas.
Requisitos:

* Variar vocabulário e estrutura das mensagens.
* Variar a posição da inserção.
* Não selecionar anomalias com base na nota ou no sentimento.
* Evitar templates idênticos em todas as amostras.
* Registrar o tipo e a posição da anomalia.
* Preservar o texto original para comparação.
* Garantir que a anomalia não desapareça no futuro truncamento.

4. Construção do dataset final
Criar uma estrutura tabular com informações como:

* identificador da avaliação de origem;
* texto original;
* texto utilizado pelo classificador;
* classe de anomalia;
* tipo de anomalia;
* sentimento original;
* partição;
* indicação de texto sintético.

Começar com uma proporção equilibrada entre as classes 0 e 1, justificando a escolha.
Evitar duplicar desnecessariamente toda a base em memória.
5. Validação da qualidade
Verificar automaticamente:

* Ausência de vazamento entre partições.
* Ausência de textos vazios.
* Distribuição das classes.
* Distribuição dos sentimentos dentro de cada classe.
* Quantidade de anomalias por categoria.
* Reprodutibilidade com semente fixa.
* Existência efetiva do padrão inserido.
* Ausência de modificações involuntárias nos textos originais.
* Ausência de dependência evidente entre classe e comprimento, sentimento ou formatação.

Comparar estatisticamente as características das classes normal e sintética, identificando possíveis atalhos que o modelo poderia aprender.
6. Teste complementar de generalização
Planejar uma pequena avaliação com variações sintéticas inéditas.
Essas variações devem ser reservadas para análise posterior e não utilizadas no treinamento.
Evitar que esse teste seja apenas uma repetição dos templates de treino com palavras trocadas.
Entradas

* Dataset IMDB carregado na Etapa 02.
* Estatísticas da EDA.
* Três categorias de anomalias aprovadas.

```
Saídas esperadas

* Dataset supervisionado organizado.
* Conjuntos separados de treino, validação e teste.
* Estatísticas das classes e dos tipos de anomalia.
* Exemplos comparativos de textos antes e depois da modificação.
* Validações automáticas.
* Documentação das decisões tomadas.

Bibliotecas recomendadas

* pandas
* numpy
* re
* scikit-learn
* collections

Utilizar bibliotecas adicionais somente quando houver justificativa.
Critérios de conclusão
A etapa estará concluída quando:

1. As duplicatas tiverem tratamento documentado.
2. As partições estiverem livres de vazamento conhecido.
3. As três categorias de anomalias estiverem implementadas.
4. A distribuição das classes e dos sentimentos estiver validada.
5. As anomalias puderem ser reproduzidas.
6. O dataset estiver pronto para a etapa de tokenização e padding.
7. O notebook executar sem erros até o final da Etapa 03.

Restrições

* Não implementar LSTM.
* Não treinar modelos.
* Não implementar LIME.
* Não criar interface Gradio.
* Não implementar tokenização definitiva ou embeddings.
* Não alterar o enunciado ou resultados aprovados.
* Evitar funções extensas e abstrações desnecessárias.
* Não modificar as partições sem registrar a justificativa.
* Não apresentar as anomalias sintéticas como fraudes comprovadas.

Atualizar a documentação de uso de IA com o prompt integral desta etapa.
Relatório final obrigatório
Informar:

1. Arquivos alterados.
2. Estratégia de deduplicação.
3. Quantidade de registros por partição e classe.
4. Distribuição dos tipos de anomalia.
5. Exemplos gerados.
6. Resultados das validações.
7. Riscos e limitações encontrados.
8. Recomendações para a próxima etapa.

Não avance para a tokenização definitiva nem para o treinamento. Aguarde aprovação.
```

Etapa 03B — Correções e Validação Final do Dataset

Objetivo. Corrigir a política de separação entre treino e teste (12 avaliações do teste original haviam sido movidas para o treino), validar o dataset antes da Etapa 04 e atualizar a documentação, sem refazer a Etapa 03, sem treinar modelos e sem iniciar a tokenização.

Resumo das alterações.

- Quase-duplicatas que unem treino e teste originais: os registros do teste passam a ser excluídos (12), com tabela de rastreabilidade, em vez de transferidos para o treino; asserções explícitas de que treino e validação só contêm avaliações do treino original.
- Divisão por partição original; estatísticas e interpretações das seções 3.1 a 3.7 atualizadas; nova seção 3.8 com o registro da correção.
- Validações revisadas (21 verificações, com fusão de redundâncias e novas verificações de origem, exclusões, identificadores, rótulos, normalização e isolamento da generalização).
- Amostras qualitativas reproduzíveis por categoria e por faixa de posição, com problemas observados documentados; mecanismos de geração mantidos.

- Confirmação do isolamento da partição de generalização e documentação de suas limitações.
- Enunciado, Fases 1 e 2 e registros anteriores de IA não foram alterados.

Observação sobre os prompts. Foram enviadas duas versões do prompt da Etapa 03B: a primeira, mais detalhada, foi interrompida e substituída pela segunda, mais curta. As duas estão registradas abaixo na íntegra; a segunda foi tratada como instrução principal, e os requisitos compatíveis da primeira foram incorporados.

Prompt 1 (versão interrompida, na íntegra).

```
ETAPA 03B — Correções e Validação Final do Dataset
Contexto
O projeto FIAP Checkpoint 2 utiliza LSTM supervisionada, anomalias sintéticas, LIME e Gradio.
A Etapa 03 foi implementada e auditada. A estrutura geral foi considerada adequada, mas há ajustes necessários antes da Etapa 04.
Não reimplementar o dataset do zero.
Objetivo
Corrigir a política de separação entre treino e teste, fortalecer as validações e preservar a estrutura existente.
Arquivo
`FIAP_2TSCPF2026_Check2_RNN.ipynb`
Atividade 1 — Corrigir separação dos dados
Revisar a seção 3.1 e a lógica de `particao_grupo`.
Requisitos:

* Nenhuma avaliação originária do teste original poderá entrar no treino ou na validação.
* Quando uma quase-duplicata do teste conflitar com um grupo do treino, remover do teste o registro conflitante, preservando o treino.
* Manter a rastreabilidade das exclusões.
* Recalcular as quantidades por partição.
* Preservar a separação por grupos.
* Não introduzir novos vazamentos.

Adicionar uma asserção explícita garantindo que `treino` e `validacao` contenham somente avaliações do treino original.
Atividade 2 — Revisar validações
Adicionar verificações para:

* Integridade das partições após a correção.
* Ausência de sobreposição entre grupos.
* Distribuição de sentimentos e classes.
* Consistência dos identificadores.
* Ausência de textos sintéticos sem padrão inserido.
* Preservação das avaliações originais.

Evitar validações redundantes.
Atividade 3 — Inspeção qualitativa
Exibir amostras reproduzíveis das três categorias de anomalia.
Para cada categoria, apresentar exemplos com inserções em diferentes posições.
Documentar problemas encontrados, caso existam.
Atividade 4 — Revisar o teste de generalização
Confirmar que:

* A partição utiliza apenas avaliações originalmente pertencentes ao teste.
* Os mecanismos inéditos não são utilizados no treinamento.
* O conjunto permanece isolado de decisões de ajuste.
* As limitações dos mecanismos sintéticos estão documentadas.

Atividade 5 — Atualizar documentação
Atualizar as estatísticas e interpretações afetadas pelas correções.
Preservar as Fases 1 e 2, o enunciado original e os registros anteriores de IA.
Adicionar o registro deste prompt corretivo.
Entradas

* Notebook atualizado da Etapa 03.
* Dataset original IMDB.
* Implementação atual de deduplicação e geração sintética.

Saídas esperadas

* Política de divisão corrigida.
* Estatísticas atualizadas.
* Verificações automáticas aprovadas.
* Exemplos qualitativos documentados.
* Notebook válido e executável.

Bibliotecas
Reutilizar pandas, NumPy, scikit-learn e as bibliotecas já presentes.
Não instalar dependências adicionais sem necessidade.
Critérios de conclusão

1. Nenhum registro do teste original entra no treino ou na validação.
2. Nenhum grupo conhecido atravessa partições.
3. As exclusões estão documentadas.
4. Os testes automáticos passam.
5. As amostras qualitativas foram inspecionadas.
```

```
6. O notebook executa sem erros.
7. O enunciado e as fases anteriores permanecem preservados.

Restrições

* Não implementar tokenização definitiva.
* Não treinar modelos.
* Não implementar LIME ou Gradio.
* Não modificar os mecanismos sintéticos sem evidência de problema.
* Não alterar o projeto além do necessário.
* Não inventar resultados.

Relatório final
Informar:

1. Arquivos e seções alterados.
2. Problema corrigido.
3. Quantidade de registros removidos.
4. Novas distribuições por partição.
5. Validações executadas.
6. Resultados da inspeção qualitativa.
7. Pendências restantes.

Parar ao concluir a Etapa 03B e aguardar aprovação.
```

Prompt 2 (versão final, na íntegra).

```
Etapa 03B — Ajustes finais do dataset
Contexto
Projeto FIAP Checkpoint 2 — LSTM supervisionada + anomalias sintéticas + LIME + Gradio.
A Etapa 03 foi concluída, mas a auditoria identificou que 12 avaliações do teste original foram movidas para o treino durante o
agrupamento de quase-duplicatas.
Objetivo
Corrigir esse problema e validar o dataset antes da Etapa 04.
Arquivo
`FIAP_2TSCPF2026_Check2_RNN.ipynb`
Tarefas

1. Corrigir a divisão: preservar o teste original. Quando houver conflito de quase-duplicatas com o treino, excluir os registros
conflitantes do teste, sem transferi-los para o treino.
2. Validar: garantir ausência de vazamento entre treino, validação e teste, preservando os grupos e a rastreabilidade.
3. Conferir anomalias: apresentar amostras das três categorias e verificar a consistência dos textos gerados.
4. Generalização: confirmar que o conjunto permanece isolado e utiliza apenas registros originalmente destinados ao teste.
5. Documentar: atualizar estatísticas, validações e registro de uso de IA.

Bibliotecas
Reutilizar pandas, NumPy e scikit-learn já presentes no notebook.
Critérios de conclusão

* Nenhum registro do teste original utilizado no treinamento ou validação.
* Nenhuma sobreposição de grupos entre partições.
* Anomalias e distribuições consistentes.
* Notebook executando sem erros.
* Fases anteriores preservadas.

Restrições
Não refazer a Etapa 03, não treinar modelos e não iniciar a tokenização.
Relatório final
Informar alterações realizadas, quantidades atualizadas, validações executadas e eventuais pendências.
Pare após concluir a Etapa 03B e aguarde aprovação.
```

Etapa 04 — Pré-processamento e Tokenização

Objetivo. Transformar `texto_modelo` em sequências numéricas para a LSTM, com vocabulário ajustado só com o treino, avaliar vocabulários de 10.000 e 20.000 e comprimentos de 256 e 400 sem usar teste nem generalização na escolha, verificar a preservação dos trechos anômalos após o truncamento e documentar a configuração, sem treinar modelos.

Resumo das alterações.

- Nova seção "Etapa 04" (4.1 a 4.7) após a seção 3.8, com 9 células de código.
- Conferência das contagens da Etapa 03B.
- Padronização com `TextVectorization` (negações, contrações e sinais mantidos; sem remoção de stopwords) e vocabulário ajustado só com o treino.
- Avaliação das quatro configurações com treino e validação, regras de escolha definidas antes dos resultados, e escolha de vocabulário 20.000 e comprimento 256.

- Matrizes `X` e `y` das quatro partições, 12 verificações automáticas (incluindo sonda de vazamento) e demonstração de compatibilidade com LIME e Gradio.
- O dataset supervisionado, o enunciado, as Fases 1 e 2, a Etapa 03 e os registros anteriores de IA não foram alterados.

Prompt utilizado (na íntegra).

```
ETAPA 04 — Pré-processamento e Tokenização
Contexto
Projeto FIAP Checkpoint 2: LSTM supervisionada + anomalias sintéticas + LIME + Gradio.
As Etapas 01, 02, 03 e 03B foram concluídas. O dataset supervisionado está preparado, com partições independentes e anomalias rastreáveis.
Objetivo
Transformar os textos em sequências numéricas para treinamento da LSTM, preservando os padrões de anomalia e evitando vazamento de informações.
Arquivo
`FIAP_2TSCPF2026_Check2_RNN.ipynb`
Atividades

1. Revisar a consistência das contagens e exclusões documentadas na Etapa 03B.
2. Utilizar `texto_modelo` como entrada e `classe` como rótulo.
3. Implementar tokenização e vocabulário ajustados exclusivamente com os dados de treino.
4. Avaliar vocabulários de 10.000 e 20.000 tokens e comprimentos máximos de 256 e 400 posições, sem utilizar teste ou generalização para escolher parâmetros.
5. Aplicar padding, truncamento e tratamento de palavras desconhecidas.
6. Preservar negações e sinais relevantes para a detecção de anomalias.
7. Verificar se os trechos sintéticos permanecem representados após o truncamento.
8. Documentar a configuração escolhida e justificar a decisão.

Bibliotecas
TensorFlow/Keras, NumPy, pandas e bibliotecas já utilizadas.
Preferir `TextVectorization` do Keras, caso seja compatível com os requisitos de explicabilidade futura.
Validações

* Vocabulário ajustado somente com treino.
* Sequências com dimensões consistentes.
* Rótulos preservados.
* Nenhuma partição misturada.
* Taxa de tokens desconhecidos documentada.
* Impacto do truncamento medido.
* Trechos anômalos preservados ou perdas explicitamente identificadas.
* Compatibilidade com futura integração LIME e Gradio.

Critérios de conclusão
Dados de treino, validação, teste e generalização preparados para a LSTM, com pré-processamento reproduzível e documentado.
Restrições

* Não treinar modelos.
* Não implementar LSTM, LIME ou Gradio.
* Não alterar o dataset supervisionado sem justificativa.
* Preservar o enunciado e as fases anteriores.
* Atualizar o registro de IA com este prompt.

Relatório final
Informar configurações testadas, escolha final, dimensões dos dados, cobertura do vocabulário, impacto do truncamento, validações e pendências.
Parar após a Etapa 04 e aguardar aprovação.
```

Etapa 05 — Construção e Treinamento da LSTM

Objetivo. Construir e treinar uma primeira LSTM supervisionada (Embedding com máscara, LSTM, Dropout, Dense com sigmoid, *Binary Crossentropy*) usando só `X["treino"]` / `y["treino"]`, com a validação para acompanhamento e *early stopping*, comparar com baselines, verificar *overfitting* e documentar arquitetura, hiperparâmetros, tempo e limitações, sem usar teste nem generalização.

Resumo das alterações.

- Nova seção "Etapa 05" (5.1 a 5.8) após a seção 4.7, com 9 células de código.
- Baselines (classe mais frequente e regressão logística sobre TF-IDF) ajustados só com o treino.
- Arquitetura LSTM, verificação de reprodutibilidade e três tentativas de treinamento (a inicial instável e duas correções), com escolha por regra fixa (menor perda de validação).

- Curvas de perda e acurácia, comparação com os baselines, análise por tipo e variante de anomalia, verificação de *overfitting* e 10 verificações automáticas.
- O dataset, a tokenização, as etapas anteriores e os registros anteriores de IA não foram alterados.

Prompt utilizado (na íntegra).

```
Etapa 05 — Construção e Treinamento da LSTM
Contexto
Projeto FIAP Checkpoint 2 — detecção de anomalias sintéticas em avaliações IMDB.
As Etapas 01 a 04 foram aprovadas. O dataset e a tokenização estão preparados, com vocabulário de 20.000 tokens e sequências de 256 posições.
Objetivo
Construir e treinar uma primeira LSTM supervisionada, avaliando o aprendizado e possíveis sinais de overfitting.
Arquivo
`FIAP_2TSCPF2026_Check2_RNN.ipynb`
Atividades

1. Criar um baseline simples para comparação, utilizando apenas treino e validação.
2. Implementar uma arquitetura inicial com Embedding, LSTM, Dropout e Dense com sigmoid.
3. Utilizar máscara para padding e Binary Crossentropy como função de perda.
4. Treinar exclusivamente com `X["treino"]` e `y["treino"]`.
5. Utilizar a validação para acompanhar o desempenho e aplicar Early Stopping.
6. Apresentar curvas de loss e accuracy para treino e validação.
7. Comparar a LSTM com o baseline.
8. Documentar arquitetura, hiperparâmetros, tempo de treinamento e limitações.

Bibliotecas
TensorFlow/Keras, scikit-learn, NumPy, pandas e Matplotlib.
Validações

* Modelo recebe entradas `(n, 256)`.
* Saída representa probabilidade da classe 1.
* Treinamento reproduzível na medida permitida pelo ambiente.
* Ausência de uso do teste e da generalização para treinamento ou seleção.
* Curvas e métricas calculadas corretamente.
* Verificação de overfitting e comparação com baseline.

Critérios de conclusão
LSTM treinada, histórico registrado, comparação com baseline e análise crítica do aprendizado.
Restrições

* Não implementar LIME ou Gradio.
* Não utilizar teste ou generalização.
* Não alterar o dataset nem a tokenização sem justificar.
* Não realizar busca extensa de hiperparâmetros.
* Preservar as etapas anteriores.
* Atualizar o registro de uso de IA.

Relatório final
Informar arquitetura, parâmetros, épocas, tempo de treinamento, métricas de treino e validação, comparação com baseline e problemas encontrados.
Pare ao concluir a Etapa 05 e aguarde aprovação.
```

Prompt complementar (priorização do prazo e persistência do modelo), na íntegra.

```
Estamos com menos de 2 horas para entregar o Checkpoint 2 da FIAP.
Priorize concluir a Etapa 05 com os resultados já obtidos.

1. Aguarde a execução atual e confirme que o notebook terminou sem erros.
2. Não realize novos treinamentos nem buscas de hiperparâmetros.
3. Preserve o melhor modelo selecionado pela menor `val_loss`.
4. Registre as métricas reais, o baseline, as curvas e a limitação de recall em repetição.
5. Salve o modelo e garanta que poderá ser reutilizado nas próximas etapas.
6. Mantenha o enunciado original e as etapas anteriores intactos.
7. Atualize a documentação de IA.

Ao concluir, informe apenas: status da execução, modelo selecionado, métricas, artefatos salvos e pendências.
Não avance automaticamente para a Etapa 06.
```

Alterações decorrentes do prompt complementar. Nenhum novo treinamento ou busca de hiperparâmetros foi realizado; o modelo escolhido pela menor `val_loss` foi salvo a partir da sessão que executou o notebook, junto com vocabulário, função de padronização, sequências e métricas (seção 5.9), e foi verificado em um processo novo. A limitação de recall em repetição excessiva está registrada nas seções 5.5, 5.6 e 5.8. O enunciado e as etapas anteriores permanecem intactos.

Etapa 06 — Avaliação final da LSTM

Objetivo. Avaliar, sem retreinar, a LSTM salva na Etapa 05 nos conjuntos de teste e de generalização (accuracy, precision, recall, F1, ROC AUC e matriz de confusão), apresentar o recall por tipo e por mecanismo inédito, analisar falsos positivos e falsos negativos (principalmente repetição), comparar com a validação em busca de memorização de templates e documentar as limitações.

Resumo das alterações.

- Nova seção "Etapa 06" (6.1 a 6.8) após a seção 5.9, com 7 células de código.
- Recarga do modelo e dos artefatos, métricas e matrizes de confusão por partição, recall por tipo e por mecanismo (com intervalos de confiança), análise de repetição e de falsos positivos, comparação com a validação e 9 verificações automáticas.
- Como a execução completa do notebook retreinaria a LSTM, apenas as células de dados (Fases 2 e 3) e as desta etapa foram executadas; as saídas das etapas anteriores permanecem as originais.
- O modelo, o dataset, a tokenização e as etapas anteriores não foram alterados.

Prompt utilizado (na íntegra).

```
Etapa 06 — Avaliação final da LSTM
Estamos próximos do prazo de entrega do Checkpoint 2 FIAP. Priorize rapidez, reprodutibilidade e resultados verificáveis.
Objetivo: avaliar o modelo LSTM já treinado nos conjuntos reservados de teste e generalização, sem retreinamento.
Arquivo: `FIAP_2TSCPF2026_Check2_RNN.ipynb`
Atividades:

1. Recarregar `artefatos/lstm_melhor_val_loss.keras` e os artefatos da Etapa 05, sem executar novamente o treinamento.
2. Avaliar teste e generalização separadamente, calculando accuracy, precision, recall, F1, ROC-AUC e matriz de confusão.
3. Apresentar recall por tipo de anomalia e por mecanismo inédito.
4. Analisar exemplos de falsos positivos e falsos negativos, principalmente casos de repetição.
5. Comparar os resultados com a validação, identificando possíveis sinais de memorização de templates ou queda de generalização.
6. Documentar as limitações e registrar este prompt na seção de uso de IA.

Validações: conferir alinhamento dos rótulos, ausência de vazamento, modelo correto e métricas reproduzíveis.
Restrições: não retreinar, não alterar dataset ou tokenização, não implementar LIME ou Gradio e preservar as etapas anteriores.
Relatório final: métricas de teste e generalização, desempenho por categoria, erros relevantes, arquivos alterados e pendências.
Pare ao concluir a Etapa 06 e aguarde aprovação.
```

Etapas 07 e 08 — LIME e Gradio

Objetivo. Explicar com LIME, sem retreinar, as decisões do modelo LSTM salvo (acertos, falso positivo e falsos negativos) e criar uma interface Gradio com texto, botão, classificação, probabilidade da classe 1, explicação visual e aviso de limitações, reutilizando o mesmo modelo e pré-processamento.

Resumo das alterações.

- Nova seção "Etapas 07 e 08" (7.1 a 8.4) após a seção 6.8, com 7 células de código.
- `predict_proba` compatível com o LIME, cinco explicações (acerto de anomalia, acerto normal, falso positivo e dois falsos negativos) com gráfico e texto destacado, e 6 verificações do LIME.
- Função `analisar`, interface Gradio iniciada no notebook e 6 testes pelo cliente do Gradio (entradas normal, suspeita, vazia e longa, e comparação das probabilidades com a inferência direta).
- Instalação de `lime` e `gradio` (exigidos pelas etapas). Como a execução completa do notebook retreinaria a LSTM, só foram executadas as células de dados (Fases 2 e 3) e as novas; as saídas das etapas anteriores permanecem as originais.
- O modelo, o dataset, a tokenização e as Etapas 01 a 06 não foram alterados.

Prompt utilizado (na íntegra).

```
Etapas 07 e 08 — LIME + Gradio | Execução prioritária
Temos aproximadamente 1 hora para finalizar o Checkpoint 2 FIAP.
A Etapa 06 foi aprovada. O modelo LSTM já está treinado e salvo em `artefatos/`. Não realizar novos treinamentos.
Arquivo principal: `FIAP_2TSCPF2026_Check2_RNN.ipynb`
Etapa 07 — Explicabilidade LIME
```

```
1. Reutilizar o modelo salvo e a função de pré-processamento existente.
2. Implementar `predict_proba(textos)` compatível com o LIME, retornando probabilidades das duas classes.
3. Gerar explicações para exemplos de classificação correta, falso positivo e falso negativo.
4. Visualizar as palavras que favorecem cada classe.
5. Documentar que LIME fornece explicações locais aproximadas e que o modelo apresenta baixa generalização para mecanismos inéditos.

Etapa 08 — Interface Gradio
Criar uma interface simples e funcional com:

* Campo para inserir uma avaliação.
* Botão para analisar.
* Classificação: normal ou anomalia sintética.
* Probabilidade da classe 1.
* Explicação visual das palavras usando LIME.
* Aviso sobre o limite de 256 tokens e as limitações do modelo.

Reutilizar exatamente o mesmo modelo e pré-processamento do notebook.
Validações

* Testar entradas normais, suspeitas, vazias e longas.
* Confirmar que as probabilidades do Gradio são iguais às da inferência direta.
* Confirmar que o LIME recebe e retorna probabilidades válidas.
* Garantir que a interface seja iniciada sem erros.

Restrições

* Não retreinar.
* Não alterar as Etapas 01 a 06.
* Não modificar dataset, tokenização ou arquitetura.
* Não executar novamente o notebook inteiro.
* Evitar dependências e funcionalidades desnecessárias.
* Registrar o prompt na documentação de IA.

Relatório final
Informar arquivos alterados, funcionamento do LIME, testes do Gradio, problemas encontrados e pendências.
Prioridade absoluta: entregar LIME e Gradio funcionais. Pare após concluir as duas etapas.
```

Etapa 09 — Finalização do Checkpoint 2

Objetivo. Preparar a entrega final sem alterar o modelo nem refazer os experimentos: conferir os requisitos do enunciado e as fases do CRISP-DM, revisar o notebook, garantir os artefatos, criar o README, registrar as dependências, testar o carregamento do modelo e o Gradio em processo independente e executar verificações de integridade.

Resumo das alterações.

- Nova seção "Resumo final e checklist de entrega", com os resultados finais, o mapeamento dos requisitos e critérios do enunciado, os desvios e lacunas conhecidos e os campos a preencher.
- Novos arquivos fora do notebook: `README.md`, `requirements.txt` e `app_gradio.py` (interface independente que reutiliza o mesmo modelo, pré-processamento e LIME do notebook, com a opção `--verificar`).
- Testes em processo independente: o modelo carregado reproduziu a acurácia de validação registrada, as probabilidades do aplicativo coincidiram com as do notebook em cinco textos, e a interface iniciou, classificou e explicou uma avaliação.
- Nenhum treinamento, dataset, métrica, gráfico ou resultado das etapas anteriores foi alterado, e o enunciado do professor foi preservado.

Prompt utilizado (na íntegra).

```
Etapa 09 — Finalização do Checkpoint 2 FIAP
As Etapas 01 a 08 estão concluídas. Estamos próximos do prazo de entrega.
Objetivo: preparar a entrega final sem alterar o modelo nem refazer os experimentos.
Arquivo principal: `FIAP_2TSCPF2026_Check2_RNN.ipynb`
Tarefas:

1. Conferir se todos os requisitos do enunciado original e as seis fases CRISP-DM estão contemplados.
2. Revisar a organização do notebook, os títulos, as conclusões e a documentação de uso de IA.
3. Garantir que os artefatos necessários para carregar a LSTM, executar o LIME e iniciar o Gradio estejam disponíveis.
4. Criar ou atualizar um README curto com instruções de instalação, execução e demonstração.
5. Registrar as dependências utilizadas e testar o carregamento do modelo em um processo independente, sem retreinar.
6. Verificar se o Gradio inicia e se uma avaliação pode ser classificada e explicada.
7. Identificar os campos de nome e matrícula que precisam ser preenchidos manualmente.
```

8. Executar verificações finais de integridade do notebook e dos arquivos.

Restrições: não retreinar, não alterar dataset, métricas, gráficos ou resultados das etapas anteriores. Não modificar o enunciado do professor. Não publicar a aplicação.
Relatório final: informar os arquivos de entrega, os testes realizados, os requisitos atendidos e qualquer pendência.
Pare após concluir a revisão. Não faça novas implementações sem autorização.